

VĂN THỊ MINH HUYỀN

KHOA HỌC MÁY TÍNH

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



ĐỒ ÁN TỐT NGHIỆP

HỌC TĂNG CƯỜNG ĐỂ TỐI ƯU HÓA
KHẢ NĂNG TƯƠNG TÁC CỦA AI TRONG GAME

Sinh viên: Văn Thị Minh Huyền

Mã số sinh viên: 22010329

Khóa: K16

Ngành: Khoa học máy tính (Tài năng)

Hệ: Đại học chính quy

Giảng viên hướng dẫn: Th.S. Nguyễn Văn Sơn

HÀ NỘI, THÁNG 8 NĂM 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA

ĐỒ ÁN TỐT NGHIỆP

HỌC TĂNG CƯỜNG ĐỂ TỐI ƯU HÓA
KHẢ NĂNG TƯƠNG TÁC CỦA AI TRONG GAME

Họ và tên: Văn Thị Minh Huyền

Mã sinh viên: 22010329

Khóa: K16

Ngành: Khoa học máy tính (Tài năng) **Hệ:** Đại học chính quy

Giảng viên hướng dẫn: Th.S. Nguyễn Văn Sơn

HÀ NỘI, THÁNG 8 NĂM 2026

LỜI CAM ĐOAN

Tên tôi là: Văn Thị Minh Huyền

Mã sinh viên: 22010329

Ngành: Khoa học máy tính

Tôi đã thực hiện đồ án tốt nghiệp với đề tài: Học tăng cường để tối ưu hóa khả năng tương tác của AI trong game

Tôi xin cam đoan đây là đề tài nghiên cứu của riêng tôi và được sự hướng dẫn của Th.S Nguyễn Văn Sơn.

Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa được các tác giả khác công bố dưới bất kỳ hình thức nào. Nếu phát hiện có bất kỳ hình thức gian lận nào tôi xin hoàn toàn chịu trách nhiệm trước pháp luật.

GIẢNG VIÊN HƯỚNG DẪN

Hà Nội, ngày ... tháng ... năm 20...

(Ký, ghi rõ họ tên)

SINH VIÊN

(Ký, ghi rõ họ tên)

LỜI CẢM ƠN

Lời đầu tiên, em xin bày tỏ lòng biết ơn sâu sắc đến quý thầy cô Trường Công nghệ thông tin Phenikaa đã tận tình giảng dạy, truyền đạt kiến thức và tạo mọi điều kiện thuận lợi cho em trong suốt quá trình học tập tại trường.

Em xin gửi lời cảm ơn chân thành đến thầy Nguyễn Văn Sơn - giảng viên hướng dẫn đồ án tốt nghiệp, đã dành nhiều thời gian, công sức định hướng, góp ý và hỗ trợ em trong từng giai đoạn thực hiện đề tài. Những chỉ dẫn tận tâm của thầy là nền tảng quan trọng giúp em hoàn thành đồ án này.

Cuối cùng, em xin bày tỏ lòng biết ơn sâu sắc đến gia đình - những người đã luôn đồng hành, động viên và tạo điều kiện tốt nhất để em có thể toàn tâm hoàn thành chương trình học.

Do kiến thức và kinh nghiệm còn hạn chế, đồ án không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của quý thầy cô và các bạn để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

TÓM TẮT ĐỒ ÁN

Đồ án nghiên cứu việc ứng dụng học tăng cường (Reinforcement Learning - RL) để tối ưu hóa khả năng tương tác của các tác nhân trí tuệ nhân tạo trong môi trường game ba chiều. Trọng tâm của đề tài là xây dựng ba môi trường game 3D đặc trưng trên nền tảng Unity ML-Agents - Football Table (đối kháng một-một), môi trường Capture The Flag (hợp tác đa tác nhân) và Cross The Road (điều hướng tránh phương tiện) - rồi tiến hành huấn luyện và so sánh có hệ thống bốn thuật toán tiêu biểu: PPO (Proximal Policy Optimization), SAC (Soft Actor-Critic), MA-POCA (Multi-Agent POsthumous Credit Assignment) và MAPPO (Multi-Agent Proximal Policy Optimization).

Mỗi thuật toán được huấn luyện trên từng môi trường với cùng điều kiện phần cứng và được đánh giá theo bốn tiêu chí định lượng: tốc độ hội tụ, tỷ lệ hoàn thành nhiệm vụ, tổng phần thưởng trung bình và độ ổn định huấn luyện. Đề tài cũng trình bày quy trình tích hợp thuật toán off-policy (SAC) ra ngoài trainer tích hợp sẵn của ML-Agents thông qua Python Low-Level API, nhằm khắc phục giới hạn của ML-Agents khi SAC không hỗ trợ cơ chế self-play và group reward.

Kết quả thực nghiệm cho thấy không tồn tại một thuật toán tối ưu duy nhất cho mọi bài toán: trong môi trường đối kháng, MA-POCA và PPO kết hợp self-play là hai thuật toán duy nhất cho thấy tiến bộ thực sự theo chỉ số xếp hạng ELO, SAC đạt hiệu quả mẫu vượt trội trong bài toán điều hướng đơn tác nhân, còn trong bài toán hợp tác đa tác nhân, MAPPO vượt qua MA-POCA nhờ cơ chế phê bình tập trung kết hợp chia sẻ tham số giữa các tác nhân đồng nhất. Trên cơ sở đó, đề tài xây dựng một ứng dụng trình diễn tương tác hoàn chỉnh: các mô hình đã huấn luyện được triển khai trực tiếp vào game dưới định dạng ONNX, người dùng có thể lựa chọn môi trường, chế độ chơi (tự điều khiển, tác nhân tự chơi, chơi cùng tác nhân, tác nhân đầu mô hình ngôn ngữ lớn), mô hình cho tác nhân và tra cứu ngay biểu đồ huấn luyện của từng thuật toán trên giao diện menu. Kết luận thực tiễn rút ra là việc chọn thuật toán RL nên xuất phát từ cơ chế của chính trò chơi - đối kháng, hợp tác hay điều hướng - chứ không từ thứ hạng của thuật toán trên các bộ benchmark chuẩn.

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	xi
1 MỞ ĐẦU	1
1.1 Lý do chọn đề tài	1
1.2 Mục tiêu nghiên cứu	2
1.3 Đối tượng và phạm vi nghiên cứu	3
1.4 Tổng quan tài liệu và khoảng trống nghiên cứu	4
1.5 Phương pháp nghiên cứu	5
1.6 Ý nghĩa khoa học và thực tiễn	6
1.7 Bố cục đồ án	6
2 CƠ SỞ LÝ THUYẾT	7
2.1 Trí tuệ nhân tạo trong game	7
2.1.1 AI truyền thống	8
2.1.2 AI học máy	8
2.2 Nền tảng Reinforcement Learning	9
2.2.1 Các khái niệm cơ bản	9
2.2.2 Markov Decision Process	11
2.2.3 Policy và Value Function	11
2.3 Học tăng cường sâu	12
2.3.1 Vai trò của mạng Neural Network trong RL	13
2.3.2 Phân loại: Policy-Based, Value-Based, Actor-Critic	13
2.3.3 Phần thưởng tò mò nội tại	15
2.4 Các thuật toán sử dụng trong đề tài	16
2.4.1 PPO	17
2.4.2 SAC	18
2.4.3 MA-POCA	19
2.4.4 MAPPO	21
2.4.5 Tổng kết và so sánh bốn thuật toán	23
2.5 Unity ML-Agents Toolkit	24
2.5.1 Kiến trúc tổng thể	24

2.5.2	Cơ chế Vật lý và Tính xác định trong Unity	25
2.5.3	Hệ thống Cảm biến và Tiền xử lý quan sát	26
2.5.4	Cơ chế Hành vi và Gom nhóm quyết định	26
2.5.5	Quy trình huấn luyện Agent	27
2.5.6	Hỗ trợ thuật toán và giới hạn tích hợp sẵn	27
3	THIẾT KẾ MÔI TRƯỜNG VÀ PHƯƠNG PHÁP THỰC NGHIỆM	28
3.1	Tổng quan thiết kế hệ thống	29
3.1.1	Kiến trúc huấn luyện tổng thể	29
3.1.2	Không gian quan sát	29
3.1.3	Không gian hành động	32
3.1.4	Nguyên tắc thiết kế hàm phần thưởng	33
3.2	Bộ môi trường thực nghiệm	36
3.2.1	Tích hợp với Unity ML-Agents	37
3.3	Môi trường Football Table	39
3.3.1	Mô tả bài toán	39
3.3.2	Không gian quan sát	40
3.3.3	Không gian hành động và cơ chế điều khiển	41
3.3.4	Hàm phần thưởng	41
3.3.5	Điều kiện kết thúc lượt chơi	44
3.3.6	Cấu hình huấn luyện	44
3.4	Môi trường Capture The Flag	44
3.4.1	Mô tả bài toán	44
3.4.2	Không gian quan sát	45
3.4.3	Không gian hành động và cơ chế điều khiển	46
3.4.4	Cơ chế cửa và pha bàn giao	47
3.4.5	Hàm phần thưởng	47
3.4.6	Quá trình hiệu chỉnh hàm phần thưởng	51
3.4.7	Điều kiện kết thúc lượt chơi	53
3.4.8	Cấu hình huấn luyện và học theo chương trình	53
3.5	Môi trường Cross The Road	54
3.5.1	Mô tả bài toán	54

3.5.2	Không gian quan sát	54
3.5.3	Không gian hành động và cơ chế điều khiển	55
3.5.4	Hàm phần thưởng	55
3.5.5	Điều kiện kết thúc lượt chơi	57
3.5.6	Cấu hình huấn luyện	57
3.6	Bảng tổng hợp đặc trưng ba môi trường	58
3.7	Triển khai thuật toán ngoài ML-Agents	59
3.7.1	Vấn đề với SAC trong ML-Agents	59
3.7.2	Cách tích hợp thuật toán tùy chỉnh qua Python API	60
3.7.3	Cấu trúc mã nguồn bổ sung	60
3.8	Thiết lập thực nghiệm	61
3.8.1	Cấu hình phần cứng và phần mềm	61
3.8.2	Tham số huấn luyện từng thuật toán	62
3.8.3	Tiêu chí đánh giá	62
4	KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	63
4.1	Kết quả môi trường Football Table	64
4.2	Kết quả môi trường Cross The Road	67
4.3	Kết quả môi trường Capture The Flag	70
4.4	Đánh giá tổng hợp	73
4.4.1	So sánh tốc độ hội tụ	73
4.4.2	So sánh mức năng lực đạt được	73
4.4.3	So sánh độ ổn định	74
4.4.4	Bảng chứng từ chỉ số huấn luyện nội tại	75
4.4.5	Phân tích ưu nhược điểm từng thuật toán	76
4.5	Thảo luận	77
4.5.1	Khả năng ứng dụng trong phát triển game	77
5	XÂY DỰNG ỨNG DỤNG TRÌNH DIỄN TƯƠNG TÁC	77
5.1	Mục tiêu và yêu cầu chức năng	78
5.2	Kiến trúc hệ thống	79
5.3	Giao diện menu và luồng sử dụng	81

5.4	Nạp mô hình ONNX tại thời gian chạy	83
5.5	Trực quan hóa kết quả huấn luyện trong ứng dụng	85
5.5.1	Đường ống dữ liệu từ TensorBoard vào game	85
5.5.2	Tab THÔNG SỐ	85
5.5.3	Tab SO SÁNH	87
5.6	Bốn chế độ chơi trong từng môi trường	88
5.6.1	Cross The Road: trải nghiệm và quan sát chính sách	88
5.6.2	Capture The Flag: phối hợp người - máy	89
5.6.3	Football Table: đối đầu người - máy và máy - máy	90
5.7	Tích hợp mô hình ngôn ngữ lớn	91
5.8	Các vấn đề kỹ thuật và giải pháp	93
5.9	Kiểm thử ứng dụng và hạn chế	95
6	KẾT LUẬN	97
6.1	Kết quả đạt được	97
6.2	Đóng góp của đề tài	98
6.3	Hạn chế	98
6.4	Hướng phát triển	99
PHỤ LỤC		xvii
Phụ lục A.	Cấu hình huấn luyện PPO	xvii
Phụ lục B.	Cấu hình huấn luyện SAC	xvii
Phụ lục C.	Cấu hình huấn luyện MA-POCA	xviii
Phụ lục D.	Mã nguồn minh họa	xix
Phụ lục E.	Kết quả TensorBoard	xx

DANH SÁCH HÌNH VẼ

2.1	Vòng lặp tương tác giữa agent và môi trường trong học tăng cường	10
2.2	Sơ đồ kiến trúc Intrinsic Curiosity Module (ICM)	16
2.3	So sánh cấu trúc giữa MAPPO và MA-POCA	22
2.4	Kiến trúc tổng thể của Unity ML-Agents Toolkit	25
3.1	Ba môi trường thực nghiệm. Trên trái: Football Table, hai đội Xanh - Đỏ mỗi đội bốn thanh gậy. Trên phải: Capture The Flag, hai tác nhân phối hợp giữ cửa để cùng tới đích. Dưới: Cross The Road, một tác nhân băng qua các làn xe đang chạy	38
3.2	Sơ đồ bố cục sân và cơ chế điều khiển thanh gậy trong Football Table	40
3.3	Sơ đồ bố cục không gian và trình tự bàn giao trong Capture The Flag	45
4.1	Tiến trình chỉ số xếp hạng ELO trong cơ chế tự chơi trên Football Table	65
4.2	Đường cong phần thưởng tích lũy trên môi trường Football Table	65
4.3	Đường cong phần thưởng trên môi trường Cross The Road	67
4.4	Độ dài lượt chơi trên môi trường Cross The Road	68
4.5	Diễn biến entropy của chính sách trên môi trường Cross The Road	69
4.6	Hàm mất mát của mạng phê bình trên môi trường Cross The Road	70
4.7	Đường cong phần thưởng trên môi trường Capture The Flag	71
4.8	Độ dài lượt chơi trên môi trường Capture The Flag: MAPPO giải xong câu đố ở giai đoạn cuối, MA-POCA luôn để hết giờ	72
4.9	So sánh phần thưởng trung bình mười phần trăm cuối của bốn thuật toán trên ba môi trường (giá trị 0 biểu thị thuật toán không được đánh giá do không tương thích với cơ chế của môi trường)	74
5.1	Luồng hoạt động của ứng dụng trình diễn	81
5.2	Tab CHƠI của menu chính: khung xem trước ba chiều (trái) và các lựa chọn chế độ chơi (phải)	82
5.3	Khu vực chọn mô hình khi Football Table ở chế độ Agent tự chơi: hai ô chọn riêng cho đội Xanh và đội Đỏ	83
5.4	Tab THÔNG SỐ: đường cong phần thưởng tích lũy và bảng thống kê của bốn thuật toán trên môi trường Cross The Road	86

5.5	Tab SO SÁNH: biểu đồ cột theo từng môi trường và bảng xếp hạng thuật toán	87
5.6	Chế độ Agent tự chơi trong Cross The Road: mô hình ctr_mappo_v1 điều khiển tác nhân, chỉ số hiển thị trực tiếp trong cảnh	89
5.7	Chế độ Chơi với Agent trong Capture The Flag: người chơi phối hợp cùng tác nhân đồng đội dùng mô hình ctf_mappo_v2	90
5.8	Chế độ Người chơi trong Football Table: người chơi cầm đội Xanh, góc trên trái hiển thị thanh gạt đang được chọn	91
5.9	Chế độ Agent đấu LLM: ô chọn mô hình RL và ô chọn LLM đối thủ; nút bắt đầu bị khóa kèm cảnh báo khi chưa cấu hình khóa API	93

DANH SÁCH BẢNG

2.1	Phân loại các thuật toán sử dụng trong đề tài	14
2.2	So sánh các thuật toán học tăng cường	23
3.1	Tốc độ huấn luyện thực đo trên ba môi trường, mỗi môi trường lấy một lần chạy đại diện; số bước là số bước huấn luyện của hành vi tương ứng	39
3.2	Hàm phần thưởng của môi trường Football Table	43
3.3	Hàm phần thưởng của môi trường Capture The Flag	50
3.4	Hàm phần thưởng của môi trường Cross The Road	57
3.5	Tổng hợp đặc trưng ba môi trường thực nghiệm	59
3.6	Cấu hình phần cứng và phần mềm thực nghiệm	61
3.7	Tóm tắt siêu tham số huấn luyện chính	62
4.1	Kết quả ba thuật toán trên môi trường Football Table	66
4.2	Kết quả bốn thuật toán trên môi trường Cross The Road	67
4.3	Kết quả hai thuật toán trên môi trường Capture The Flag	71
4.4	Chỉ số huấn luyện nội tại (các môi trường hành động rời rạc)	75
4.5	Xếp hạng thuật toán theo từng môi trường	76
5.1	Các thành phần chính của ứng dụng trình diễn	79
5.2	Các mô hình ONNX được đóng gói sẵn trong ứng dụng	84
5.3	Ánh xạ bốn chế độ chơi vào ba môi trường	88
5.4	Kết quả kiểm thử các luồng chức năng của ứng dụng	96
6.1	Tóm tắt chỉ số cuối thực đo từ TensorBoard	xx

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Diễn giải
AI	Artificial Intelligence	Trí tuệ nhân tạo
RL	Reinforcement Learning	Học tăng cường
Deep RL	Deep Reinforcement Learning	Học tăng cường sâu
MDP	Markov Decision Process	Quá trình quyết định Markov
ONNX	Open Neural Network Exchange	Định dạng trao đổi mô hình mạng neural mở
PPO	Proximal Policy Optimization	Thuật toán tối ưu chính sách vùng lân cận
SAC	Soft Actor-Critic	Thuật toán actor-critic cực đại hóa entropy
MA-POCA	Multi-Agent POsthumous Credit Assignment	Thuật toán gán công lao hậu định cho đa tác nhân
MAPPO	Multi-Agent Proximal Policy Optimization	Tối ưu hóa chính sách cận biên đa tác nhân
TRPO	Trust Region Policy Optimization	Tối ưu chính sách theo vùng tin cậy
DQN	Deep Q-Network	Mạng Q sâu
GAE	Generalized Advantage Estimation	Ước lượng advantage tổng quát
FSM	Finite State Machine	Máy trạng thái hữu hạn
GOAP	Goal-Oriented Action Planning	Lập kế hoạch hành động hướng mục tiêu
NPC	Non-Player Character	Nhân vật phi người chơi
MLP	Multi-Layer Perceptron	Mạng perceptron nhiều lớp
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
CTDE	Centralized Training Decentralized Execution	Huấn luyện tập trung, thực thi phân tán
ELO	Elo Rating System	Hệ thống xếp hạng Elo
API	Application Programming Interface	Giao diện lập trình ứng dụng

Từ viết tắt	Tiếng Anh	Diễn giải
gRPC	google Remote Procedure Call	Giao thức gọi thủ tục từ xa của Google
YAML	YAML Ain't Markup Language	Định dạng tệp cấu hình
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
SB3	Stable-Baselines3	Thư viện hiện thực các thuật toán RL

1. MỞ ĐẦU

1.1 Lý do chọn đề tài

Trò chơi điện tử giữ một vị trí khác thường trong nghiên cứu trí tuệ nhân tạo: chúng đồng thời là phòng thí nghiệm và là sản phẩm. Một môi trường game cho phép chạy hàng triệu lần thử với chi phí gần bằng không và không có rủi ro nào, điều mà robot thật hay hệ thống giao thông thật không bao giờ cho phép. Ở chiều ngược lại, chính ngành công nghiệp game có nhu cầu thương mại trực tiếp với những nhân vật máy biết cư xử thuyết phục. Hiếm lĩnh vực nào mà quãng đường từ một kết quả nghiên cứu tới một tính năng bán được lại ngắn đến thế.

Học tăng cường là hướng tiếp cận ăn khớp nhất với bối cảnh này. Nó không đòi hỏi tập dữ liệu gán nhãn sẵn, thứ gần như không thể có cho câu hỏi “nước đi nào là đúng trong tình huống này”, mà để tác nhân tự rút ra chiến lược từ hậu quả của chính những hành động nó đã thử.

Khả năng của học tăng cường đã được chứng minh qua hàng loạt thành tựu mang tính bước ngoặt. Một hệ thống học tăng cường có thể chơi thành thạo hàng chục trò chơi điện tử cổ điển chỉ từ hình ảnh thô của màn hình [14], có thể đánh bại kỳ thủ cờ vây hàng đầu thế giới [23], hay phối hợp nhiều tác nhân để chiến thắng trong những trò chơi chiến thuật phức tạp [16, 26]. Điểm chung của các hệ thống này là không một hành vi nào trong số đó được lập trình sẵn: chúng đều nảy sinh từ quá trình thử và sai của chính tác nhân.

Tuy nhiên, một trong những rào cản lớn nhất khi nghiên cứu và ứng dụng học tăng cường chính là việc xây dựng môi trường huấn luyện phù hợp. Các môi trường hai chiều đơn giản hoặc các bộ dữ liệu chuẩn thường được dùng trong nghiên cứu học thuật tuy thuận tiện nhưng lại không phản ánh được độ phức tạp của trò chơi thực tế. Một môi trường ba chiều với mô phỏng vật lý chân thực, không gian quan sát phong phú và các tình huống thay đổi liên tục mới thực sự thử thách khả năng học của tác nhân, đồng thời gần với bài toán phát triển game thương mại hơn. Việc thiếu vắng những môi trường như vậy, cùng với chi phí xây dựng chúng, khiến cho việc so sánh các thuật toán học tăng cường trong bối cảnh game ba chiều vẫn còn là một khoảng trống đáng chú ý.

Nền tảng Unity kết hợp với bộ công cụ ML-Agents cung cấp một hệ sinh thái đủ mạnh để giải quyết vấn đề trên. Unity là một trong những công cụ phát triển game phổ biến nhất hiện nay, hỗ trợ mô phỏng vật lý và dựng hình ba chiều ở mức chuyên nghiệp; còn ML-Agents nối thẳng một cảnh Unity đang chạy với các thuật toán học tăng cường được cài đặt trong hệ sinh thái Python. Sự kết hợp này biến Unity thành một nền tảng lý tưởng để thiết kế các môi trường huấn luyện đa dạng, từ bài toán đối kháng, hợp tác cho đến điều hướng tránh vật cản trong không gian ba chiều.

Xuất phát từ những lý do trên, tôi lựa chọn đề tài “Học tăng cường để tối ưu hóa khả năng tương tác của AI trong game” với trọng tâm là xây dựng ba môi trường game ba chiều mang tính chất khác nhau trên nền tảng Unity ML-Agents, sau đó huấn luyện và so sánh một cách có hệ thống bốn thuật toán tiêu biểu là PPO, SAC, MA-POCA và MAPPO trên từng loại môi trường. Đích đến của đề tài không phải là tìm ra thuật toán mạnh nhất - kết quả thực nghiệm cho thấy câu hỏi đó không có lời giải - mà là chỉ ra được mỗi thuật toán thắng ở đâu và vì sao, đồng thời để lại một bộ môi trường dùng lại được cho các nghiên cứu và dự án phát triển game về sau.

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là nghiên cứu, xây dựng và đánh giá quy trình ứng dụng học tăng cường để huấn luyện tác nhân trí tuệ nhân tạo trong môi trường game ba chiều trên nền tảng Unity ML-Agents. Từ mục tiêu tổng quát đó, đề tài hướng đến năm mục tiêu cụ thể.

- **Xây dựng bộ môi trường thực nghiệm.** Đề tài xây dựng ba môi trường game ba chiều có tính chất khác biệt để đại diện cho ba dạng bài toán học tăng cường phổ biến: Football Table mô phỏng một bàn bi lắc ba chiều, đại diện cho bài toán đối kháng một đối một với không gian hành động liên tục; Capture The Flag đòi hỏi nhiều tác nhân phối hợp với nhau để cùng hoàn thành một mục tiêu chung; và Cross The Road buộc tác nhân điều hướng vượt qua dòng phương tiện di chuyển liên tục để đến đích.
- **Thiết kế quan sát, hành động và phần thưởng.** Đề tài thiết kế hệ thống quan

sát, hành động và phần thưởng phù hợp với đặc thù của từng môi trường, sao cho tác nhân có đủ thông tin cần thiết để học được hành vi hiệu quả trong không gian ba chiều mà không rơi vào những hành vi bất thường ngoài mong muốn.

- **Huấn luyện và thu thập số liệu.** Đề tài triển khai huấn luyện bốn thuật toán PPO, SAC, MA-POCA và MAPPO trên từng môi trường trong cùng điều kiện phần cứng, đồng thời thu thập và phân tích kết quả theo bốn tiêu chí định lượng là tốc độ hội tụ, tỷ lệ hoàn thành nhiệm vụ, tổng phần thưởng trung bình và độ ổn định của quá trình huấn luyện.
- **So sánh và khuyến nghị lựa chọn thuật toán.** Trên cơ sở các kết quả thu được, đề tài so sánh và phân tích ưu nhược điểm của từng thuật toán ứng với từng loại bài toán, từ đó đưa ra khuyến nghị lựa chọn thuật toán phù hợp cho các cơ chế game khác nhau trong thực tiễn phát triển.
- **Xây dựng ứng dụng trình diễn.** Đề tài xây dựng một ứng dụng trình diễn tương tác hoàn chỉnh, trong đó các mô hình đã huấn luyện (định dạng ONNX) được triển khai trực tiếp vào game. Ứng dụng cung cấp menu lựa chọn môi trường, bốn chế độ chơi (người chơi tự điều khiển, tác nhân tự chơi, người chơi cùng tác nhân, tác nhân đấu mô hình ngôn ngữ lớn), cơ chế lựa chọn mô hình cho tác nhân và hai tab tra cứu số liệu huấn luyện được trích xuất trực tiếp từ nhật ký TensorBoard, qua đó khép kín quy trình từ huấn luyện đến trải nghiệm thực tế trong game.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài gồm ba nhóm chính. Nhóm môi trường bao gồm ba môi trường game ba chiều được xây dựng trên nền tảng Unity, gồm môi trường đối kháng Football Table, môi trường Capture The Flag đa tác nhân và môi trường điều hướng Cross The Road. Nhóm thuật toán bao gồm bốn thuật toán học tăng cường được khảo sát, gồm PPO (thuật toán tối ưu chính sách vùng lân cận), SAC (thuật toán actor-critic cực đại hóa entropy), MA-POCA (thuật toán gán công lao cho đa tác nhân) và MAPPO (mở rộng PPO cho đa tác nhân với chia sẻ tham số).

Nhóm công cụ bao gồm bản thân nền tảng Unity ML-Agents cùng giao diện lập trình cấp thấp bằng Python phục vụ việc tích hợp thuật toán tùy chỉnh.

Về phạm vi nội dung, đề tài tập trung vào thiết kế môi trường ba chiều, xây dựng hệ thống quan sát, hành động, phần thưởng, huấn luyện tác nhân và đánh giá hiệu năng thuật toán. Đề tài không đi sâu vào các khía cạnh tối ưu hóa đồ họa, âm thanh hay hoàn thiện sản phẩm game thương mại. Ba môi trường được lựa chọn nhằm bao phủ ba dạng bài toán học tăng cường điển hình là đối kháng một đối một, hợp tác đa tác nhân và điều hướng đơn tác nhân.

Về phạm vi thuật toán, bốn thuật toán PPO, SAC, MA-POCA và MAPPO được chọn dựa trên mức độ phổ biến trong nghiên cứu, tính sẵn có trong hệ sinh thái ML-Agents và sự đa dạng về mặt kiến trúc, khi chúng đại diện cho bốn hướng tiếp cận khác nhau lần lượt là on-policy đơn tác nhân, off-policy, đa tác nhân dựa trên attention và đa tác nhân dựa trên chia sẻ tham số. Toàn bộ thực nghiệm được tiến hành trên một cấu hình máy tính cố định để bảo đảm tính công bằng khi so sánh, chi tiết cấu hình này được trình bày trong Mục 3.8.1.

1.4 Tổng quan tài liệu và khoảng trống nghiên cứu

Nghiên cứu về học tăng cường trong game đã trải qua nhiều giai đoạn phát triển. Mnih và cộng sự [14] lần đầu chứng minh rằng học tăng cường sâu có thể đạt trình độ con người trong các trò chơi điện tử cổ điển chỉ từ dữ liệu điểm ảnh thô, mở ra hướng ứng dụng học tăng cường trực tiếp vào môi trường game. Tiếp nối thành tựu đó, Silver và cộng sự [23, 24] đạt bước đột phá với các hệ thống chơi cờ vây ở trình độ kiện tướng thế giới, khi kết hợp học tăng cường với mạng nơ-ron sâu và tìm kiếm cây.

Ở quy mô lớn hơn với môi trường phức tạp và nhiều tác nhân, các hệ thống chơi trò chơi chiến thuật thời gian thực [16, 26] đã cho thấy học tăng cường có thể xử lý những bài toán với không gian trạng thái khổng lồ, nhiều tác nhân hoạt động đồng thời và đòi hỏi chiến lược dài hạn. Về phía thuật toán, Schulman và cộng sự [22] đề xuất PPO, một phương pháp on-policy chấp nhận đánh đổi hiệu quả mẫu để lấy sự ổn định và tính dễ triển khai, trong khi Haarnoja và cộng sự [7] giới thiệu SAC với cơ chế điều hòa entropy giúp cân bằng giữa khám phá và khai thác, đặc biệt

hiệu quả với không gian hành động liên tục. Trong lĩnh vực đa tác nhân, Cohen và cộng sự [4] phát triển MA-POCA với cơ chế phê bình tập trung nhằm giải quyết bài toán gán công lao trong môi trường hợp tác, trong khi Yu và cộng sự [29] chứng minh rằng MAPPO - một mở rộng đơn giản của PPO sang nhiều tác nhân với chia sẻ tham số - đạt hiệu năng cạnh tranh hoặc vượt trội trên nhiều bộ benchmark hợp tác chuẩn.

Về mặt công cụ, Juliani và cộng sự [9] giới thiệu Unity ML-Agents như một hệ sinh thái mã nguồn mở tích hợp sẵn khả năng xây dựng môi trường ba chiều với mô phỏng vật lý chân thực, hỗ trợ huấn luyện tác nhân trực tiếp và được sử dụng rộng rãi trong cả nghiên cứu lẫn phát triển game.

Điều khiến các kết quả trên khó dùng lại nằm ở chỗ chúng gần như luôn được đo trên những bộ benchmark đã chuẩn hóa: lưới hai chiều, vector trạng thái rút gọn, động lực học đơn giản. Một thứ hạng thuật toán rút ra từ đó nói được rất ít về hành vi của cùng thuật toán ấy khi phải đương đầu với va chạm vật lý liên tục, quan sát một phần qua cảm biến tia và phần thưởng thưa của một cảnh ba chiều thật. Mỗi công trình lại thường chỉ khảo sát một hoặc hai thuật toán trên bộ môi trường riêng của mình, nên việc đặt PPO, SAC, MA-POCA và MAPPO lên cùng một bàn cân gần như không thực hiện được chỉ bằng cách đối chiếu tài liệu đã công bố. Đề tài này chấp nhận tự dựng lấy bàn cân đó: ba môi trường ba chiều thuộc ba dạng bài toán khác nhau, chung một cấu hình phần cứng và chung một quy trình đo.

1.5 Phương pháp nghiên cứu

Đề tài sử dụng kết hợp ba nhóm phương pháp nghiên cứu bổ trợ lẫn nhau. Phương pháp nghiên cứu lý thuyết được dùng để tổng hợp và phân tích các tài liệu khoa học về học tăng cường, học tăng cường sâu và bốn thuật toán PPO, SAC, MA-POCA và MAPPO, dựa trên các bài báo gốc cùng tài liệu kỹ thuật của Unity ML-Agents; đây là cơ sở cho việc thiết kế môi trường và lựa chọn tham số huấn luyện.

Phương pháp thực nghiệm được sử dụng để xây dựng ba môi trường game ba chiều trong Unity với đầy đủ hệ thống vật lý, quan sát, hành động và phần thưởng, sau đó triển khai và huấn luyện từng thuật toán trên mỗi môi trường trong cùng điều kiện phần cứng và số bước huấn luyện. Trong quá trình này, dữ liệu huấn luyện được

thu thập và trực quan hóa theo thời gian thực bằng công cụ TensorBoard.

Cuối cùng, phương pháp so sánh và đánh giá được dùng để đối chiếu hiệu năng của các thuật toán dựa trên bốn tiêu chí định lượng là tốc độ hội tụ, tỷ lệ hoàn thành nhiệm vụ, tổng phần thưởng trung bình và độ ổn định huấn luyện, kết hợp với phân tích định tính về mức độ phù hợp của từng thuật toán với từng dạng môi trường.

1.6 Ý nghĩa khoa học và thực tiễn

Về mặt khoa học, đề tài đề xuất và hiện thực hóa một quy trình xây dựng môi trường game ba chiều phục vụ huấn luyện học tăng cường trên nền tảng Unity, có thể được tái sử dụng như một bộ môi trường tham chiếu mở cho các nghiên cứu tiếp theo. Bên cạnh đó, bộ thực nghiệm so sánh có hệ thống giữa PPO, SAC, MA-POCA và MAPPO trên ba dạng môi trường đặc trưng góp phần làm rõ điều kiện phù hợp của từng thuật toán trong bối cảnh game ba chiều. Ngoài ra, quy trình tích hợp thuật toán off-policy vào Unity thông qua giao diện lập trình cấp thấp bằng Python cũng được trình bày chi tiết, mở ra một hướng đi có thể tái sử dụng cho các nghiên cứu về học off-policy trong môi trường Unity.

Về mặt thực tiễn, ba môi trường ba chiều được xây dựng trong đề tài có thể được sử dụng trực tiếp làm nền tảng thử nghiệm trí tuệ nhân tạo trong các dự án game Unity khác. Kết quả so sánh thuật toán giúp các nhà phát triển có cơ sở lựa chọn thuật toán học tăng cường phù hợp với từng cơ chế game, chẳng hạn cơ chế đối kháng, hợp tác hay điều hướng, mà không phải tiến hành lại toàn bộ thực nghiệm từ đầu. Quy trình thiết kế hàm phần thưởng và không gian quan sát trong môi trường ba chiều được trình bày trong đề tài cũng có giá trị tham khảo cho các nhóm phát triển game có nhu cầu tích hợp trí tuệ nhân tạo.

1.7 Bố cục đồ án

Ngoài phần Mở đầu và Kết luận, nội dung chính của đồ án được tổ chức thành bốn chương. Chương 2 trình bày cơ sở lý thuyết, bao gồm tổng quan về trí tuệ nhân tạo trong game, nền tảng lý thuyết của học tăng cường với quá trình quyết định Markov, chính sách và hàm giá trị, các kiến trúc học tăng cường sâu, cùng đặc điểm của bốn thuật toán PPO, SAC, MA-POCA và MAPPO; chương này cũng giới thiệu nền

tảng Unity ML-Agents và các giới hạn tích hợp liên quan đến việc triển khai thuật toán tùy chỉnh.

Chương 3 trình bày chi tiết quá trình thiết kế ba môi trường game ba chiều trong Unity, bao gồm không gian quan sát, không gian hành động và hàm phần thưởng của từng môi trường; đồng thời mô tả phương pháp tích hợp thuật toán SAC ra ngoài bộ huấn luyện tích hợp sẵn của ML-Agents, thiết lập thực nghiệm và các tiêu chí đánh giá được sử dụng. Chương 4 trình bày kết quả huấn luyện của từng thuật toán trên từng môi trường thông qua biểu đồ và bảng số liệu, tiến hành so sánh tổng hợp, phân tích ưu nhược điểm của từng thuật toán theo từng dạng bài toán và thảo luận về khả năng ứng dụng thực tiễn cùng hướng phát triển tiếp theo.

Cuối cùng, Chương 5 trình bày quá trình xây dựng ứng dụng trình diễn tương tác: kiến trúc hệ thống lựa chọn chế độ chơi, giao diện menu, cơ chế nạp mô hình ONNX tại thời gian chạy, đường ống đưa số liệu huấn luyện từ TensorBoard vào giao diện trong game, cách ánh xạ bốn chế độ chơi (người chơi, tác nhân tự chơi, người chơi cùng tác nhân, tác nhân đấu mô hình ngôn ngữ lớn) vào từng môi trường, cùng các vấn đề kỹ thuật phát sinh và kết quả kiểm thử ứng dụng.

2. CƠ SỞ LÝ THUYẾT

Chương này thiết lập nền tảng lý thuyết cho toàn bộ đề án, đi từ bối cảnh lịch sử của trí tuệ nhân tạo trong game tới bộ khái niệm hình thức của học tăng cường, rồi tới bốn thuật toán mà đề tài đem ra so sánh. Phần cuối chương dành cho Unity ML-Agents, và dành nhiều nhất cho những giới hạn tích hợp của nền tảng này: chính các giới hạn ấy, chứ không phải một lựa chọn học thuật nào, đã quy định cách đề tài phải bố trí thực nghiệm ở Chương 3.

2.1 Trí tuệ nhân tạo trong game

Trí tuệ nhân tạo trong game có lịch sử phát triển song hành với sự ra đời của ngành công nghiệp game từ những thập niên 1970-1980. Khác với AI trong nghiên cứu học thuật, AI trong game không nhất thiết phải tối ưu hay thông minh tuyệt đối - mục tiêu cốt lõi là tạo ra những đối thủ hoặc nhân vật phi người chơi (NPC) có hành vi đủ thuyết phục, góp phần mang lại trải nghiệm thú vị và thử thách cho

người chơi [12]. Theo thời gian, AI trong game đã trải qua hai giai đoạn phát triển rõ rệt: từ các kỹ thuật lập trình hành vi cứng nhắc dựa trên quy tắc đến các phương pháp học máy có khả năng thích nghi và tự cải thiện.

2.1.1 AI truyền thống

Trong giai đoạn đầu, AI game chủ yếu dựa trên các hệ thống quy tắc cứng (rule-based systems), trong đó lập trình viên định nghĩa trước toàn bộ hành vi của NPC dưới dạng tập hợp các điều kiện và phản ứng tương ứng. Cách tiếp cận này dễ kiểm soát và dự đoán được, nhưng thiếu tính linh hoạt khi gặp các tình huống ngoài dự kiến.

Để khắc phục hạn chế đó, máy trạng thái hữu hạn (Finite State Machine - FSM) được ứng dụng rộng rãi trong những thập niên 1990-2000. FSM mô hình hóa hành vi của NPC thông qua một tập hữu hạn các trạng thái (tuần tra, truy đuổi, tấn công, rút lui) và các điều kiện chuyển trạng thái tương ứng. Nhờ tính đơn giản và hiệu quả tính toán cao, FSM vẫn được sử dụng phổ biến trong nhiều thể loại game đến tận ngày nay [12]. Tuy nhiên, khi số lượng trạng thái và điều kiện chuyển tiếp tăng lên, FSM nhanh chóng trở nên phức tạp và khó bảo trì, dẫn đến hiện tượng thường được gọi là “state explosion”.

Để giải quyết bài toán lập kế hoạch hành động phức tạp hơn, Goal-Oriented Action Planning (GOAP) được giới thiệu và phổ biến qua tựa game F.E.A.R. (2005) [12]. Thay vì định nghĩa cứng các chuyển trạng thái, GOAP cho phép NPC tự lập kế hoạch hành động dựa trên mục tiêu hiện tại và trạng thái thế giới, mang lại hành vi linh hoạt và tự nhiên hơn đáng kể. Dù vậy, toàn bộ tri thức trong GOAP vẫn phải được lập trình thủ công bởi con người, khiến chi phí xây dựng và mở rộng hệ thống tăng cao khi độ phức tạp của game tăng lên.

2.1.2 AI học máy

Sự bùng nổ của học máy trong thập niên 2010 đã mở ra một hướng tiếp cận hoàn toàn mới cho AI trong game. Thay vì lập trình hành vi thủ công, các kỹ thuật học máy cho phép agent tự học từ dữ liệu hoặc từ tương tác với môi trường. Trong số các hướng tiếp cận đó, học tăng cường (Reinforcement Learning - RL) nổi lên như phương pháp phù hợp nhất với bản chất của game: agent học cách hành động thông

qua thử và sai, nhận phần thưởng hoặc hình phạt từ môi trường, và dần dần tối ưu hóa chiến lược của mình để đạt điểm số cao nhất [25].

Bước ngoặt quan trọng đến vào năm 2013 khi DeepMind công bố thuật toán DQN (Deep Q-Network) [14], lần đầu tiên chứng minh một agent RL có thể học chơi thành thạo 49 trò chơi Atari khác nhau chỉ từ dữ liệu pixel thô mà không cần bất kỳ tri thức nào được lập trình sẵn. Thành tựu này đánh dấu sự ra đời của Deep Reinforcement Learning - kết hợp sức mạnh biểu diễn của mạng nơ-ron sâu với khả năng học từ tương tác của RL.

Những năm tiếp theo chứng kiến hàng loạt thành tựu ấn tượng: AlphaGo và AlphaGo Zero [23, 24] chinh phục cờ vây ở trình độ kiện tướng thế giới; OpenAI Five [16] đánh bại các đội chuyên nghiệp trong Dota 2; AlphaStar [26] đạt danh hiệu Grandmaster trong StarCraft II. Các kết quả này vừa có giá trị học thuật, vừa mở ra tiềm năng ứng dụng RL trực tiếp trong phát triển game thương mại, từ việc tạo NPC thông minh đến sinh nội dung tự động và hỗ trợ kiểm thử game [28].

2.2 Nền tảng Reinforcement Learning

Học tăng cường (Reinforcement Learning - RL) là một nhánh của học máy trong đó một tác nhân hay còn gọi là agent học cách hành động thông qua tương tác trực tiếp với môi trường. Khác với học có giám sát, RL không yêu cầu dữ liệu gán nhãn sẵn có; thay vào đó, agent tự khám phá không gian hành động và nhận phản hồi dưới dạng phần thưởng hoặc hình phạt để dần dần cải thiện chiến lược của mình [25]. Đây là đặc tính cốt lõi khiến RL trở nên phù hợp với các bài toán game, nơi hành vi tối ưu thường khó định nghĩa tường minh nhưng lại dễ đánh giá thông qua kết quả.

2.2.1 Các khái niệm cơ bản

Vòng lặp tương tác trong RL được xây dựng xung quanh năm khái niệm nền tảng. Agent là thực thể ra quyết định - trong bối cảnh game, agent có thể là nhân vật người chơi, NPC hay robot mô phỏng.

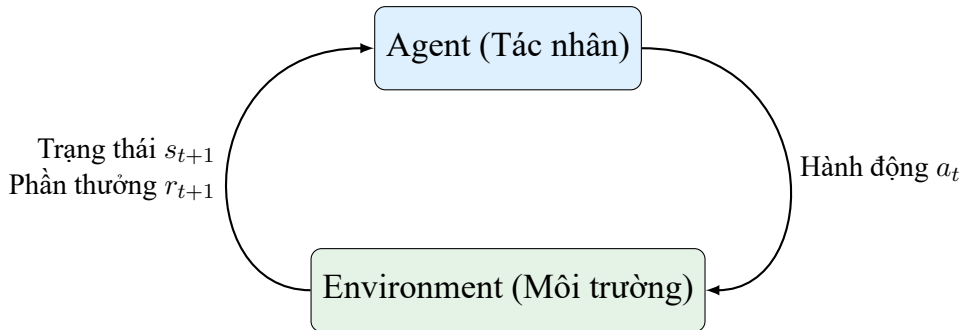
Environment hay môi trường là thế giới mà agent tương tác, bao gồm toàn bộ các đối tượng, luật chơi và cơ chế phản hồi; trong đề tài này, môi trường là các

scene 3D được xây dựng trong Unity.

State hay trạng thái $s \in \mathcal{S}$ là biểu diễn của môi trường tại một thời điểm nhất định mà agent có thể quan sát được. Trong môi trường 3D, state thường bao gồm vị trí, vận tốc của các đối tượng, khoảng cách đến mục tiêu và các thông tin không gian liên quan. Action hay hành động $a \in \mathcal{A}$ là tập hợp các lựa chọn mà agent có thể thực hiện tại mỗi bước, có thể là rời rạc như di chuyển lên, xuống, trái, phải hoặc liên tục như lực đẩy theo trục x, y, z .

Reward hay phần thưởng $r_t \in \mathbb{R}$ là tín hiệu phản hồi mà môi trường trả về sau mỗi hành động của agent, đóng vai trò là thước đo duy nhất để agent đánh giá chất lượng quyết định của mình. Việc thiết kế hàm phần thưởng hợp lý là một trong những bước quan trọng và khó nhất trong quá trình xây dựng hệ thống RL, vì một hàm phần thưởng không phù hợp có thể dẫn đến hành vi không mong muốn dù thuật toán hoạt động đúng về mặt lý thuyết [25].

Tại mỗi bước thời gian t , agent quan sát trạng thái s_t , thực hiện hành động a_t , nhận phần thưởng r_t và chuyển sang trạng thái mới s_{t+1} . Vòng lặp này lặp lại liên tục cho đến khi episode kết thúc, tức khi agent hoàn thành nhiệm vụ, thất bại hoặc vượt quá số bước tối đa, và hình thành nên một quỹ đạo tương tác $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$. Toàn bộ vòng lặp tương tác giữa agent và môi trường được minh họa trong Hình 2.1. Đây là sơ đồ khái quát nhất của mọi hệ thống học tăng cường: agent quan sát trạng thái hiện tại của môi trường, lựa chọn một hành động dựa trên chính sách của mình, và môi trường phản hồi lại bằng một trạng thái mới cùng một phần thưởng, cứ thế lặp đi lặp lại để agent dần cải thiện chính sách.



Hình 2.1: Vòng lặp tương tác giữa agent và môi trường trong học tăng cường

2.2.2 Markov Decision Process

Hầu hết các bài toán RL được hình thức hóa dưới dạng Quá trình Quyết định Markov (Markov Decision Process - MDP), một khung toán học cung cấp nền tảng lý thuyết chặt chẽ cho việc mô hình hóa bài toán ra quyết định tuần tự trong môi trường ngẫu nhiên [18]. Một MDP được định nghĩa bởi bộ năm thành phần $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, trong đó $\mathcal{S}$ là không gian trạng thái, $\mathcal{A}$ là không gian hành động, $\mathcal{P}$ là hàm chuyển trạng thái, $\mathcal{R}$ là hàm phần thưởng và $\gamma \in [0, 1)$ là hệ số chiết khấu.

Hàm chuyển trạng thái $\mathcal{P}(s' | s, a)$ xác định xác suất chuyển từ trạng thái s sang trạng thái s' khi thực hiện hành động a . Tính chất Markov - hay còn gọi là tính không nhớ - đòi hỏi rằng trạng thái tiếp theo s_{t+1} chỉ phụ thuộc vào trạng thái hiện tại s_t và hành động a_t , không phụ thuộc vào lịch sử trước đó:

$$\mathcal{P}(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots) = \mathcal{P}(s_{t+1} | s_t, a_t) \quad (2.1)$$

Giả thiết Markov là thứ giữ cho bài toán còn giải được: nhờ nó, agent chỉ cần quan sát trạng thái hiện tại để ra quyết định tối ưu thay vì phải lưu trữ và xử lý toàn bộ lịch sử tương tác. Trong thực tế, nhiều môi trường game thỏa mãn gần đúng tính chất này nếu state được thiết kế đủ phong phú để mang đầy đủ thông tin cần thiết.

Hệ số chiết khấu γ xác định mức độ agent coi trọng phần thưởng tương lai so với phần thưởng hiện tại. Khi γ gần 1, agent có xu hướng lập kế hoạch dài hạn; khi γ nhỏ, agent tập trung vào phần thưởng tức thời. Tổng phần thưởng tích lũy có chiết khấu mà agent hướng đến tối đa hóa được định nghĩa là:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (2.2)$$

2.2.3 Policy và Value Function

Policy (chính sách) π là hàm ánh xạ từ trạng thái sang hành động, định nghĩa chiến lược hành động của agent. Policy có thể là tất định $\pi : \mathcal{S} \rightarrow \mathcal{A}$ hoặc ngẫu nhiên $\pi(a | s) = P(A_t = a | S_t = s)$, cho biết xác suất chọn hành động a khi ở trạng thái s . Mục tiêu của RL là tìm policy tối ưu π^* sao cho tổng phần thưởng tích lũy

kỳ vọng là lớn nhất.

Value Function hay còn được gọi là hàm giá trị đo lường mức độ tốt của một trạng thái hoặc một cặp trạng thái-hành động khi agent tuân theo một policy nhất định. Hàm giá trị trạng thái $V^\pi(s)$ được định nghĩa là tổng phần thưởng tích lũy kỳ vọng khi bắt đầu từ trạng thái s và tuân theo policy π :

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid S_t = s \right] \quad (2.3)$$

Hàm giá trị hành động $Q^\pi(s, a)$ (hay Q-function) mở rộng khái niệm trên bằng cách đánh giá giá trị của việc thực hiện hành động a tại trạng thái s rồi sau đó tuân theo policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a] \quad (2.4)$$

Mối quan hệ giữa V^π và Q^π được thể hiện qua phương trình Bellman [2], nền tảng lý thuyết của hầu hết các thuật toán RL hiện đại:

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a) \quad (2.5)$$

Policy tối ưu π^* và hàm giá trị tối ưu $V^*(s)$, $Q^*(s, a)$ thỏa mãn phương trình tối ưu Bellman:

$$Q^*(s, a) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid S_t = s, A_t = a \right] \quad (2.6)$$

Trong thực tế, với không gian trạng thái lớn và liên tục như các môi trường game 3D, việc biểu diễn chính xác V^π hay Q^π dưới dạng bảng tra cứu là không khả thi. Đây là động lực dẫn đến sự ra đời của Deep Reinforcement Learning, trong đó các hàm giá trị và policy được xấp xỉ bằng mạng nơ-ron sâu, được trình bày chi tiết trong Mục 2.3.

2.3 Học tăng cường sâu

Học tăng cường sâu (Deep Reinforcement Learning) là sự kết hợp giữa học tăng cường và mạng nơ-ron sâu (deep neural network), ra đời nhằm giải quyết hạn chế

cốt lõi của RL truyền thống khi đối mặt với các bài toán có không gian trạng thái và hành động lớn, liên tục. Thay vì lưu trữ giá trị của từng cặp trạng thái-hành động trong bảng tra cứu, Deep RL sử dụng mạng nơ-ron để xấp xỉ các hàm giá trị hoặc policy, cho phép tổng quát hóa sang các trạng thái chưa từng gặp trong quá trình huấn luyện [10]. Đây là đặc tính then chốt khiến Deep RL có thể xử lý được các môi trường game 3D phức tạp như trong đề tài này.

2.3.1 *Vai trò của mạng Neural Network trong RL*

Trong RL truyền thống, hàm giá trị $Q(s, a)$ hoặc policy $\pi(a | s)$ được biểu diễn dưới dạng bảng tra cứu (lookup table), đòi hỏi mỗi cặp (s, a) phải được thăm viếng đủ nhiều lần để ước lượng chính xác. Cách tiếp cận này chỉ khả thi với không gian trạng thái hữu hạn và nhỏ. Với các môi trường game 3D, không gian trạng thái có thể là liên tục và có số chiều rất lớn - ví dụ, vector quan sát của agent trong môi trường Football Table bao gồm vị trí, vận tốc của bóng và các thanh gạt trong không gian ba chiều - khiến bảng tra cứu hoàn toàn không khả thi.

Mạng nơ-ron sâu giải quyết vấn đề này bằng cách đóng vai trò là bộ xấp xỉ hàm tổng quát (universal function approximator). Thay vì lưu trữ giá trị cho từng trạng thái, mạng học một ánh xạ tham số hóa $f_\theta : \mathcal{S} \rightarrow \mathbb{R}$ (đối với value function) hoặc $f_\theta : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ (đối với policy), trong đó θ là tập tham số của mạng được tối ưu hóa qua quá trình huấn luyện. Nhờ khả năng tổng quát hóa, mạng có thể ước lượng giá trị hoặc đưa ra hành động cho các trạng thái chưa từng gặp dựa trên sự tương đồng với các trạng thái đã học [25].

Trong thực tế, kiến trúc mạng được lựa chọn tùy theo dạng dữ liệu quan sát. Với dữ liệu dạng vector số thực (vị trí, vận tốc, khoảng cách), mạng fully-connected nhiều lớp (MLP) là lựa chọn phổ biến và hiệu quả. Với dữ liệu dạng ảnh hoặc dữ liệu không gian, mạng tích chập (CNN) thường được ưu tiên. Unity ML-Agents hỗ trợ cả hai kiến trúc này và cho phép tùy chỉnh số lớp ẩn cũng như số đơn vị trong mỗi lớp thông qua tệp cấu hình YAML.

2.3.2 *Phân loại: Policy-Based, Value-Based, Actor-Critic*

Các thuật toán Deep RL hiện đại được phân loại thành ba nhóm chính dựa trên đối tượng mà mạng nơ-ron học để xấp xỉ, mỗi nhóm có ưu nhược điểm riêng và phù

hợp với các dạng bài toán khác nhau.

Value-Based là nhóm thuật toán trong đó mạng nơ-ron học xấp xỉ hàm giá trị $Q(s, a)$, từ đó suy ra policy tối ưu bằng cách chọn hành động có giá trị Q cao nhất. Đại diện tiêu biểu là DQN [14]. Nhóm này hoạt động tốt với không gian hành động rời rạc nhưng gặp khó khăn với không gian hành động liên tục do không thể tối ưu $\max_a Q(s, a)$ một cách hiệu quả khi $\mathcal{A}$ là liên tục.

Policy-Based là nhóm thuật toán trực tiếp tham số hóa và tối ưu policy $\pi_\theta(a | s)$ bằng cách tính gradient của hàm mục tiêu theo tham số θ thông qua phương pháp policy gradient [27]. Nhóm này xử lý tốt cả không gian hành động rời rạc lẫn liên tục, nhưng thường có phương sai cao trong ước lượng gradient, dẫn đến quá trình huấn luyện kém ổn định. PPO [22] thuộc nhóm này, bổ sung thêm cơ chế clipping để giới hạn độ lớn của mỗi bước cập nhật, giúp ổn định quá trình huấn luyện đáng kể.

Actor-Critic là nhóm kết hợp ưu điểm của cả hai nhóm trên bằng cách duy trì đồng thời hai mạng nơ-ron: mạng actor học policy $\pi_\theta(a | s)$ và mạng critic học hàm giá trị $V_\phi(s)$ hoặc $Q_\phi(s, a)$. Critic đóng vai trò đánh giá chất lượng hành động của actor, cung cấp tín hiệu học ổn định hơn so với việc chỉ dựa vào phần thưởng thực tế [13]. SAC [7] và MA-POCA [4] đều thuộc nhóm Actor-Critic, trong đó SAC bổ sung thêm entropy regularization để khuyến khích agent khám phá, còn MA-POCA mở rộng kiến trúc critic sang bài toán đa tác nhân với cơ chế centralized critic cho phép mỗi agent học dựa trên thông tin toàn cục trong quá trình huấn luyện.

Bảng 2.1 tóm tắt vị trí của bốn thuật toán được sử dụng trong đề tài theo cách phân loại trên.

Bảng 2.1: Phân loại các thuật toán sử dụng trong đề tài

Thuật toán	Nhóm	On/Off-policy	Phù hợp với
PPO	Policy-Based	On-policy	Đối kháng, điều hướng
SAC	Actor-Critic	Off-policy	Hành động liên tục
MA-POCA	Actor-Critic	On-policy	Hợp tác đa tác nhân
MAPPO	Policy-Based	On-policy	Hợp tác, tác nhân đồng nhất

2.3.3 *Phần thưởng tò mò nội tại*

Trong các bài toán học tăng cường thực tế, đặc biệt là các môi trường game 3D phức tạp như Capture The Flag hay Cross The Road, tín hiệu phần thưởng ngoại sinh thường rất thưa thớt hoặc bị trì hoãn. Nếu tác nhân chỉ dựa vào phần thưởng từ môi trường để học, nó có thể dành phần lớn thời gian đầu khám phá ngẫu nhiên mà không nhận được bất kỳ tín hiệu hữu ích nào, dẫn đến việc không thể học được chính sách hiệu quả. Để giải quyết vấn đề này, cơ chế phần thưởng tò mò nội tại (Intrinsic Curiosity) đã được đề xuất nhằm khuyến khích tác nhân chủ động khám phá những trạng thái mới mẻ của môi trường.

Trong đồ án này, cơ chế tò mò được tích hợp thông qua mô hình Intrinsic Curiosity Module (ICM) do Pathak và cộng sự [17] đề xuất. Ý tưởng cốt lõi của ICM là sử dụng sai số dự đoán (prediction error) làm tín hiệu phần thưởng nội tại: tác nhân sẽ nhận được phần thưởng cao khi nó chuyển đến một trạng thái mà mô hình nội tại của nó không thể dự đoán chính xác.

Kiến trúc của ICM bao gồm hai mạng nơ-ron con hoạt động song song: mô hình ngược (Inverse Model) và mô hình thuận (Forward Model). Để tránh việc tác nhân bị thu hút bởi những yếu tố ngẫu nhiên không thể dự đoán trong môi trường (hiện tượng tivi nhiễu hạt - noisy TV problem), ICM không dự đoán trực tiếp trạng thái tiếp theo s_{t+1} , mà dự đoán đặc trưng ẩn $\phi(s_{t+1})$ được trích xuất qua một mạng mã hóa (Encoder).

Mô hình ngược (Inverse Model): Mạng này nhận đầu vào là đặc trưng của trạng thái hiện tại $\phi(s_t)$ và đặc trưng của trạng thái tiếp theo $\phi(s_{t+1})$, sau đó dự đoán hành động $\hat{a}_t$ đã gây ra sự chuyển tiếp này. Hàm mất mát của Inverse Model được định nghĩa là:

$$L_{\text{inv}} = \mathbb{E} [-\log P(\hat{a}_t = a_t \mid \phi(s_t), \phi(s_{t+1}))] \quad (2.7)$$

Việc huấn luyện Inverse Model buộc mạng mã hóa ϕ chỉ giữ lại những thông tin trạng thái có liên quan trực tiếp đến hành động của tác nhân, loại bỏ các chi tiết nhiễu vô ích.

Mô hình thuận (Forward Model): Mạng này nhận đầu vào là đặc trưng trạng thái hiện tại $\phi(s_t)$ và hành động a_t , từ đó dự đoán đặc trưng của trạng thái tiếp theo $\phi(s_{t+1})$.

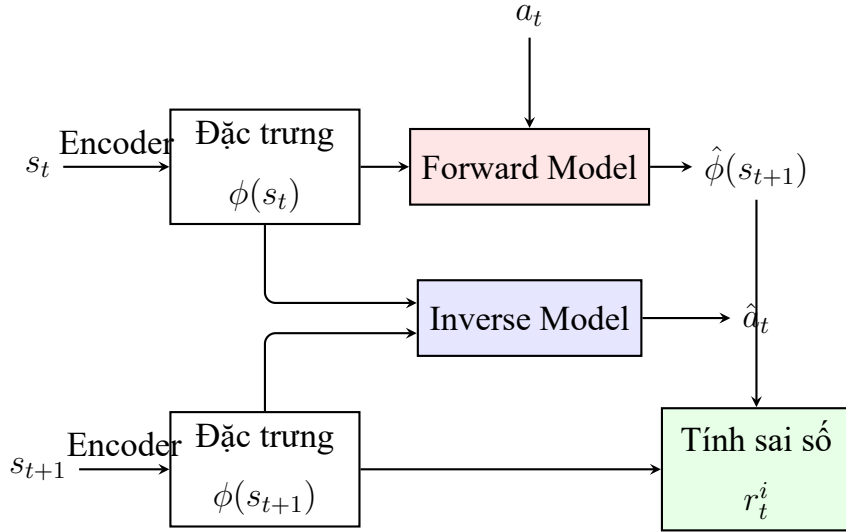
theo $\hat{\phi}(s_{t+1})$. Hàm mất mát của Forward Model thường là sai số bình phương trung bình:

$$L_{\text{fwd}} = \frac{1}{2} \left\| \hat{\phi}(s_{t+1}) - \phi(s_{t+1}) \right\|_2^2 \quad (2.8)$$

Phần thưởng tò mò: Sai số của Forward Model chính là thước đo sự “ngạc nhiên” của tác nhân. Phần thưởng tò mò r_t^i được định nghĩa tỷ lệ thuận với sai số này:

$$r_t^i = \eta \frac{1}{2} \left\| \hat{\phi}(s_{t+1}) - \phi(s_{t+1}) \right\|_2^2 \quad (2.9)$$

trong đó η là hệ số tỷ lệ. Tổng phần thưởng mà tác nhân nhận được tại mỗi bước sẽ là sự kết hợp có trọng số giữa phần thưởng ngoại sinh và phần thưởng nội tại: $r_t = r_t^e + \beta r_t^i$. Trong các thực nghiệm của đề tài, β thường được đặt ở mức 0,02 nhằm bảo đảm tác nhân vẫn hướng tới mục tiêu chính của trò chơi.



Hình 2.2: Sơ đồ kiến trúc Intrinsic Curiosity Module (ICM)

2.4 Các thuật toán sử dụng trong đề tài

Đề tài sử dụng bốn thuật toán học tăng cường đại diện cho các hướng tiếp cận khác nhau: PPO thuộc nhóm policy-based on-policy đơn tác nhân, SAC thuộc nhóm actor-critic off-policy, MA-POCA là thuật toán chuyên biệt cho bài toán đa tác nhân dựa trên cơ chế attention, và MAPPO là mở rộng của PPO cho đa tác nhân thông qua chia sẻ tham số. Phần này trình bày nguyên lý hoạt động và đặc điểm của từng thuật toán, làm cơ sở cho việc lựa chọn và phân tích kết quả thực nghiệm ở các chương sau.

2.4.1 PPO

PPO (Proximal Policy Optimization) được Schulman và cộng sự đề xuất vào năm 2017 [22] như một cải tiến của TRPO (Trust Region Policy Optimization) [21] nhằm giữ lại tính ổn định trong cập nhật policy nhưng đơn giản hơn đáng kể về mặt triển khai. PPO hiện là một trong những thuật toán RL phổ biến nhất trong cả nghiên cứu lẫn ứng dụng thực tế, và là thuật toán on-policy mặc định trong Unity ML-Agents.

Về nguyên lý hoạt động, PPO là thuật toán on-policy, nghĩa là dữ liệu huấn luyện được thu thập trực tiếp từ policy hiện tại và bị loại bỏ sau mỗi lần cập nhật. Ý tưởng cốt lõi của PPO là giới hạn độ lớn thay đổi của policy trong mỗi bước cập nhật, tránh tình trạng policy thay đổi quá đột ngột dẫn đến suy giảm hiệu năng không thể phục hồi - một vấn đề phổ biến trong các thuật toán policy gradient đơn giản.

PPO đạt được điều này thông qua hàm mục tiêu clipped surrogate:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (2.10)$$

trong đó $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ là tỷ lệ xác suất giữa policy mới và policy cũ, $\hat{A}_t$ là ước lượng advantage (mức độ hành động tốt hơn hoặc kém hơn kỳ vọng trung bình) và ε là siêu tham số clipping thường được đặt trong khoảng $[0.1, 0.2]$. Toán tử clip giới hạn $r_t(\theta)$ trong khoảng $[1 - \varepsilon, 1 + \varepsilon]$, ngăn policy mới lệch quá xa policy cũ trong một bước cập nhật. Hàm min đảm bảo mục tiêu luôn là cận dưới thận trọng (pessimistic lower bound) của mục tiêu không clipped.

Giá trị advantage $\hat{A}_t$ xuất hiện trong biểu thức trên thường được ước lượng theo phương pháp Generalized Advantage Estimation (GAE), một cách dung hòa giữa độ lệch và phương sai của ước lượng thông qua tham số λ :

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (2.11)$$

trong đó $\lambda \in [0, 1]$ là tham số cân bằng giữa bias và phương sai của ước lượng advantage, $V(s_t)$ là giá trị trạng thái được ước lượng bởi mạng critic.

Trong mỗi iteration, PPO thu thập một batch dữ liệu từ môi trường bằng policy hiện tại, sau đó thực hiện nhiều lần cập nhật gradient trên cùng batch đó (thường từ 3 đến 10 epochs) trước khi thu thập batch mới. Nhờ cơ chế clipping, các lần cập nhật lặp lại này không làm policy lệch quá xa điểm xuất phát, đảm bảo tính ổn định mà không cần tính toán tốn kém như TRPO.

Ưu điểm: Thế mạnh thật sự của PPO không nằm ở chất lượng chính sách cuối cùng mà ở chỗ nó hiếm khi hỏng. Clipping chặn trên độ lớn của mỗi bước cập nhật, nên một tốc độ học đặt sai thường chỉ làm chậm quá trình học chứ không phá sập chính sách như ở các phương pháp policy gradient thuần túy. Thuật toán chạy được trên cả không gian hành động rời rạc lẫn liên tục và đi kèm sẵn cơ chế self-play trong ML-Agents. Vì đặc tính bền này, đề tài chọn PPO làm mốc so sánh: mọi thuật toán khác đều được đối chiếu với nó.

Nhược điểm: Cái giá phải trả là hiệu quả mẫu. Là thuật toán on-policy, PPO vứt bỏ toàn bộ dữ liệu sau mỗi lần cập nhật, nên cần nhiều lần tương tác với môi trường hơn hẳn để đạt cùng một mức năng lực. Trong một môi trường Unity có mô phỏng vật lý, nơi một bước tương tác đắt hơn nhiều so với một bước lan truyền ngược, đây là hạn chế nặng nhất của PPO, và cũng chính là lý do đề tài đưa SAC vào so sánh.

2.4.2 SAC

SAC (Soft Actor-Critic) được Haarnoja và cộng sự đề xuất vào năm 2018 [7,8] như một thuật toán actor-critic off-policy dựa trên nguyên lý maximum entropy, hướng đến giải quyết đồng thời hai thách thức lớn của RL: hiệu quả mẫu thấp và sự nhạy cảm với siêu tham số.

Điểm khác biệt cốt lõi của SAC so với các thuật toán RL thông thường nằm ở hàm mục tiêu. Thay vì chỉ tối đa hóa tổng phần thưởng kỳ vọng, SAC tối đa hóa đồng thời tổng phần thưởng và entropy của policy:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t \left(r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right) \right] \quad (2.12)$$

trong đó $\mathcal{H}(\pi(\cdot | s_t)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a | s_t)]$ là entropy của policy tại trạng thái s_t và $\alpha > 0$ là hệ số nhiệt độ (temperature) cân bằng giữa phần thưởng và entropy.

Entropy cao đồng nghĩa với policy ngẫu nhiên hơn, khuyến khích agent khám phá nhiều hành động khác nhau thay vì hội tụ sớm về một hành động tham lam.

SAC duy trì ba mạng nơ-ron: mạng actor $\pi_\theta(a \mid s)$ sinh ra phân phối xác suất trên không gian hành động, và hai mạng critic $Q_{\phi_1}(s, a)$, $Q_{\phi_2}(s, a)$ được huấn luyện độc lập để ước lượng hàm giá trị hành động. Việc sử dụng hai critic song song và lấy giá trị nhỏ hơn (clipped double-Q trick) giúp giảm thiểu hiện tượng overestimation phổ biến trong các thuật toán Q-learning:

$$Q_{\text{target}}(s_t, a_t) = r_t + \gamma \mathbb{E}_{a' \sim \pi} \left[\min_{i=1,2} Q_{\phi_i}(s_{t+1}, a') - \alpha \log \pi_\theta(a' \mid s_{t+1}) \right] \quad (2.13)$$

Là thuật toán off-policy, SAC lưu trữ toàn bộ kinh nghiệm tương tác vào replay buffer và lấy mẫu ngẫu nhiên (mini-batch) từ buffer này để cập nhật mạng, cho phép tái sử dụng dữ liệu cũ nhiều lần và đạt hiệu quả mẫu cao hơn đáng kể so với PPO. Hệ số nhiệt độ α có thể được cố định hoặc tự động điều chỉnh theo mục tiêu entropy mong muốn $\mathcal{H}_{\text{target}}$, giảm thiểu nhu cầu tinh chỉnh siêu tham số thủ công.

Ưu điểm: Replay buffer cho phép mỗi mẫu kinh nghiệm được dùng lại nhiều lần, nhờ đó SAC cần ít lần tương tác với môi trường hơn hẳn các thuật toán on-policy; trên môi trường điều hướng của đề tài, khoảng cách này lên tới một bậc độ lớn. Cơ chế cực đại hóa entropy giữ cho chính sách còn ngẫu nhiên ngay cả khi phần thưởng đã bắt đầu tăng, qua đó chống lại việc hội tụ sớm về một cực trị cục bộ. SAC cũng chịu được tốc độ học đặt sai tốt hơn PPO [8].

Nhược điểm: Đổi lại, SAC phải huấn luyện đồng thời ba mạng nơ-ron và phải giữ một bộ đệm hàng trăm nghìn phần tử trong bộ nhớ, nên lợi thế về số mẫu bị thu hẹp lại khi tính theo thời gian đồng hồ. Nghiêm trọng hơn nhiều đối với đề tài là việc SAC không đi cùng được với self-play: khi đối thủ liên tục được thay mới, kinh nghiệm cũ trong buffer phản ánh những đối thủ đã không còn tồn tại và trở thành nhiễu thay vì thành dữ liệu học. Ràng buộc này chi phối toàn bộ cách đề tài bố trí thực nghiệm trên môi trường đối kháng, và được phân tích ở Mục 3.7.

2.4.3 MA-POCA

MA-POCA được Cohen và cộng sự [4] đề xuất năm 2021 như thuật toán chuyên biệt cho bài toán học tăng cường đa tác nhân hợp tác (cooperative multi-agent), và

là thuật toán đa tác nhân mặc định trong Unity ML-Agents. MA-POCA giải quyết hai thách thức đặc thù của môi trường đa tác nhân mà PPO hay SAC nguyên bản khó xử lý tốt: bài toán gán công lao (credit assignment) - xác định đóng góp của từng agent vào phần thưởng chung của cả nhóm - và bài toán tác nhân biến động (posthumous credit) - gán công lao đúng cách cho những agent đã kết thúc hoạt động (bị loại hoặc hoàn thành nhiệm vụ) trước khi phần thưởng nhóm được trao.

Về nguyên lý hoạt động, MA-POCA mở rộng kiến trúc actor-critic theo mô hình huấn luyện tập trung, thực thi phân tán (centralized training with decentralized execution). Trong quá trình huấn luyện, một mạng critic tập trung (centralized critic) được phép quan sát trạng thái và hành động của toàn bộ các agent trong nhóm để ước lượng giá trị chung, nhờ đó đánh giá chính xác hơn đóng góp của từng agent. Tuy nhiên, khi triển khai thực tế, mỗi agent chỉ dựa trên quan sát cục bộ của riêng mình để ra quyết định, đảm bảo tính khả thi và khả năng mở rộng.

Điểm cải tiến cốt lõi của MA-POCA nằm ở việc sử dụng cơ chế attention để xử lý số lượng agent thay đổi động trong một episode. Thay vì cố định số lượng agent như các phương pháp trước [6, 11], MA-POCA cho phép agent gia nhập hoặc rời khỏi nhóm giữa chừng mà vẫn gán công lao chính xác - đúng tình huống xảy ra thường xuyên trong game hợp tác, nơi agent có thể “chết” hoặc hoàn thành nhiệm vụ ở những thời điểm khác nhau. Hàm giá trị nhóm được học dựa trên tổng phần thưởng nhóm (group reward) do môi trường cung cấp thông qua cơ chế SimpleMultiAgentGroup của ML-Agents.

Về phần cập nhật policy, MA-POCA kế thừa hàm mục tiêu clipped surrogate và cơ chế ước lượng advantage tương tự PPO (đều là thuật toán on-policy), nhưng thay hàm giá trị trạng thái đơn tác nhân $V(s)$ bằng hàm giá trị nhóm dựa trên baseline tập trung, cho phép mỗi agent nhận được tín hiệu học phản ánh đúng đóng góp của mình vào kết quả chung.

Ưu điểm: MA-POCA trả lời được câu hỏi mà một PPO độc lập cho từng agent không trả lời nổi: khi cả nhóm cùng nhận một phần thưởng, agent nào đã thực sự làm nên kết quả đó. Cơ chế attention cho phép nhóm thay đổi quân số giữa chừng mà việc gán công lao vẫn đúng đắn, một tình huống rất thường gặp trong game hợp tác nơi nhân vật có thể bị loại hoặc hoàn thành phần việc của mình sớm. Trong

ML-Agents, thuật toán ăn khớp sẵn với cơ chế group reward nên môi trường hợp tác gần như không phải sửa gì thêm.

Nhược điểm: Là thuật toán on-policy, MA-POCA thừa hưởng nguyên vẹn vấn đề hiệu quả mẫu của PPO, đồng thời gánh thêm chi phí của mạng phê bình tập trung khi quân số đông. Điểm đáng bàn hơn là sự linh hoạt ấy không miễn phí: attention phải học cách phân bổ trọng số giữa các agent, và trong một đội hình chỉ gồm hai tác nhân giống hệt nhau thì đó là công sức bỏ ra cho một bài toán vốn không tồn tại. Thực nghiệm ở Chương 4 cho thấy chính tình huống này khiến MA-POCA thua MAPPO. Với bài toán đơn tác nhân, thuật toán suy biến gần về PPO nhưng vẫn mang theo toàn bộ phần kiến trúc thừa.

2.4.4 MAPPO

Được đề xuất bởi Yu và cộng sự (2022) [29], MAPPO (Multi-Agent Proximal Policy Optimization) là một thuật toán giải quyết bài toán đa tác nhân thông qua việc mở rộng trực tiếp PPO. Trong khi MA-POCA sử dụng mạng critic tập trung (centralized critic) kết hợp cơ chế attention phức tạp, MAPPO tiếp cận theo hướng đơn giản hơn: mở rộng PPO sang môi trường đa tác nhân bằng cách áp dụng framework CTDE (Centralized Training with Decentralized Execution) kết hợp với việc chia sẻ tham số và một hàm giá trị tập trung.

Kiến trúc CTDE trong MAPPO: Trong quá trình thực thi, mỗi agent i đưa ra hành động dựa trên policy cá nhân $\pi_{\theta_i}(a_i|o_i)$, chỉ phụ thuộc vào quan sát cục bộ o_i . Tuy nhiên, trong quá trình huấn luyện (centralized training), hàm giá trị tập trung $V_\phi(s)$ được sử dụng để ước lượng giá trị của trạng thái toàn cục s , giúp đánh giá chính xác hơn giá trị của một trạng thái chung và giảm phương sai trong quá trình tính toán policy gradient. Hàm giá trị tập trung trong MAPPO có dạng:

$$V_\phi(s) = V_\phi(o_1, o_2, \dots, o_n, s_{\text{global}})$$

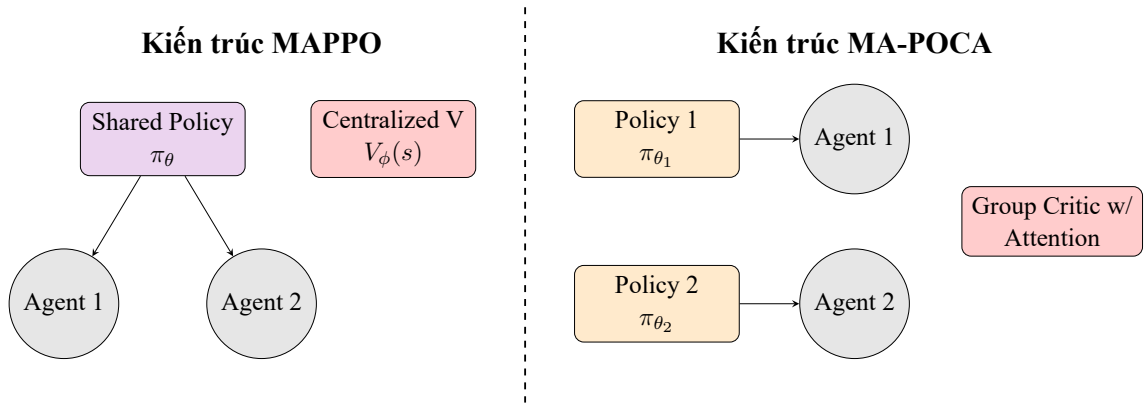
với $o_1, \dots, o_n$ là quan sát của các agent và s_{global} là các thông tin toàn cục khác của môi trường.

Hàm mục tiêu của policy đối với mỗi agent i trong MAPPO được định nghĩa là:

$$L_i^{\text{MAPPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t^i, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t^i \right) \right] \quad (2.14)$$

trong đó $\hat{A}_t^i$ là hàm lợi thế (advantage function) cho agent i , được tính toán dựa trên hàm giá trị tập trung $V_\phi(s)$, và $r_t(\theta) = \frac{\pi_\theta(a_t^i|o_t^i)}{\pi_{\theta_{\text{old}}}(a_t^i|o_t^i)}$ là tỷ số xác suất.

Cơ chế chia sẻ tham số (Parameter sharing): Mặc dù mỗi agent có thể có một policy riêng, MAPPO thường ứng dụng việc chia sẻ tham số mạng nơ-ron giữa tất cả các agent đồng nhất (homogeneous agents), tức là $\theta_1 = \theta_2 = \dots = \theta_n = \theta$. Điều này mang lại lợi ích lớn trong việc giảm thiểu chi phí tính toán và bộ nhớ, đồng thời cho phép các agent học hỏi nhanh hơn nhờ việc chia sẻ kinh nghiệm (shared experience) trong quá trình huấn luyện. Danh tính (identity) của từng agent có thể được mã hóa thành một vector one-hot và đưa vào như một phần của quan sát (observation) để giúp mạng phân biệt và có các hành vi khác nhau.



Hình 2.3: So sánh cấu trúc giữa MAPPO và MA-POCA

Ưu điểm: Sức mạnh của MAPPO đến từ chính chỗ nó chẳng thêm gì mới. Vì mọi agent dùng chung một bộ tham số, mỗi lượt chơi sinh ra n luồng kinh nghiệm cùng chảy về một mạng duy nhất; với n tác nhân, cùng một ngân sách mô phỏng cho ra gấp n lần dữ liệu huấn luyện hữu ích. Thuật toán chạy được trên toàn bộ hạ tầng đã tối ưu sẵn cho PPO. Yu và cộng sự [29] ghi nhận MAPPO bám sát hoặc vượt các thuật toán đa tác nhân nặng ký hơn như MADDPG và QMIX trên nhiều bộ benchmark hợp tác.

Nhược điểm: Chia sẻ tham số chỉ hợp lý chừng nào các agent thật sự giống nhau về quan sát và hành động; với đội hình có vai trò bất đối xứng, một mạng duy nhất buộc phải học lẫn lộn nhiều hành vi mâu thuẫn. Hàm giá trị tập trung $V_\phi(s)$ cũng phình theo quân số vì không gian trạng thái toàn cục lớn dần. Và khác MA-POCA, MAPPO không có sẵn cách xử lý gọn gàng cho trường hợp số agent thay

đổi giữa lượt chơi.

2.4.5 Tổng kết và so sánh bốn thuật toán

Bốn thuật toán vừa trình bày không xếp được thành một thang bậc từ yếu đến mạnh. Chúng phân hóa theo ba trục độc lập: cách lấy dữ liệu (on-policy hay off-policy), cách xử lý nhiều tác nhân (không có, attention hay chia sẻ tham số) và chi phí tính toán của mỗi bước cập nhật. Chính ba trục này, chứ không phải một thứ hạng chung, quyết định thuật toán nào hợp với bài toán nào.

Bảng 2.2: So sánh các thuật toán học tăng cường

Thuật toán	Loại policy	On/Off-policy	Đa tác nhân	Replay buffer	Entropy	Chia sẻ tham số	Độ phức tạp
PPO	Policy gradient	On-policy	Không native	Không	Không	Không	Thấp
SAC	Actor-Critic	Off-policy	Không native	Có	Có (max entropy)	Không	Trung bình
MA-POCA	Actor-Critic	On-policy	Có (attention)	Không	Không	Có (chia sẻ)	Cao
MAPPO	Policy gradient	On-policy	Có (CTDE)	Không	Không	Có (bắt buộc)	Trung bình

Từ bảng trên rút ra được một khuyến nghị đủ cụ thể để dùng ngay. Khi chưa nắm rõ đặc thù bài toán, nên bắt đầu bằng PPO, vì nó chịu đựng được siêu tham số đặt sai và cho một mốc so sánh đáng tin để đánh giá mọi thuật toán khác. Chỉ nên đổi sang SAC khi mỗi bước tương tác với môi trường đắt hơn hẳn mỗi bước cập nhật mạng: đó đúng là lúc replay buffer trả lại đủ giá trị để bù cho chi phí huấn luyện ba mạng song song. Với bài toán nhiều tác nhân, ranh giới nằm ở chỗ đội hình có đồng nhất hay không. Các tác nhân giống nhau và cố định về số lượng thì chia sẻ tham số của MAPPO khai thác dữ liệu triệt để nhất; ngược lại, khi vai trò bất đối xứng hoặc khi tác nhân có thể rời nhóm giữa chừng, cơ chế attention của MA-POCA mới thực sự cần thiết. Chương 4 kiểm chứng từng khuyến nghị này bằng số liệu thực đo.

2.5 Unity ML-Agents Toolkit

Unity ML-Agents Toolkit [9] là một framework mã nguồn mở do Unity Technologies phát triển, cho phép xây dựng môi trường game và mô phỏng 3D làm nền tảng huấn luyện các agent thông minh bằng học tăng cường. Được giới thiệu lần đầu vào năm 2017 và liên tục cập nhật đến nay, ML-Agents đã trở thành một trong những nền tảng phổ biến nhất cho nghiên cứu RL trong môi trường game nhờ khả năng tích hợp liền mạch giữa engine đồ họa Unity và hệ sinh thái Python của cộng đồng học máy.

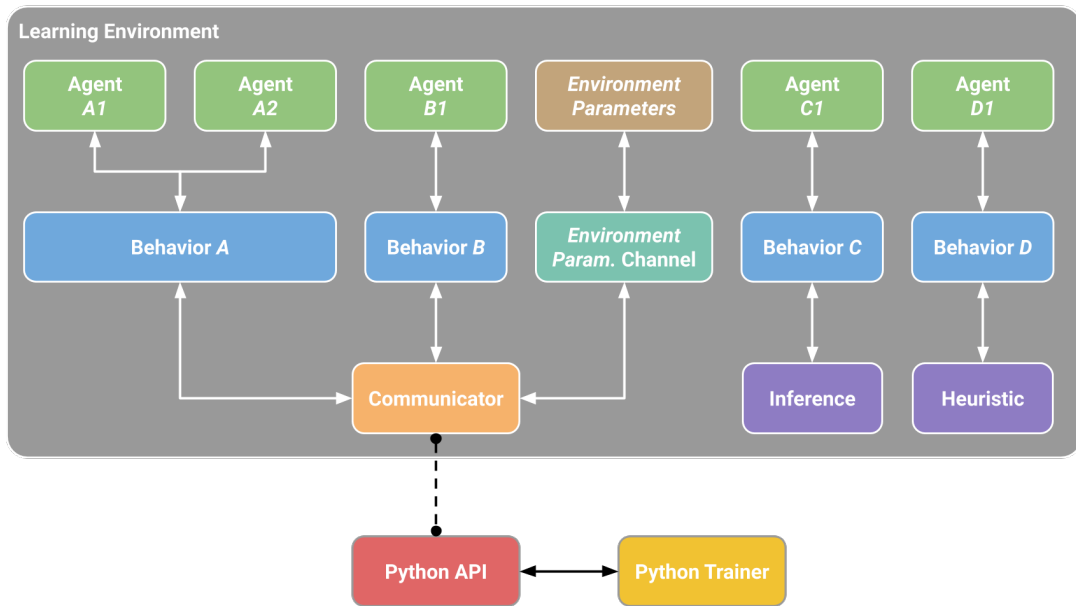
2.5.1 Kiến trúc tổng thể

Kiến trúc của ML-Agents được xây dựng theo mô hình client-server, trong đó Unity Engine đóng vai trò môi trường mô phỏng và Python đóng vai trò backend huấn luyện. Hai thành phần này giao tiếp với nhau thông qua giao thức gRPC qua socket, cho phép chạy đồng thời nhiều instance môi trường Unity để tăng tốc thu thập dữ liệu.

Về phía Unity, thành phần trung tâm là Agent - một MonoBehaviour được gắn vào GameObject trong scene, chịu trách nhiệm thu thập quan sát CollectObservations, nhận hành động OnActionReceived và tính toán phần thưởng. Mỗi Agent được liên kết với một BehaviorParameters component định nghĩa không gian quan sát, không gian hành động và loại hành vi (học hoặc heuristic). Academy là singleton quản lý toàn bộ vòng lặp huấn luyện, đồng bộ hóa các bước thời gian giữa tất cả các agent trong scene và điều phối giao tiếp với Python backend.

Về phía Python, công cụ dòng lệnh mlagents-learn đóng vai trò điểm vào chính của quá trình huấn luyện, tiếp nhận tệp cấu hình YAML và khởi tạo trainer tương ứng với từng behavior khai báo trong đó. Các lớp Trainer, tiêu biểu là PPOTrainer và POCATrainer, chịu trách nhiệm thực thi toàn bộ logic huấn luyện, bao gồm thu thập trajectory, tính toán advantage, cập nhật tham số mạng và lưu checkpoint định kỳ. Bên cạnh lớp trainer, thư viện mlagents_envs còn cung cấp một Python API cấp thấp cho phép tương tác trực tiếp với môi trường Unity mà không cần đi qua trainer tích hợp sẵn.

Hình 2.4 minh họa kiến trúc tổng thể của hệ thống ML-Agents và luồng dữ liệu giữa các thành phần.



Hình 2.4: Kiến trúc tổng thể của Unity ML-Agents Toolkit

2.5.2 Cơ chế Vật lý và Tính xác định trong Unity

Một khía cạnh cốt lõi khi sử dụng Unity cho học tăng cường là sự đồng bộ giữa hệ thống vật lý và quá trình ra quyết định. Trong Unity, các khung hình đồ họa được kết xuất qua hàm `Update()` với tần số biến thiên phụ thuộc vào hiệu năng máy tính (FPS). Tuy nhiên, nếu Agent cũng đưa ra quyết định hoặc tính toán va chạm theo tần số biến thiên này, môi trường sẽ trở nên không xác định (non-deterministic): cùng một chuỗi hành động có thể dẫn đến các kết quả vật lý khác nhau trong những lần chạy khác nhau.

Để giải quyết vấn đề này, ML-Agents đồng bộ hóa toàn bộ quy trình RL vào hàm `FixedUpdate()`. Đây là chu kỳ cập nhật vật lý có tần số cố định (mặc định là 0.02 giây mỗi bước, tương đương 50 lần/giây). Hệ thống vật lý PhysX của Unity sẽ tính toán động lực học, cập nhật tọa độ của các Rigidbody và xử lý va chạm (Collider) hoàn toàn dựa trên các bước thời gian cố định này. Kết quả là, nếu cung cấp cùng một seed ngẫu nhiên ban đầu, môi trường Unity hoạt động như một hệ thống xác định: cùng một chuỗi hành động luôn cho ra cùng một quỹ đạo. Không có tính chất này thì mọi so sánh giữa hai lần chạy đều mất căn cứ.

2.5.3 Hệ thống Cảm biến và Tiền xử lý quan sát

Để Agent hiểu được thế giới 3D phức tạp, ML-Agents cung cấp các thành phần Cảm biến nhằm tự động thu thập và chuyển đổi thông tin không gian thành các vector số thực (Input Layer). Có ba loại cảm biến phổ biến:

- **Cảm biến tia (Ray Perception Sensor 3D):** Bắn ra các tia raycast xung quanh Agent theo một góc quét nhất định. Khi tia chạm vào vật thể có nhãn (Tags) tương ứng, nó sẽ ghi nhận một vector one-hot đại diện cho loại vật thể và một giá trị vô hướng chuẩn hóa $[0, 1]$ biểu diễn khoảng cách. Đây là loại cảm biến đặc biệt hiệu quả và nhẹ về mặt tính toán cho các bài toán điều hướng, được áp dụng chủ đạo trong đề tài này.
- **Grid Sensor:** Chiếu một lưới 2D lên không gian và đếm số lượng vật thể rơi vào từng ô. Phù hợp cho các bài toán cần cái nhìn toàn cảnh như cờ bàn.
- **Camera Sensor:** Sử dụng trực tiếp dữ liệu pixel RGB từ camera, buộc mô hình phải dùng đến mạng tích chập (CNN). Dù cung cấp thông tin toàn diện, nó đòi hỏi chi phí tính toán rất lớn và thường hội tụ chậm hơn nhiều so với cảm biến tia.

2.5.4 Cơ chế Hành vi và Gom nhóm quyết định

Một module quan trọng gắn trên mỗi Agent là BehaviorParameters, nơi định nghĩa kích thước không gian quan sát và không gian hành động. Tại đây, ML-Agents cung cấp cơ chế Action Masking (Mặt nạ hành động) để chặn các hành động không hợp lệ. Ví dụ, nếu Agent đang đứng sát tường, hệ thống có thể “đeo mặt nạ” loại bỏ hành động tiến lên, ép xác suất chọn hành động đó về 0. Điều này giúp thu hẹp không gian tìm kiếm, giúp mô hình hội tụ nhanh hơn thay vì phải tự học cách đâm đầu vào tường và bị phạt.

Ngoài ra, thành phần DecisionRequester cho phép gom nhóm các bước mô phỏng (Decision Period). Thay vì đưa ra một quyết định mới ở mỗi bước FixedUpdate (0.02s), Agent có thể duy trì hành động cũ trong 5 bước liên tiếp (Decision Period = 5). Việc này giúp Agent có đủ thời gian hoàn thành một quỹ đạo vật lý trước khi quyết định tiếp, đồng thời giảm thiểu số lần gọi mạng nơ-ron

lên nhiều lần, tối ưu hóa đáng kể tốc độ huấn luyện. ML-Agents cũng hỗ trợ Heuristic Policy, cho phép người dùng trực tiếp điều khiển Agent bằng bàn phím để gỡ lỗi và trải nghiệm độ khó của môi trường trước khi khởi động AI.

2.5.5 Quy trình huấn luyện Agent

Quy trình huấn luyện một agent trong ML-Agents diễn ra theo vòng lặp lặp lại liên tục cho đến khi đạt số bước tối đa hoặc người dùng dừng thủ công. Tại mỗi bước thời gian, Unity thu thập quan sát từ tất cả các agent đang hoạt động trong scene, gửi dữ liệu sang Python backend thông qua giao thức gRPC. Python trainer xử lý quan sát, suy luận hành động từ policy hiện tại (hoặc lấy mẫu ngẫu nhiên trong giai đoạn khởi động) và trả hành động về Unity. Unity thực thi hành động, cập nhật trạng thái vật lý, tính toán phần thưởng và gửi kết quả trở lại Python để tích lũy vào buffer.

Sau khi buffer tích lũy đủ một batch dữ liệu với kích thước do tham số `buffer_size` quy định, trainer tiến hành cập nhật tham số của mạng nơ-ron. Đối với PPO, mỗi lần cập nhật gồm nhiều epoch chạy trên cùng một batch với hàm mục tiêu clipped surrogate như đã trình bày tại Mục 2.4.1. Khi cập nhật hoàn tất, buffer được xóa để mở đầu một chu kỳ thu thập dữ liệu mới, đồng thời các chỉ số huấn luyện như cumulative reward, episode length và policy loss được ghi lại liên tục và có thể theo dõi theo thời gian thực qua TensorBoard.

Tập cấu hình YAML kiểm soát toàn bộ quá trình huấn luyện. Tập này định nghĩa loại trainer (`trainer_type`), các siêu tham số huấn luyện (`hyperparameters`), kiến trúc mạng nơ-ron (`network_settings`), cấu hình tín hiệu phần thưởng (`reward_signals`) và số bước huấn luyện tối đa (`max_steps`). Mỗi behavior trong scene tương ứng với một khối cấu hình riêng, cho phép huấn luyện đồng thời nhiều loại agent với các thiết lập khác nhau trong cùng một session. Cấu hình chi tiết cho từng thuật toán được trình bày trong Phụ lục A, B và C.

2.5.6 Hỗ trợ thuật toán và giới hạn tích hợp sẵn

Unity ML-Agents tích hợp sẵn hai trainer chính thức: PPO cho bài toán đơn tác nhân và đối kháng (kết hợp self-play), và POCA (MA-POCA) cho bài toán hợp tác đa tác nhân. Cả hai đều là thuật toán on-policy. Bên cạnh đó, ML-Agents cũng cung cấp

trainer SAC như một lựa chọn off-policy, cùng các tín hiệu phần thưởng nội tại như curiosity và RND để hỗ trợ khám phá trong môi trường phần thưởng thưa. Người dùng lựa chọn thuật toán và cấu hình siêu tham số hoàn toàn thông qua tệp YAML mà không cần thay đổi mã nguồn C# của môi trường.

Tuy nhiên, việc tích hợp sẵn này cũng đi kèm một số giới hạn quan trọng đối với đề tài. Cơ chế self-play - vốn cần thiết cho môi trường đối kháng như Football Table - chỉ được hỗ trợ chính thức cho PPO và POCA, mà không hỗ trợ cho SAC. Nguyên nhân là replay buffer của SAC lưu trữ kinh nghiệm từ các phiên bản đối thủ cũ, trở nên lỗi thời và mâu thuẫn khi đối thủ liên tục được cập nhật trong quá trình self-play. Cơ chế group reward phục vụ bài toán hợp tác cũng được thiết kế tối ưu cho POCA; khi áp dụng cho PPO hoặc SAC, phần thưởng nhóm buộc phải được phân bổ thủ công về từng agent, làm suy giảm khả năng gán công lao. Ngoài ra, các trainer tích hợp sẵn không cho phép người dùng dễ dàng can thiệp sâu vào vòng lặp huấn luyện (ví dụ tùy biến hàm mất mát hay lịch trình cập nhật).

Để so sánh công bằng cả ba thuật toán PPO, SAC và MA-POCA trên môi trường đối kháng, đề tài cần một hướng tiếp cận vượt ra ngoài trainer tích hợp sẵn của ML-Agents. Thư viện `mlagents_envs` cung cấp Python Low-Level API cho phép điều khiển trực tiếp môi trường Unity như một môi trường kiểu Gym, từ đó kết nối với các thư viện RL bên ngoài như Stable-Baselines3 [19]. Cách tiếp cận này được trình bày chi tiết trong Mục 3.7.

3. THIẾT KẾ MÔI TRƯỜNG VÀ PHƯƠNG PHÁP THỰC NGHIỆM

Chương này thuật lại quá trình biến ý tưởng của đề tài thành một hệ thống thực nghiệm chạy được. Phần đầu chương dành cho những gì áp dụng chung cho cả ba môi trường: kiến trúc huấn luyện, cách chọn quan sát và hành động, và bốn nguyên tắc thiết kế hàm phần thưởng mà mọi con số trong các bảng về sau đều phải giải thích được bằng chúng. Ba mục kế tiếp đi vào từng môi trường một. Chương khép lại bằng lối đi vòng qua Python Low-Level API - thứ mà đề tài buộc phải dùng đến vì trainer tích hợp sẵn không đáp ứng được - rồi tới thiết lập thực nghiệm và tiêu chí đánh giá.

3.1 Tổng quan thiết kế hệ thống

3.1.1 Kiến trúc huấn luyện tổng thể

Toàn bộ hệ thống thực nghiệm được xây dựng theo kiến trúc client-server của Unity ML-Agents đã trình bày ở Mục 2.5.1. Trong kiến trúc này, Unity đảm nhận vai trò mô phỏng: nó dựng nên thế giới ba chiều, tính toán vật lý va chạm, thu thập quan sát của tác nhân và thực thi các hành động mà tác nhân lựa chọn. Phía Python, thông qua chương trình `mlagents-learn`, đảm nhận vai trò huấn luyện: nó nhận quan sát từ Unity, suy luận ra hành động dựa trên chính sách hiện tại và cập nhật tham số của mạng nơ-ron dựa trên phần thưởng thu về. Hai phía trao đổi dữ liệu với nhau theo từng bước thời gian thông qua giao thức truyền thông nội bộ.

Ba môi trường của đề tài được tổ chức thành ba khu vực độc lập trong cùng một dự án Unity, tương ứng với ba thư mục `Assets/Football`, `Assets/Collaboration` và `Assets/CrossTheRoad`. Mỗi môi trường được gắn với một loại hành vi riêng, lần lượt mang tên `Football`, `PuzzleBehavior` và `CrossTheRoad`. Việc tách biệt này cho phép huấn luyện từng môi trường một cách độc lập mà không gây nhiễu lẫn nhau, đồng thời giúp việc nạp đúng tệp cấu hình cho từng lần chạy trở nên rõ ràng.

Ở mỗi môi trường, toàn bộ nội dung của một lượt chơi được gom vào một khu vực huấn luyện và đóng gói thành một prefab độc lập. Cách tổ chức này cho phép nhân bản khu vực thành nhiều bản sao đặt cạnh nhau trong cùng một cảnh để thu thập nhiều mẫu kinh nghiệm cùng lúc; trong các lần huấn luyện được báo cáo ở Chương 4, mỗi cảnh chỉ chứa đúng một khu vực, nên tốc độ thu thập dữ liệu hoàn toàn đến từ hệ số tăng tốc mô phỏng. Hệ số này được đặt ở mức gấp hai mươi lần thời gian thực, cho phép Unity mô phỏng nhanh hơn nhiều so với tốc độ hiển thị thông thường mà không ảnh hưởng đến tính đúng đắn của vật lý. Toàn bộ chín lần huấn luyện của đề tài được điều phối tự động bằng một chương trình Python trình bày ở Mục 3.7.3.

3.1.2 Không gian quan sát

Trong một hệ thống học tăng cường, cần phân biệt rành mạch giữa trạng thái của môi trường và quan sát mà tác nhân nhận được. Trạng thái là toàn bộ lượng thông

tin mô tả đầy đủ tình huống hiện tại của thế giới mô phỏng, bao gồm cả những đại lượng mà tác nhân không thể trực tiếp cảm nhận; trong khi đó quan sát chỉ là phần thông tin mà tác nhân thực sự tiếp cận được thông qua các cảm biến của nó. Sự phân biệt này là điểm xuất phát cho mọi quyết định thiết kế về sau, vì thế trước hết đề tài phát biểu nó một cách hình thức.

Định nghĩa 3.1 (Không gian quan sát). Cho môi trường có không gian trạng thái S . Không gian quan sát Ω là tập hợp tất cả các quan sát khả dĩ mà tác nhân có thể nhận được. Hàm quan sát $O : S \rightarrow \Omega$ ánh xạ mỗi trạng thái $s \in S$ sang một quan sát $o = O(s)$ mà tác nhân dùng làm căn cứ duy nhất để ra quyết định tại bước thời gian tương ứng.

Khi hàm quan sát O là song ánh, mỗi quan sát tương ứng với đúng một trạng thái, và tác nhân có thể coi như “nhìn thấy” toàn bộ trạng thái; bài toán khi đó là một quá trình quyết định Markov (MDP) đầy đủ như đã trình bày ở Mục 2.2.2. Ngược lại, khi O không song ánh, tức khi nhiều trạng thái khác nhau cùng cho ra một quan sát, tác nhân không thể phân biệt các trạng thái đó và bài toán trở thành một quá trình quyết định Markov quan sát một phần (Partially Observable MDP, viết tắt POMDP). Phần lớn các môi trường trong game, kể cả ba môi trường của đề tài, về bản chất là POMDP: một tác nhân không thể biết chính xác vận tốc tương lai của một phương tiện, cũng như không thể nhìn xuyên qua tường để biết vị trí đồng đội. Nhiệm vụ của người thiết kế, do đó, không phải là cung cấp toàn bộ trạng thái, mà là chọn ra một tập quan sát đủ để bài toán xấp xỉ được tính chất Markov, tức là để quan sát hiện tại một mình đã chứa gần như toàn bộ thông tin cần thiết cho một quyết định hợp lý, mà không buộc tác nhân phải ghi nhớ toàn bộ lịch sử.

Định nghĩa 3.2 (Tính chất Markov của quan sát). Một biểu diễn quan sát được gọi là thỏa mãn tính chất Markov nếu phân phối xác suất của trạng thái kế tiếp và phần thưởng chỉ phụ thuộc vào quan sát và hành động hiện tại, mà không phụ thuộc thêm vào bất kỳ quan sát nào trong quá khứ:

$$\Pr(o_{t+1}, r_{t+1} \mid o_t, a_t, o_{t-1}, a_{t-1}, \dots) = \Pr(o_{t+1}, r_{t+1} \mid o_t, a_t).$$

Trong thực tế, hiếm khi đạt được tính chất Markov một cách tuyệt đối, nên mục tiêu thiết kế là làm cho nó gần đúng nhất có thể với chi phí quan sát thấp nhất. Có

hai kỹ thuật hỗ trợ thường dùng khi một quan sát tức thời chưa đủ Markov. Kỹ thuật chồng khung ghép nối một số quan sát liên tiếp gần nhất thành một vector duy nhất, nhờ đó thông tin về xu hướng chuyển động, chẳng hạn vận tốc suy ra từ hiệu vị trí giữa hai khung, được đưa vào tường minh. Cách còn lại là dùng mạng có bộ nhớ hồi tiếp để tác nhân tự tổng hợp lịch sử. Đề tài chủ yếu dùng chồng khung và bổ sung trực tiếp các đại lượng động như vận tốc vào vector quan sát, nhằm giữ cho mạng chính sách đơn giản và ổn định.

Cả ba môi trường của đề tài đều dùng dạng quan sát vector, tức một chuỗi số thực có độ dài cố định, trong đó mỗi phần tử mã hóa một đại lượng có ý nghĩa vật lý hoặc logic. Ba loại đại lượng thường gặp gồm các đại lượng liên tục như tọa độ, vận tốc, khoảng cách hay góc xoay; các cờ nhị phân biểu diễn trạng thái logic, chẳng hạn một bàn đạp đã được nhấn hay chưa, được mã hóa bằng giá trị không hoặc một; và mã hóa một-nóng dùng khi cần biểu diễn một biến rời rạc có nhiều mức. Nguyên tắc xuyên suốt là chỉ đưa vào những đại lượng tối thiểu nhưng đủ; mỗi phần tử thừa đều làm tăng số chiều đầu vào của mạng và khiến việc học chậm hơn, trong khi mỗi phần tử thiếu có thể phá vỡ tính chất Markov và khiến bài toán trở nên bất khả học.

Để quá trình huấn luyện ổn định, các giá trị quan sát liên tục được chuẩn hóa về lân cận khoảng $[-1, 1]$ trước khi đưa vào mạng nơ-ron. Với một đại lượng x có kỳ vọng μ và độ lệch chuẩn σ ước lượng trực tuyến trong quá trình chạy, giá trị chuẩn hóa là $\hat{x} = (x - \mu)/\sigma$. Việc chuẩn hóa đặc biệt thiết yếu với môi trường Football Table, nơi các đại lượng như vận tốc bóng và góc xoay thanh gạt có thang đo chênh nhau nhiều bậc; nếu để nguyên, những đại lượng giá trị lớn sẽ tạo ra gradient lẫn át và làm sai lệch việc học của các đại lượng nhỏ.

Bên cạnh quan sát vector, hai môi trường Cross The Road và Capture The Flag còn được trang bị cảm biến tia, một cơ chế quan sát không gian mô phỏng thị giác của thực thể trong thế giới ba chiều. Cảm biến phóng ra một chùm gồm nhiều tia từ vị trí tác nhân, trải đều trong một góc quạt phía trước. Mỗi tia dò tìm vật thể đầu tiên mà nó chạm tới trong một tầm xa giới hạn, rồi trả về hai loại thông tin: một là loại vật thể bị chạm, mã hóa một-nóng trên tập các nhãn cần phát hiện, hai là khoảng cách chuẩn hóa từ tác nhân tới điểm chạm. Nếu gọi k là số tia và m là số

nhân vật thể cần phát hiện, thì mỗi tia sinh ra $m + 2$ giá trị, gồm m giá trị một-nóng cho nhân, một giá trị báo tia có chạm gì không, và một giá trị khoảng cách; tổng số chiều quan sát do cảm biến tia đóng góp là $k \times (m + 2)$. Nhờ cảm biến này, một tác nhân trong môi trường Capture The Flag có thể phát hiện được tường, cửa, bàn đạp, điểm đích và cả đồng đội ở trong tầm nhìn, mà không cần được cung cấp tọa độ tuyệt đối của chúng.

3.1.3 Không gian hành động

Tương ứng với quan sát ở đầu vào, hành động là đầu ra mà chính sách của tác nhân sinh ra ở mỗi bước để tác động lên môi trường.

Định nghĩa 3.3 (Không gian hành động). Không gian hành động A là tập hợp tất cả các hành động mà tác nhân được phép thực hiện. Không gian này được gọi là rời rạc nếu A là một tập hữu hạn các lựa chọn, và được gọi là liên tục nếu A là một tập con của không gian số thực nhiều chiều $\mathbb{R}^n$, khi đó mỗi hành động là một vector số thực bị chặn trong một khoảng cho trước.

Sự khác nhau giữa hai dạng không gian hành động dẫn tới hai cách mà mạng chính sách biểu diễn đầu ra. Với không gian rời rạc, mạng xuất ra một phân phối xác suất trên tập hành động và tác nhân lấy mẫu một hành động từ phân phối đó; trường hợp có nhiều nhóm hành động độc lập, ML-Agents dùng dạng đa rời rạc, trong đó mỗi nhóm là một nhánh có phân phối riêng. Với không gian liên tục, mạng thường xuất ra các tham số của một phân phối chuẩn nhiều chiều, gồm kỳ vọng và độ lệch chuẩn cho mỗi chiều, rồi hành động được lấy mẫu và cắt về khoảng hợp lệ. Chính vì cách biểu diễn khác nhau này mà một số thuật toán tỏ ra phù hợp hơn với một dạng không gian nhất định; đặc biệt, SAC được thiết kế nguyên thủy cho điều khiển liên tục và phát huy thế mạnh rõ nhất ở đó.

Đề tài chủ ý lựa chọn ba môi trường phủ cả hai dạng không gian hành động nhằm đánh giá tính linh hoạt của từng thuật toán. Môi trường Football Table dùng không gian hành động liên tục tám chiều: bốn thanh gạt, mỗi thanh gồm một chiều điều khiển lực trượt ngang và một chiều điều khiển lực xoay, mỗi chiều nhận một giá trị thực phản ánh bản chất điều khiển vật lý mượt mà của trò chơi bi lắc. Hai môi trường còn lại dùng không gian hành động rời rạc một nhánh. Cross The Road

có bốn hành động gồm đứng yên, sang trái, sang phải và tiến; Capture The Flag có bảy hành động gồm đứng yên, tiến, lùi, xoay trái, xoay phải, đi ngang sang trái và đi ngang sang phải. Một điểm cần nhấn mạnh là quan hệ giữa hành động được chọn và chuyển động thực tế của tác nhân, vốn khác nhau ở hai môi trường rời rạc. Trong Capture The Flag, mỗi hành động di chuyển không dịch chuyển tác nhân tức thời mà cộng thêm một lượng vận tốc vào thân vật lý của nó, sau đó hệ thống vật lý của Unity mới quyết định chuyển động cuối cùng dưới tác động của lực cản. Ngược lại, trong Cross The Road, mỗi hành động đặt ra một ô đích cách vị trí hiện tại một đơn vị và tác nhân trượt đều tới ô đó, hoàn toàn không đi qua mô phỏng lực. Chi tiết định lượng của hai cơ chế được phân tích trong phần thiết kế của từng môi trường.

3.1.4 Nguyên tắc thiết kế hàm phần thưởng

Hàm phần thưởng là thành phần khó thiết kế nhất trong một hệ thống học tăng cường, bởi nó là tín hiệu duy nhất định hướng hành vi của tác nhân: tác nhân không được cho biết phải làm gì, mà chỉ học cách tối đa hóa tổng phần thưởng chiết khấu mà nó nhận về.

Định nghĩa 3.4 (Hàm phần thưởng). Hàm phần thưởng $R : S \times A \times S \rightarrow \mathbb{R}$ gán cho mỗi phép chuyển trạng thái, khi tác nhân thực hiện hành động a ở trạng thái s và môi trường chuyển sang trạng thái s' , một số thực $r = R(s, a, s')$. Mục tiêu của tác nhân là tối đa hóa kỳ vọng của tổng phần thưởng chiết khấu $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$, với $\gamma \in [0, 1)$ là hệ số chiết khấu.

Trong cả ba môi trường của đề tài, hàm phần thưởng được xây dựng theo cùng một khuôn dạng, phân rã thành ba lớp thành phần có vai trò tách bạch:

$$R(s, a, s') = \underbrace{R_{\text{mt}}(s')}_{\text{thưa, tại cột mốc}} + \underbrace{F(s, s')}_{\text{định hình theo thể năng}} + \underbrace{R_b(s)}_{\text{dày, mỗi bước}}. \quad (3.1)$$

Thành phần mục tiêu R_{mt} chỉ khác không tại một số ít trạng thái đánh dấu kết quả của nhiệm vụ, chẳng hạn bóng vào lưới hoặc cả nhóm tới đích. Đây là thành phần định nghĩa bài toán: nếu chỉ giữ lại một mình nó, chính sách tối ưu vẫn là chính sách mà người thiết kế mong muốn. Nhược điểm của nó là tín hiệu quá thưa: ở giai đoạn đầu, khi tác nhân hầu như chưa bao giờ tình cờ chạm tới cột mốc, gradient chính sách gần như bằng không và quá trình học đình trệ [25].

Thành phần định hình F có nhiệm vụ lấp khoảng trống tín hiệu đó bằng cách cho điểm liên tục theo mức tiến bộ hướng tới mục tiêu. Thành phần theo bước R_b gồm các khoản cộng hoặc trừ nhỏ phát sinh ở mỗi bước, ví dụ phạt thời gian hoặc thưởng cho việc duy trì một trạng thái có ích. Hai thành phần sau làm cho tín hiệu học dày đặc hơn, nhưng cũng chính là nơi phát sinh rủi ro lớn nhất của kỹ thuật thiết kế phần thưởng.

Rủi ro đó là lợi dụng phần thưởng: tác nhân tìm ra một hành vi tối đa hóa tổng phần thưởng mà không hề thực hiện nhiệm vụ mà người thiết kế nhắm tới. Ví dụ kinh điển là thí nghiệm điều khiển xe đạp của Randaløv và Alstrøm [20], trong đó tác nhân được thưởng cho mỗi lần tiến lại gần đích đã học cách chạy vòng tròn quanh điểm xuất phát để liên tục thu phần thưởng tiến gần mà không bao giờ tới nơi. Amodei và cộng sự [1] xếp hiện tượng này vào nhóm các vấn đề an toàn cốt lõi của học máy và chỉ ra rằng nó phát sinh gần như tất yếu mỗi khi hàm phần thưởng chỉ là một đại lượng thay thế cho mục tiêu thật. Trong quá trình thực nghiệm của đề tài, môi trường Capture The Flag đã gặp đúng hiện tượng này ở phiên bản phần thưởng đầu tiên; diễn biến cụ thể cùng cách chẩn đoán được trình bày ở Mục 3.4.6.

Một khuôn khổ lý thuyết cho phép thêm tín hiệu định hướng dày đặc một cách an toàn, tức có bảo đảm toán học rằng lời giải tối ưu không bị thay đổi, là định hình phần thưởng theo thế năng của Ng và cộng sự [15]. Đây là cơ sở lý thuyết chính mà đề tài dựa vào khi đặt các khoản thưởng dẫn đường.

Định nghĩa 3.5 (Định hình phần thưởng theo thế năng). Cho một hàm thế năng $\Phi : S \rightarrow \mathbb{R}$ gán cho mỗi trạng thái một giá trị vô hướng. Phần thưởng định hình được cộng thêm vào phép chuyển từ s sang s' có dạng

$$F(s, s') = \gamma \Phi(s') - \Phi(s).$$

Khi đó chính sách tối ưu của bài toán với phần thưởng $R + F$ trùng đúng với chính sách tối ưu của bài toán gốc với phần thưởng R ; nói cách khác, phần thưởng định hình dạng này không làm thay đổi lời giải tối ưu, mà chỉ định hướng lại quá trình học.

Tính chất bất biến này là thứ biến việc thêm tín hiệu định hướng từ một canh bạc thành một thao tác an toàn: người thiết kế có thể rải tín hiệu dày đặc mà không sợ

làm sai lệch mục tiêu, đồng thời loại trừ luôn khả năng lợi dụng phần thưởng bằng cách đi tới đi lui. Lý do là với mọi chu trình khép kín $s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n = s_0$, tổng phần thưởng định hình thu về khi $\gamma \rightarrow 1$ bằng

$$\sum_{i=0}^{n-1} [\Phi(s_{i+1}) - \Phi(s_i)] = \Phi(s_n) - \Phi(s_0) = 0, \quad (3.2)$$

tức mọi hành vi lặp đi lặp lại đều cho lợi ích ròng bằng không. Chính đẳng thức này loại bỏ được kiểu lợi dụng phần thưởng của thí nghiệm xe đạp nói trên ngay từ trong thiết kế, chứ không phải bằng cách dò tìm hệ số. Một lựa chọn tự nhiên cho hàm thế năng trong các bài toán điều hướng là lấy $\Phi(s) = -d(s)$ với $d(s)$ là khoảng cách từ tác nhân tới mục tiêu, khi đó F thưởng cho mỗi bước làm giảm khoảng cách và phạt cho mỗi bước làm tăng khoảng cách. Nguyên tắc này được đề tài vận dụng trực tiếp trong môi trường Capture The Flag và sẽ được trình bày chi tiết ở Mục 3.4.

Bên cạnh việc gán công lao theo thời gian, các môi trường hợp tác còn đặt ra bài toán gán công lao giữa nhiều tác nhân: khi cả nhóm cùng nhận một phần thưởng chung, làm sao xác định được đóng góp thực sự của từng tác nhân để cập nhật chính sách cho đúng. Đây là bài toán mà cơ chế phần thưởng nhóm và thuật toán MA-POCA [4] được thiết kế để giải quyết, dựa trên ý tưởng cơ sở phản thực của COMA [6]; mối liên hệ giữa thiết kế phần thưởng nhóm và khả năng gán công lao sẽ được làm rõ trong môi trường Capture The Flag.

Tổng hợp các cơ sở lý thuyết trên, đề tài áp dụng thống nhất cho cả ba môi trường bốn nguyên tắc thiết kế sau, và mọi giá trị số trong các bảng phần thưởng ở các mục tiếp theo đều được giải thích bằng chính bốn nguyên tắc này:

- **Bám theo lời giải mẫu.** Trước khi đặt bất kỳ hệ số nào, người thiết kế phải mô tả được một quỹ đạo mẫu hoàn thành nhiệm vụ, rồi kiểm tra rằng không có thành phần phần thưởng nào phạt một trạng thái nằm trên quỹ đạo đó. Nguyên tắc này bắt nguồn từ nhận xét của Amodei và cộng sự [1] rằng phần lớn lỗi thiết kế phần thưởng đến từ việc mô tả sai mục tiêu chứ không từ việc chọn sai hệ số.
- **Ưu tiên sự kiện một lần hoặc thế năng thay vì cộng dồn theo bước.** Một khoản thưởng cộng dồn ở mỗi bước cho một trạng thái nào đó luôn tạo ra

động cơ nấn ná tại trạng thái ấy. Khi cần thưởng cho một cột mốc, đề tài dùng phần thưởng sự kiện chỉ trao một lần trong mỗi lượt chơi; khi cần tín hiệu dẫn đường liên tục, đề tài dùng dạng thể năng ở Định nghĩa 3.5 với bảo đảm (3.2).

- **Giữ trật tự độ lớn.** Ký hiệu $|R_{mt}|$ là độ lớn của phần thưởng mục tiêu và Σ_b là tổng tuyệt đối tối đa mà các khoản theo bước có thể tích lũy trong trọn một lượt chơi, đề tài luôn giữ $\Sigma_b < |R_{mt}|$. Bất đẳng thức này bảo đảm rằng một lượt chơi hoàn thành nhiệm vụ luôn có tổng phần thưởng cao hơn mọi lượt chơi không hoàn thành, bất kể tác nhân xoay xở thế nào với các khoản phụ.
- **Sửa cơ chế môi trường trước khi tăng hệ số phần thưởng.** Khi một hành vi mục tiêu quá khó để khám phá bằng thử - sai, việc tăng hệ số thưởng cho nó không giúp ích, vì tác nhân chưa từng chạm tới hành vi ấy để nhận thưởng. Trong trường hợp đó đề tài điều chỉnh chính cơ chế môi trường hoặc dùng học theo chương trình [3] để hạ độ khó khám phá ban đầu.

Nguyên tắc giữ trật tự độ lớn được cụ thể hóa bằng cách chia các khoản theo bước cho số bước tối đa N của một lượt chơi. Nếu một khoản có giá trị c/N ở mỗi bước thì dù tác nhân duy trì trạng thái đó suốt lượt chơi, tổng thu về cũng chỉ là c ; việc chọn c nhỏ hơn $|R_{mt}|$ do đó lập tức thỏa mãn bất đẳng thức nói trên và làm cho hệ số không còn phụ thuộc vào độ dài lượt chơi. Chi tiết hàm phần thưởng của từng môi trường, cùng lý do đằng sau từng con số, được trình bày trong các mục tương ứng dưới đây.

3.2 Bộ môi trường thực nghiệm

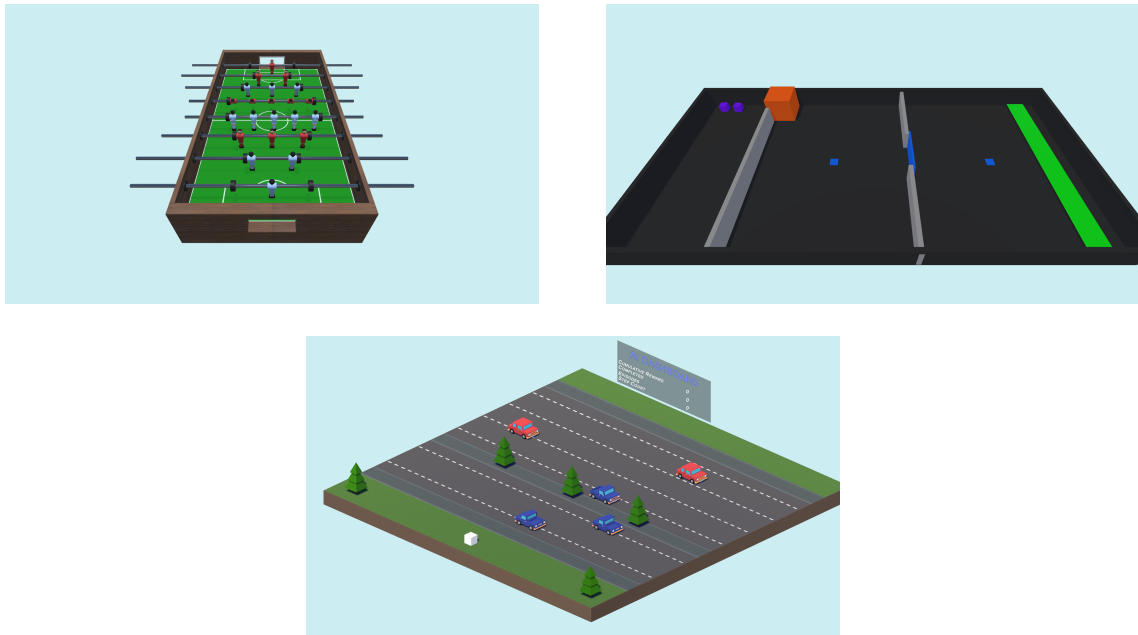
Bộ thực nghiệm của đề tài gồm ba môi trường ba chiều: Football Table, Capture The Flag và Cross The Road. Ba môi trường được chọn để phủ ba dạng bài toán học tăng cường điển hình trong game, lần lượt là đối kháng một đối một, hợp tác đa tác nhân và điều hướng tránh vật cản động, đồng thời phủ cả hai dạng không gian hành động là liên tục và rời rạc. Mục này trình bày những quy ước dùng chung cho cả ba môi trường và một bản tóm lược thống nhất về quan sát, hành động, phần thưởng và điều kiện kết thúc của từng môi trường; các mục tiếp theo đi sâu vào thiết kế của từng môi trường theo cùng một bố cục.

Trừ những chỗ được nói rõ, cả ba môi trường đều tuân theo các quy ước sau. Mỗi môi trường là một cảnh Unity riêng, chứa đúng một khu vực huấn luyện được đóng gói thành prefab để có thể nhân bản khi cần tăng tốc thu thập kinh nghiệm. Mỗi môi trường gắn với đúng một hành vi, nhờ đó một lần huấn luyện chỉ nạp một tệp cấu hình và không gây nhiễu sang môi trường khác. Toàn bộ tác nhân đều quan sát bằng vector số thực, không dùng quan sát hình ảnh; hành động của tác nhân được tính bằng một mạng nơ-ron nhiều lớp kết nối đầy đủ. Mọi lần huấn luyện đều chạy trên cùng một cấu hình máy với hệ số tăng tốc mô phỏng gấp hai mươi lần thời gian thực và bước vật lý cố định 0,02 giây, tức năm mươi bước vật lý cho mỗi giây mô phỏng.

3.2.1 Tích hợp với Unity ML-Agents

Ba môi trường được xây dựng trên gói ML-Agents phiên bản 4.0.2 chạy trong Unity 6000.3, theo kiến trúc mô phỏng - huấn luyện đã trình bày ở Mục 2.5.1. Mỗi loại tác nhân được cài đặt bằng một lớp riêng kế thừa lớp Agent của ML-Agents, trong đó ba hàm quan trọng nhất được viết đè: hàm thu thập quan sát, hàm nhận hành động và hàm khởi tạo lại đầu lượt chơi. Ba lớp tác nhân tương ứng là FootballAgent, PuzzleAgent và CrossTheRoadAgent, gắn với ba hành vi mang tên Football, PuzzleBehavior và CrossTheRoad.

Ngoài quan sát do lớp tác nhân tự cung cấp, hai môi trường Capture The Flag và Cross The Road còn gắn thêm cảm biến tia; phần quan sát do cảm biến sinh ra được ML-Agents ghép tự động vào vector quan sát mà không cần viết thêm mã. Tần suất ra quyết định của mỗi môi trường được đặt riêng qua số bước vật lý giữa hai lần quyết định; ở những bước vật lý nằm giữa hai lần quyết định, hành động cũ được giữ nguyên, nhờ đó chuyển động liên tục và chi phí suy luận của mạng nơ-ron giảm đáng kể. Sau khi huấn luyện, chính sách của mỗi môi trường được xuất ra tệp ONNX và nạp ngược lại vào Unity để chạy suy luận, đây cũng là dạng mô hình mà ứng dụng trình diễn ở Chương 5 sử dụng.



Hình 3.1: Ba môi trường thực nghiệm. Trên trái: Football Table, hai đội Xanh - Đỏ mỗi đội bốn thanh gạt. Trên phải: Capture The Flag, hai tác nhân phối hợp giữ cửa để cùng tới đích. Dưới: Cross The Road, một tác nhân băng qua các làn xe đang chạy

Tác nhân Football Table có tám hành động liên tục và sáu mươi quan sát. Hàm phần thưởng gồm một phần thưởng lớn khi ghi bàn, một hình phạt khi bị thủng lưới và một hình phạt thời gian nhỏ ở mỗi bước để thúc đẩy tác nhân dứt điểm sớm. Điều kiện kết thúc được kích hoạt khi bóng vào một trong hai khung thành, hoặc khi lượt chơi chạm mốc ba nghìn bước, tương ứng khoảng một phút mô phỏng.

Tác nhân Capture The Flag có bảy hành động rời rạc và sáu mươi một quan sát. Hàm phần thưởng gồm một phần thưởng nhóm khi cả hai tác nhân cùng tới đích, một phần thưởng nhóm trao một lần cho mỗi tác nhân khi vượt được cổng đang khóa, một phần thưởng cá nhân định hình theo thể năng khoảng cách tới đích, một phần thưởng nhỏ khi đứng trên bàn đạp và một hình phạt thời gian nhóm ở mỗi bước. Điều kiện kết thúc được kích hoạt khi cả hai tác nhân cùng chạm đích, hoặc khi khu vực huấn luyện chạy đủ một trăm nghìn bước vật lý.

Tác nhân Cross The Road có bốn hành động rời rạc và một trăm ba mươi hai quan sát. Hàm phần thưởng gồm một phần thưởng khi chạm vạch đích và một hình phạt nhỏ khi va chạm, kèm một tín hiệu tò mò cường độ thấp để khuyến khích khám

phá. Điều kiện kết thúc được kích hoạt ngay khi tác nhân chạm vạch đích hoặc va chạm với xe, tường hay cây; lượt chơi không bị giới hạn số bước.

Bảng 3.1 đối chiếu tốc độ huấn luyện thực đo của ba môi trường trên cùng một máy. Chênh lệch giữa chúng rất lớn và bắt nguồn từ hai yếu tố: khối lượng vật lý phải mô phỏng trong mỗi khu vực và số bước vật lý giữa hai lần ra quyết định. Cross The Road chỉ mô phỏng một khối tác nhân trượt theo ô lưới cùng năm chiếc xe chuyển động thẳng nên đạt tốc độ cao nhất. Football Table phải giải các khớp vật lý của tám thanh gạt cùng va chạm liên tục của quả bóng nên chậm hơn gần mười lần. Capture The Flag chậm nhất vì mỗi quyết định của tác nhân trải trên hai mươi bước vật lý và mỗi lượt chơi kéo dài tới một trăm nghìn bước.

Bảng 3.1: Tốc độ huấn luyện thực đo trên ba môi trường, mỗi môi trường lấy một lần chạy đại diện; số bước là số bước huấn luyện của hành vi tương ứng

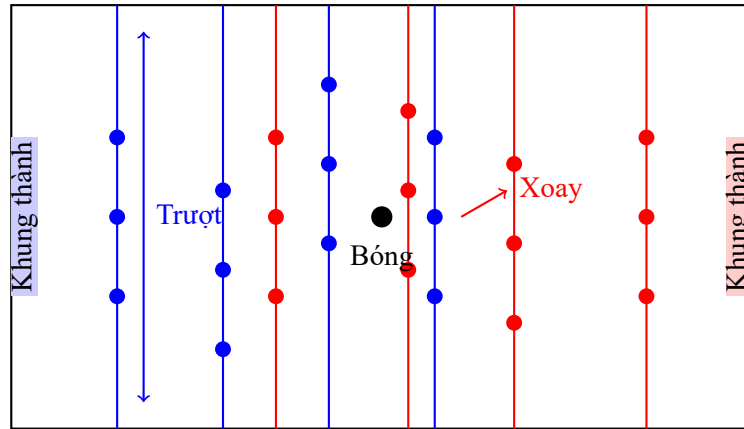
Môi trường	Số bước huấn luyện	Thời gian	Số bước mỗi giây	Quan sát	Hành động
Football Table	1.590.000	1 giờ 41 phút	262	60	8 liên tục
Capture The Flag	1.520.000	8 giờ 19 phút	51	61	7 rời rạc
Cross The Road	2.000.000	14 phút 32 giây	2.294	132	4 rời rạc

3.3 Môi trường Football Table

3.3.1 Mô tả bài toán

Football Table mô phỏng một bàn bi lắc ba chiều với đầy đủ vật lý va chạm giữa quả bóng và các cầu thủ gắn trên thanh gạt. Đây là bài toán đối kháng một đối một: hai đội đối đầu trực tiếp, mỗi đội điều khiển bốn thanh gạt với mục tiêu đưa bóng vào khung thành đối phương đồng thời bảo vệ khung thành của mình. Đặc trưng của bài toán nằm ở tính đối kháng và ở động lực học liên tục của quả bóng: bóng lăn, nảy và đổi hướng liên tục dưới tác động của va chạm, buộc tác nhân phải phản ứng nhanh và phải học cả kỹ năng tấn công lẫn phòng thủ. Chính tính đối kháng khiến đây là trường hợp điển hình để kiểm chứng cơ chế tự chơi, trong đó tác nhân tiến

bộ bằng cách liên tục thi đấu với những phiên bản ngày càng mạnh hơn của chính nó. Bố cục sân và hai bậc tự do của mỗi thanh gạt được thể hiện trong Hình 3.2.



Hình 3.2: Sơ đồ bố cục sân và cơ chế điều khiển thanh gạt trong Football Table

3.3.2 Không gian quan sát

Vector quan sát của mỗi đội gồm sáu mươi giá trị, chia thành ba nhóm bổ sung cho nhau. Mười hai giá trị mô tả trạng thái quả bóng, gồm ba giá trị vận tốc đã nén về khoảng $[-1, 1]$ và chín giá trị mã hóa vị trí bóng. Ba mươi hai giá trị tiếp theo mô tả trạng thái đội nhà: với mỗi thanh gạt, tác nhân quan sát vận tốc trượt, vận tốc xoay, vị trí trượt và góc xoay hiện tại, tức đủ để biết chính xác tư thế của toàn bộ hàng cầu thủ của mình. Mười sáu giá trị còn lại mô tả trạng thái đội đối phương với bốn đại lượng tương tự cho mỗi thanh gạt, cho phép tác nhân dự đoán ý đồ và phản ứng của đối thủ.

Hai chi tiết trong cách mã hóa quan sát đáng được nhấn mạnh. Kỹ thuật tách chữ số thập phân thay việc đưa vào mạng một giá trị duy nhất cho vị trí bóng hay góc xoay thanh gạt bằng cách tách giá trị đó thành nhiều thành phần ứng với các hàng thập phân liên tiếp. Nhờ vậy, một thay đổi rất nhỏ ở hàng thập phân sâu vẫn tạo ra biến thiên rõ rệt ở một thành phần đầu vào, giúp mạng nhạy hơn với các sai lệch tinh vi về vị trí, vốn quyết định thành bại của một cú chạm bóng. Phép đối xứng hóa quan sát thì đưa mọi vị trí và vận tốc về hệ tọa độ địa phương của sân rồi nhân với một hệ số dấu tùy theo phía của đội. Sau phép biến đổi này, cả hai phía đều thấy khung thành đối phương ở cùng một hướng quy ước, nhờ đó một chính sách duy nhất vận hành được cho cả hai phía và cơ chế tự chơi trở nên nhất quán.

vì hai đội thực sự dùng chung một mạng nơ-ron.

Do các đại lượng vật lý trong môi trường này có thang đo chênh nhau nhiều bậc, chế độ chuẩn hóa quan sát được bật trong cấu hình huấn luyện, theo đúng nguyên tắc ở Mục 3.1.2.

3.3.3 Không gian hành động và cơ chế điều khiển

Mỗi đội được điều khiển bởi một tác nhân duy nhất, quản lý đồng thời cả bốn thanh gạt. Tại mỗi bước quyết định, tác nhân xuất ra tám giá trị thực trong khoảng $[-1, 1]$; với mỗi thanh gạt có một giá trị điều khiển chuyển động trượt ngang để đưa cầu thủ đến đúng vị trí và một giá trị điều khiển chuyển động xoay để thực hiện cú đá:

$$a = (f_1^{\text{trượt}}, f_1^{\text{xoay}}, \dots, f_4^{\text{trượt}}, f_4^{\text{xoay}}) \in [-1, 1]^8. \quad (3.3)$$

Việc gộp cả đội vào một tác nhân duy nhất, thay vì mỗi thanh gạt một tác nhân, có hai lợi ích. Cách tổ chức này biến bài toán phối hợp giữa bốn thanh gạt thành một bài toán điều khiển tập trung mà một chính sách duy nhất học được, tránh sự phức tạp của việc đồng bộ nhiều tác nhân độc lập. Nó cũng phù hợp trực tiếp với cơ chế tự chơi, vì khi đó mỗi phía của bàn chỉ cần đúng một chính sách để thi đấu.

Về mặt vật lý, mỗi giá trị hành động không dịch chuyển thanh gạt tức thời mà được diễn giải thành một mức thay đổi vận tốc trượt hoặc vận tốc xoay đặt lên thân vật lý của thanh gạt tương ứng, với hệ số lần lượt là một và hai mươi lăm; hệ thống vật lý của Unity mới quyết định chuyển động cuối cùng cùng va chạm giữa thanh gạt và quả bóng. Tác nhân ra quyết định sau mỗi hai bước vật lý, tức hai mươi lăm quyết định mỗi giây mô phỏng, và hành động được giữ nguyên ở bước vật lý xen giữa. Chính tính liên tục này khiến Football Table trở thành phép thử tiêu biểu cho các thuật toán điều khiển liên tục, khác hẳn hai môi trường rời rạc còn lại.

3.3.4 Hàm phần thưởng

Hàm phần thưởng của môi trường được tổng hợp trong Bảng 3.2. Chiếu theo khuôn dạng (3.1), môi trường này chỉ dùng hai lớp: một thành phần mục tiêu thừa tại trạng thái kết thúc, gồm cộng năm điểm khi ghi bàn và trừ một điểm khi bị thủng lưới, cùng một thành phần theo bước là hình phạt thời gian bằng $1/3000$ điểm ở mỗi bước. Thành phần định hình theo thể năng không được sử dụng, vì lý do trình bày

ở cuối mục này.

Việc đặt phần thưởng ghi bàn lớn gấp năm lần hình phạt thùng lưới không phải một lựa chọn tùy ý mà xuất phát từ đặc thù của bài toán đối kháng: với hàm phần thưởng đối xứng, chính sách an toàn nhất là giữ bóng ở phần sân nhà và không bao giờ dâng lên, bởi mọi pha tấn công đều mở ra nguy cơ phản công. Có thể lượng hóa nhận định này. Xét một tình huống mà tác nhân có thể chọn giữa cầm cự, tức giữ bóng đến hết lượt chơi, và dứt điểm, tức tổ chức một pha tấn công thành công với xác suất p và bị phản công thùng lưới với xác suất $1 - p$. Gọi R_+ là phần thưởng ghi bàn, R_- là hình phạt thùng lưới và bỏ qua hình phạt thời gian trong nhánh dứt điểm vì lượt chơi kết thúc sớm, giá trị kỳ vọng của hai lựa chọn là

$$V_{\text{cầm cự}} = -T \cdot \frac{1}{3000} = -1, \quad V_{\text{dứt điểm}} = p R_+ - (1 - p) R_-, \quad (3.4)$$

với $T = 3000$ là số bước tối đa của một lượt chơi. Lối chơi tấn công chỉ được ưa chuộng khi $V_{\text{dứt điểm}} > V_{\text{cầm cự}}$, tức khi

$$p > \frac{R_- - 1}{R_+ + R_-}. \quad (3.5)$$

Với cấu hình đối xứng $R_+ = R_- = 5$, ngưỡng này bằng 0,4: tác nhân chỉ chịu tấn công khi tự tin thắng ít nhất bốn mươi phần trăm số pha bóng, một điều gần như không bao giờ đúng ở giai đoạn đầu huấn luyện, nên chính sách hội tụ về phòng ngự tuyệt đối. Với cấu hình được chọn $R_+ = 5$ và $R_- = 1$, ngưỡng tụt xuống đúng bằng không: mọi cơ hội ghi bàn dù nhỏ đến đâu cũng đáng để thử. Đây chính là lý do của tỉ lệ năm trên một, và nó cũng cho thấy hình phạt thời gian không phải một chi tiết phụ mà là thành phần quyết định dấu của vế phải trong (3.5).

Giá trị $1/3000$ của hình phạt thời gian được chọn theo nguyên tắc giữ trật tự độ lớn ở Mục 3.1.4: nhân với $T = 3000$ bước, tổng phạt tích lũy trọn một lượt chơi bằng đúng một điểm, tức bằng đúng hình phạt thùng lưới. Cách chuẩn hóa này tạo ra một hệ quả có chủ đích: một lượt chơi hòa không bàn thắng bị đánh giá ngang bằng với việc để thùng lưới một bàn, nên tác nhân không thể coi thế trận bế tắc là một kết quả chấp nhận được. Đồng thời tổng phạt vẫn nhỏ hơn nhiều phần thưởng ghi bàn, nên nó không bao giờ lấn át được mục tiêu chính.

Bảng 3.2: Hàm phần thưởng của môi trường Football Table

Sự kiện	Giá trị	Loại
Ghi bàn vào khung thành đối phương	+5,0	Thưởng mục tiêu
Bị thủng lưới	-1,0	Phạt mục tiêu
Mỗi bước trôi qua	-1/3000	Phạt thời gian

Mã nguồn của tác nhân còn cài đặt sẵn hai thành phần định hướng tùy chọn: một phần thưởng tỉ lệ với thành phần vận tốc của bóng chiếu lên hướng nối tới khung thành đối phương, chỉ kích hoạt khi tác nhân đang kiểm soát bóng, và một hình phạt khi thanh gạt xoay quá mạnh. Trong toàn bộ các lần huấn luyện được báo cáo ở Chương 4, hai hệ số này đều được đặt bằng không, tức hai thành phần không tham gia vào tín hiệu học. Quyết định này có ba căn cứ.

Phần thưởng theo vận tốc bóng không có dạng thế năng. Nó phụ thuộc vào vận tốc, tức một đại lượng của phép chuyển chứ không phải một hàm của riêng trạng thái, nên không thể viết thành $\gamma \Phi(s') - \Phi(s)$ và do đó nằm ngoài phạm vi bảo đảm bất biến của Định nghĩa 3.5. Hệ quả trực tiếp là đẳng thức (3.2) không còn đúng: tác nhân hoàn toàn có thể đẩy bóng tới lui theo hướng khung thành đối phương để tích lũy phần thưởng vô hạn mà không bao giờ dứt điểm, đúng theo cơ chế đã quan sát được trong thí nghiệm xe đạp của Randløv và Alstrøm [20].

Hình phạt xoay mạnh vi phạm nguyên tắc bám theo lời giải mẫu. Một cú sút mạnh bắt buộc phải đi kèm vận tốc xoay lớn của thanh gạt, nên khoản phạt này giáng đúng vào hành động nằm trên quỹ đạo hoàn thành nhiệm vụ; giữ nó ở mức khác không sẽ đẩy tác nhân về lối chơi rê bóng chậm chạp thay vì dứt điểm.

Cuối cùng, cơ chế tự chơi đã tự cung cấp một chương trình học có độ khó tăng dần mà không cần bất kỳ tín hiệu định hướng thủ công nào: đối thủ luôn mạnh tương đương với tác nhân ở thời điểm hiện tại, nên tỉ lệ thắng thua giữ ở mức cân bằng và tín hiệu thua vẫn xuất hiện đủ thường xuyên để việc học tiến triển. Đây chính là lập luận đã được kiểm chứng ở quy mô lớn hơn nhiều trong AlphaGo Zero [24], nơi một chính sách mạnh hơn con người được học chỉ từ tín hiệu thắng thua ở cuối ván mà không có bất kỳ phần thưởng trung gian nào. Giữ nguyên hai thành phần

trong mã nguồn cho phép bật lại khi cần khảo sát ảnh hưởng của phần thưởng định hướng.

3.3.5 Điều kiện kết thúc lượt chơi

Một lượt chơi kết thúc khi có bàn thắng, tức bóng đi vào một trong hai khung thành; khi đó cả hai tác nhân nhận phần thưởng tương ứng rồi lượt chơi được đặt lại, bóng và các thanh gạt trở về vị trí ban đầu. Lượt chơi cũng kết thúc khi chạm mốc ba nghìn bước, tương ứng khoảng một phút mô phỏng; giới hạn này ngăn những thế trận bế tắc kéo dài vô hạn và bảo đảm dữ liệu huấn luyện luôn được làm mới.

3.3.6 Cấu hình huấn luyện

Do bản chất đối kháng và không gian hành động liên tục, môi trường này được huấn luyện chính bằng PPO kết hợp cơ chế tự chơi. Mạng chính sách gồm hai lớp ẩn với hai trăm năm mươi sáu đơn vị mỗi lớp và bật chuẩn hóa quan sát. Cơ chế tự chơi được cấu hình để lưu lại một phiên bản đối thủ sau mỗi năm mươi nghìn bước, đổi phía sau mỗi một trăm nghìn bước, giữ một cửa sổ mười đối thủ gần nhất với xác suất đấu với phiên bản mới nhất là một nửa, và theo dõi tiến bộ qua chỉ số xếp hạng khởi tạo ở mức một nghìn hai trăm. Số bước huấn luyện tối đa được đặt ở mười triệu bước. Hai thuật toán còn lại được đưa vào so sánh là SAC và MA-POCA, huấn luyện dưới cùng điều kiện để bảo đảm mặt bằng so sánh công bằng. Cấu hình siêu tham số đầy đủ được liệt kê trong các Phụ lục A, B và C.

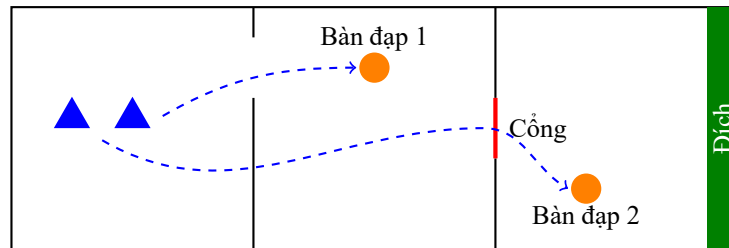
3.4 Môi trường Capture The Flag

3.4.1 Mô tả bài toán

Capture The Flag là một câu đố không gian mang tính hợp tác, trong đó hai tác nhân phải phối hợp chặt chẽ để cùng về đích. Điểm mấu chốt là nhiệm vụ được thiết kế sao cho không một tác nhân nào có thể tự mình hoàn thành: để đi từ khu vực xuất phát tới đích, cả hai bắt buộc phải thực hiện một màn bàn giao, trong đó lần lượt mỗi tác nhân giữ cửa mở cho tác nhân kia đi qua. Sự phụ thuộc lẫn nhau này biến bài toán thành trường hợp điển hình để làm nổi bật thách thức gán công lao trong học tăng cường hợp tác, tức việc xác định đóng góp thực sự của từng tác nhân vào thành quả chung của cả nhóm; đây cũng chính là dạng bài toán mà

MA-POCA được thiết kế để giải quyết.

Bố cục không gian, thể hiện trong Hình 3.3, là một hành lang dài trải theo một trục, được chia thành ba khoang bởi hai bức tường ngăn có khe hở lệch nhau. Khoang đầu tiên là phòng xuất phát, nơi hai tác nhân và một khối vật cản được đặt ban đầu; lối ra khỏi khoang này là một khe hở nằm lệch về một bên của bức tường ngăn thứ nhất. Khoang ở giữa chứa bàn đạp áp lực thứ nhất. Ngăn cách khoang giữa với khoang cuối là một cổng cửa nằm ở khe hở chính giữa của bức tường ngăn thứ hai; cổng gồm hai cánh cửa có thể hạ xuống để mở và nâng lên để chặn. Khoang cuối chứa bàn đạp áp lực thứ hai và, ở tận cùng, là vạch đích trải suốt chiều rộng hành lang. Điểm thiết kế cốt lõi là cả hai bàn đạp đều điều khiển chung một cổng cửa, và chúng nằm ở hai phía đối diện của cổng: bàn đạp thứ nhất ở phía trong, cùng khoang với điểm xuất phát; bàn đạp thứ hai ở phía ngoài, đã qua cổng. Chính sự bố trí đối xứng này tạo nên cấu trúc bàn giao mà lời giải bắt buộc phải tuân theo.



Hình 3.3: Sơ đồ bố cục không gian và trình tự bàn giao trong Capture The Flag

3.4.2 Không gian quan sát

Quan sát của mỗi tác nhân gồm sáu mươi một giá trị đến từ ba nguồn bổ sung cho nhau. Bốn giá trị đầu là các cờ nhị phân mang tính logic: bàn đạp thứ nhất đã được nhấn hay chưa, bàn đạp thứ hai đã được nhấn hay chưa, bản thân tác nhân đã vượt qua cổng hay chưa, và đồng đội đã vượt qua cổng hay chưa. Hai cờ đầu cho tác nhân biết cửa hiện có đang được giữ mở hay không mà không cần nhìn thấy cổng; hai cờ sau chính là mấu chốt để một tác nhân biết khi nào nên rời bàn đạp, bởi chỉ khi đồng đội đã sang bên kia thì việc nhả bàn đạp mới hợp lý. Nếu thiếu cờ trạng thái của đồng đội, chính sách phi tập trung không có căn cứ trực tiếp để định thời điểm bàn giao và bài toán trở nên khó học hơn nhiều. Kèm theo đó là một cờ nhị phân riêng ghi nhận tác nhân đã chạm đích hay chưa.

Năm mươi sáu giá trị còn lại đến từ cảm biến tia mô tả ở Mục 3.1.2. Cấu hình cụ thể gồm bảy tia trải trong góc quạt bảy mươi độ phía trước, tầm xa hai mươi đơn vị, nhận diện sáu nhãn vật thể là đồng đội, tường, khối vật cản, bàn đạp, cửa và đích. Mỗi tia sinh ra sáu giá trị mã hóa một-nóng cho nhãn, một giá trị báo tia có chạm vật thể nào không và một giá trị khoảng cách chuẩn hóa, tức tám giá trị cho mỗi tia. Nhờ cảm biến này, tác nhân nhận biết được cấu trúc không gian xung quanh cùng vị trí tương đối của đồng đội trong tầm nhìn mà không cần được cung cấp tọa độ tuyệt đối. Sự kết hợp giữa các cờ logic tường minh và quan sát không gian cho phép tác nhân vừa suy luận về trạng thái phối hợp của cả nhóm, vừa điều hướng chính xác trong hành lang hẹp.

3.4.3 Không gian hành động và cơ chế điều khiển

Mỗi tác nhân điều khiển một khối hộp có thân vật lý, di chuyển trong không gian ba chiều bằng bảy hành động rời rạc: đứng yên, tiến, lùi, xoay trái, xoay phải, đi ngang sang trái và đi ngang sang phải. Tập hành động này đủ phong phú để tác nhân vừa di chuyển linh hoạt vừa định hướng chính xác khi cần đứng vào đúng vị trí bàn đạp. Đúng như đã nêu ở Mục 3.1.3, mỗi lệnh di chuyển không dịch chuyển tác nhân tức thời mà cộng thêm một lượng vận tốc bằng 1,4 đơn vị trên giây vào thân vật lý theo hướng tương ứng ở mỗi bước vật lý, trong khi hệ số cản tuyến tính được đặt ở mức bốn để liên tục hãm vận tốc lại. Hai lệnh xoay làm tác nhân quay quanh trục đứng với tốc độ hai trăm độ mỗi giây. Tác nhân ra quyết định sau mỗi hai mươi bước vật lý, tức hai phẩy năm quyết định mỗi giây mô phỏng, và hành động được giữ nguyên ở các bước vật lý xen giữa.

Sự cân bằng giữa lượng vận tốc cộng thêm và hệ số cản làm cho tác nhân có một vận tốc tối đa hữu hạn thay vì tăng tốc vô hạn. Ở trạng thái dừng, vận tốc tối đa đạt được là

$$v_{\max} = \frac{\Delta v}{c \cdot \Delta t} = \frac{1,4}{4 \times 0,02} = 17,5 \text{ đơn vị/giây}, \quad (3.6)$$

với Δv là lượng vận tốc cộng thêm mỗi bước, c là hệ số cản và Δt là độ dài một bước vật lý. Ngoài ý nghĩa vật lý thuần túy, giá trị này còn là yếu tố then chốt bảo đảm tính đúng đắn của cơ chế bàn giao, như sẽ phân tích ngay dưới đây.

3.4.4 Cơ chế cửa và pha bàn giao

Cổng cửa chỉ mở khi có tác nhân đang đứng trên một trong hai bàn đạp. Tuy nhiên, nếu cửa đóng lại ngay lập tức vào đúng khoảnh khắc tác nhân rời bàn đạp thì màn bàn giao gần như không thể khám phá được bằng thử - sai, vì nó đòi hỏi một sự đồng bộ thời gian hoàn hảo. Để hành vi hợp tác trở nên khả học, đề tài đưa vào một khoảng ân hạn: sau khi tác nhân rời khỏi bàn đạp, cửa vẫn giữ mở thêm một giây mô phỏng rồi mới đóng. Khoảng ân hạn này tạo ra một cửa sổ thời gian đủ để đồng đội kịp lên giữ bàn đạp còn lại, qua đó làm mượt quá trình học mà không làm thay đổi bản chất bài toán.

Một câu hỏi thiết kế quan trọng là liệu khoảng ân hạn có vô tình cho phép một tác nhân tự mình vượt cổng hay không, tức bước lên bàn đạp thứ nhất rồi chạy thật nhanh qua cổng trước khi cửa kịp đóng, khiến sự hợp tác trở nên không cần thiết. Có thể loại trừ khả năng này bằng một lập luận định lượng. Khoảng cách từ bàn đạp thứ nhất tới cổng là mười chín đơn vị, trong khi khoảng ân hạn chỉ dài một giây. Ngay cả khi giả định tác nhân đã chạy sẵn ở vận tốc tối đa 17,5 đơn vị trên giây theo công thức (3.6), quãng đường nó đi được trong một giây vẫn nhỏ hơn mười chín đơn vị. Trên thực tế tác nhân xuất phát từ trạng thái đứng yên trên bàn đạp và phải mất thời gian tăng tốc, nên quãng đường thực tế chỉ khoảng mười ba phẩy hai đơn vị, tức chưa tới bảy mươi phần trăm khoảng cách cần vượt. Do đó không tồn tại quỹ đạo nào giúp một tác nhân đơn độc vượt cổng trong khoảng ân hạn. Kết luận này bảo đảm rằng mọi lần vượt cổng thành công đều là bằng chứng của sự hợp tác thực sự, chứ không phải của một lối tắt cá nhân; nó cũng là cơ sở để diễn giải đúng các chỉ số phần thưởng ở Chương 4.

3.4.5 Hàm phần thưởng

Đây là môi trường có hàm phần thưởng phức tạp nhất của đề tài, và cũng là môi trường duy nhất phải trải qua một vòng thiết kế lại sau khi phiên bản đầu tiên thất bại. Mục này trình bày thiết kế cuối cùng cùng lý do của từng thành phần; Mục 3.4.6 thuật lại quá trình đi tới thiết kế đó.

Điểm xuất phát của toàn bộ thiết kế là lời giải mẫu, theo đúng nguyên tắc bám theo lời giải mẫu ở Mục 3.1.4. Với bố cục đã mô tả, một lượt chơi thành công bắt

buộc phải đi qua chuỗi trạng thái sau: hai tác nhân rời phòng xuất phát; một tác nhân bước lên bàn đạp phía trong và đứng yên giữ cửa; tác nhân còn lại vượt cổng; tác nhân vừa vượt cổng bước lên bàn đạp phía ngoài để giữ cửa ngược lại; tác nhân đang đứng ở bàn đạp phía trong rời chỗ và vượt cổng; cuối cùng cả hai cùng tiến tới vạch đích. Mọi thành phần phần thưởng đều được kiểm tra đối chiếu với chuỗi này, và yêu cầu bắt buộc là không thành phần nào được phạt một trạng thái nằm trên chuỗi. Yêu cầu tưởng chừng hiển nhiên này chính là điều mà phiên bản phần thưởng đầu tiên vi phạm.

Phần thưởng mục tiêu. Khoản lớn nhất, bằng một điểm, chỉ được trao khi cả hai tác nhân cùng đến được đích, và được trao cho nhóm chứ không cho cá nhân. Việc chọn phạm vi nhóm là một quyết định có hệ quả sâu xa: nó khiến hàm giá trị mà mạng phê bình phải học trở thành giá trị của cả đội, nhờ đó một tác nhân đứng yên giữ bàn đạp vẫn được ghi nhận công lao khi đồng đội về đích. Nếu trao phần thưởng theo cá nhân, tác nhân giữ cửa sẽ không nhận được gì cho hành vi hy sinh của mình và chính sách sẽ nhanh chóng học cách bỏ mặc đồng đội. Chính cấu hình phần thưởng nhóm này là điều kiện để các thuật toán gán công lao đa tác nhân như MA-POCA [4] phát huy tác dụng, vì phê bình tập trung của chúng được thiết kế để phân rã một phần thưởng chung thành đóng góp của từng tác nhân [6].

Phần thưởng định hình theo khoảng cách. Thành phần dẫn đường được xây dựng đúng theo khuôn khổ ở Định nghĩa 3.5, với hàm thế năng lấy bằng đối số của khoảng cách từ tác nhân tới đích, $\Phi(s) = -d(s)$. Phần thưởng cộng thêm ở mỗi bước do đó có dạng

$$F(s_{t-1}, s_t) = \kappa [d(s_{t-1}) - d(s_t)], \quad \kappa = 0,01, \quad (3.7)$$

tức thưởng cho mỗi bước rút ngắn khoảng cách và phạt cho mỗi bước nới rộng nó. Cài đặt dùng hiệu thế năng thuần túy thay vì dạng chiết khấu $\gamma \Phi(s') - \Phi(s)$; với $\gamma = 0,99$ trong cấu hình huấn luyện, sai khác giữa hai dạng là $(1 - \gamma)\Phi$, nhỏ hơn hai bậc so với chính tín hiệu dẫn đường, nên tính bất biến chính sách vẫn được bảo toàn ở mức xấp xỉ có thể chấp nhận [5]. Ba hệ quả của lựa chọn này đáng được nêu rõ. Tín hiệu dẫn đường phủ kín toàn bộ hành lang, kể cả những đoạn không có cột mốc nào, nên tác nhân không bao giờ rơi vào vùng hoàn toàn không có gradient học. Đẳng thức (3.2) bảo đảm mọi hành vi đi tới đi lui đều cho lợi ích ròng bằng

không, nên thành phần này không thể bị lợi dụng. Và quan trọng nhất đối với bài toán hợp tác, một tác nhân đứng yên giữ bàn đạp có $d(s_t) = d(s_{t-1})$ nên không bị thành phần này phạt, tức hành vi hy sinh nằm trên lời giải mẫu được giữ trung lập thay vì bị trừng phạt.

Tổng phần thưởng định hình mà một tác nhân có thể thu về trong trọn một lượt chơi được xác định hoàn toàn bởi tính chất khớp nối của tổng (3.2):

$$\sum_{t=1}^T F(s_{t-1}, s_t) = \kappa [d(s_0) - d(s_T)] \leq \kappa d(s_0), \quad (3.8)$$

tức chỉ phụ thuộc vào khoảng cách ban đầu chứ không phụ thuộc vào độ dài lượt chơi hay vào quỹ đạo mà tác nhân chọn. Hệ số $\kappa = 0,01$ được chọn theo đúng ràng buộc này: để thành phần dẫn đường không lấn át phần thưởng mục tiêu, cần $\kappa d(s_0) < 1$, tức $d(s_0) < 100$ đơn vị, điều mà chiều dài hành lang của bố cục hiện tại thỏa mãn với biên độ an toàn rộng rãi.

Phần thưởng bàn giao. Thành phần hợp tác then chốt được trao mỗi khi một tác nhân vừa vượt qua cổng đang khóa: cả nhóm được cộng nửa điểm, và mỗi tác nhân chỉ nhận khoản này một lần duy nhất trong mỗi lượt chơi. Ba chi tiết trong định nghĩa trên đều có lý do. Điều kiện cổng đang khóa, kết hợp với kết luận định lượng ở Mục 3.4.4 rằng không tồn tại quỹ đạo giúp một tác nhân đơn độc vượt cổng, khiến sự kiện kích hoạt phần thưởng tương đương logic với sự kiện hợp tác thành công; nhờ vậy tín hiệu không phải một đại lượng thay thế gần đúng cho hành vi mong muốn mà là chỉ báo chính xác của nó, tránh được đúng cái bẫy mà Amodei và cộng sự [1] cảnh báo. Phạm vi nhóm bảo đảm tác nhân đang giữ bàn đạp cũng được ghi nhận công lao cho cú vượt cổng của đồng đội. Còn đặc tính một lần khiến khoản thưởng không thể bị khai thác bằng cách lặp lại động tác ra vào cổng, khác hẳn một khoản cộng dồn theo từng bước; đây chính là bài học rút ra từ phiên bản thiết kế đầu tiên.

Các khoản theo bước. Một khoản thưởng nhỏ được trao cho tác nhân đang đứng trên bàn đạp, và một khoản tương tự cho việc đẩy khối vật cản. Hai khoản này có ý đi ngược nguyên tắc ưu tiên sự kiện một lần, vì lý do khám phá: giữ bàn đạp là một hành vi phải được duy trì theo thời gian nên không thể diễn đạt bằng một sự kiện tức thời, và nếu không có tín hiệu nào cho nó thì xác suất để tác nhân

tình cờ đứng yên đủ lâu trên bàn đạp là quá nhỏ. Rủi ro nấn ná được không chế bằng cách chuẩn hóa theo độ dài lượt chơi: khoản thưởng bằng $0,5/N$ mỗi bước, nên ngay cả khi tác nhân đứng lì trên bàn đạp suốt lượt chơi, tổng thu về cũng chỉ đúng 0,5 điểm, thấp hơn hẳn mức 1,5 điểm mà nó nhận được nếu tham gia trọn vẹn một lời giải bàn giao. Cuối cùng, một khoản phạt thời gian nhóm bằng $0,1/N$ mỗi bước thúc đẩy nhóm hoàn thành nhanh hơn; giá trị này được giữ rất nhỏ để tổng phạt trọn lượt chơi chỉ là 0,1 điểm, đủ để phá thế hòa giữa hai chính sách cùng hoàn thành nhiệm vụ nhưng không đủ để chi phối tín hiệu học, một sai lầm mà phiên bản đầu tiên đã mắc phải.

Bảng 3.3: Hàm phần thưởng của môi trường Capture The Flag

Sự kiện	Giá trị	Phạm vi
Cả hai tác nhân đến đích	+1,0	Nhóm
Vượt được cổng đang khóa (một lần mỗi tác nhân)	+0,5	Nhóm
Giảm khoảng cách tới đích (định hình thể năng)	$0,01 \times \Delta d$	Cá nhân
Đứng trên bàn đạp áp lực	$+0,5/N$ mỗi bước	Cá nhân
Đẩy khối vật cản	$+0,25/N$ mỗi bước	Cá nhân
Phạt thời gian mỗi bước	$-0,1/N$	Nhóm

trong đó Δd là mức giảm khoảng cách từ tác nhân tới đích giữa hai bước liên tiếp, và $N = 100\,000$ là số bước tối đa của một lượt chơi, dùng để giữ tổng phần thưởng tích lũy ở một thang đo hợp lý.

Đặt cạnh nhau, các thành phần trên tạo thành một trật tự độ lớn được kiểm soát chặt chẽ. Một lượt chơi hoàn thành trọn vẹn mang lại cho nhóm một điểm cho mục tiêu cuối, cộng tối đa một điểm cho hai lần vượt cổng, cộng phần định hình bị chặn trên bởi $\kappa d(s_0)$; trong khi toàn bộ các khoản theo bước, dù tác nhân xoay xở thế nào, cũng chỉ nằm trong khoảng từ $-0,1$ tới $+0,75$ điểm. Bất đẳng thức $\Sigma_b < |R_{\text{mt}}|$ của nguyên tắc giữ trật tự độ lớn vì thế được thỏa mãn, và không tồn tại chính sách nào thu được tổng phần thưởng cao hơn một chính sách hoàn thành nhiệm vụ mà lại không hoàn thành nhiệm vụ. Đây chính là tính chất mà phiên bản phần thưởng

đầu tiên không có.

3.4.6 Quá trình hiệu chỉnh hàm phần thưởng

Thiết kế trình bày ở trên là kết quả của một vòng hiệu chỉnh, không phải phương án đầu tiên. Vì kỹ thuật thiết kế phần thưởng là phần khó nhất và mang tính phương pháp luận nhất của đề tài, mục này ghi lại trung thực phiên bản thất bại cùng cách chẩn đoán, qua đó cho thấy các nguyên tắc ở Mục 3.1.4 được rút ra từ đâu.

Phiên bản đầu tiên được xây dựng theo trực giác thông thường là thưởng cho các cột mốc trung gian và phạt cho những trạng thái bị coi là “xấu”. Ngoài phần thưởng nhóm khi cả hai tới đích, nó gồm một khoản thưởng cá nhân khi đứng trên bàn đạp, một khoản khi đã rời phòng xuất phát, một khoản nhóm khi cả hai đã rời phòng, một khoản cho việc đẩy khối vật cản và một khoản phạt thời gian mỗi bước. Đáng chú ý nhất, thiết kế này còn có hai khoản phạt nhóm nhằm ép hai tác nhân hành động đồng bộ: phạt khi một tác nhân đứng trên bàn đạp trong lúc đồng đội chưa rời phòng, và phạt nặng hơn khi một tác nhân đã ra ngoài còn tác nhân kia vẫn kẹt lại.

Khi huấn luyện với MA-POCA, phần thưởng nhóm trung bình mắc kẹt ở đúng mức sàn tương ứng với khoản phạt thời gian tích lũy trọn một lượt chơi và không nhích lên trong suốt hàng triệu bước. Việc chỉ số đứng yên ở đúng giá trị của riêng khoản phạt thời gian là một manh mối định lượng quan trọng: nó cho thấy không lượt chơi nào chạm đích, đồng thời không hành vi trung gian nào được tưởng thưởng đủ để kéo chỉ số lên. Phân tích kỹ hơn phát hiện ba khuyết tật đan xen.

Khuyết tật nghiêm trọng nhất là hai khoản phạt nhóm phạt đúng những trạng thái mà lời giải bắt buộc phải đi qua. Như chuỗi trạng thái mẫu ở đầu mục đã chỉ rõ, màn bàn giao nhất thiết phải trải qua giai đoạn bất đối xứng, khi một tác nhân giữ bàn đạp còn tác nhân kia vượt cổng. Nhưng đó chính xác là cấu hình mà hai khoản phạt nhóm giáng xuống, nên mọi con đường dẫn tới lời giải đều đi qua vùng bị phạt và chính sách tối ưu cục bộ trở thành “không ai cố gắng bàn giao cả”. Đây là dạng lỗi mà nguyên tắc bám theo lời giải mẫu được đặt ra để phòng ngừa.

Khuyết tật tiếp theo là hiện tượng lợi dụng phần thưởng ở giai đoạn cửa mở. Do các khoản thưởng trung gian được cộng theo từng bước mà không chuẩn hóa theo

độ dài lượt chơi, tác nhân phát hiện ra rằng chỉ cần nán ná ở một cấu hình cho điểm là đã tích lũy được phần thưởng mà không cần hoàn thành nhiệm vụ. Chỉ số phần thưởng nhóm khi đó có thể phình lên tới gần gấp đôi giá trị của một lời giải sạch, tạo ra ảo giác về tiến bộ trong khi hành vi thực chất là vô nghĩa. Đây là biểu hiện điển hình của việc vi phạm nguyên tắc giữ trật tự độ lớn.

Khuyết tật còn lại nằm ở cơ chế môi trường chứ không ở phần thưởng: cửa đóng lại tức thì khi tác nhân rời bàn đạp, khiến màn bàn giao đòi hỏi một sự đồng bộ thời gian gần như hoàn hảo và do đó có xác suất khám phá bằng thử - sai cực thấp. Ngay cả khi hàm phần thưởng hoàn toàn đúng, hành vi cần học vẫn gần như không bao giờ xuất hiện để mà được tưởng thưởng.

Bốn thay đổi phối hợp đã được thực hiện để khắc phục. Toàn bộ các khoản phạt nhóm cho trạng thái bất đối xứng bị gỡ bỏ và thay bằng phần thưởng bàn giao một lần, tức chuyển từ trừng phạt sang tưởng thưởng đúng hành vi mà bài toán đòi hỏi. Các khoản thưởng định hướng cộng dồn theo bước được thay bằng định hình theo thể năng [15] với bảo đảm bất biến chính sách. Cơ chế cửa được bổ sung khoảng ân hạn một giây, đủ ngắn để không cho phép một tác nhân đơn độc vượt cổng nhưng đủ dài để biến màn bàn giao từ chỗ gần như không thể khám phá thành chỗ khả học. Và quan sát của mỗi tác nhân được bổ sung hai cờ trạng thái vượt cổng của bản thân cùng đồng đội, giúp chính sách phi tập trung có căn cứ trực tiếp để định thời điểm nhả bàn đạp, đi kèm một chương trình học hai bài trình bày ở Mục 3.4.8.

Với thiết kế mới, phần thưởng nhóm không còn mắc kẹt ở mức sàn mà tăng lên tới giá trị tương ứng với một lời giải bàn giao trọn vẹn. Để loại trừ khả năng chỉ số đẹp đến từ một lối tắt cá nhân, đề tài kiểm chứng bằng lập luận định lượng ở Mục 3.4.4: với vận tốc tối đa hữu hạn của tác nhân và khoảng ân hạn một giây, không tồn tại quỹ đạo giúp một tác nhân đơn độc vượt cổng, nên phần thưởng cao chỉ có thể là bằng chứng của hợp tác thực sự. Kinh nghiệm đáng giá nhất rút ra từ vòng hiệu chỉnh này là một hàm phần thưởng hồng thưởng để lại dấu vết định lượng rõ ràng trên chính các chỉ số huấn luyện, chẳng hạn việc mắc kẹt ở đúng giá trị của một thành phần phạt; biết đọc những dấu vết đó là công cụ chẩn đoán hiệu quả hơn nhiều so với việc dò tìm hệ số một cách mù quáng.

3.4.7 Điều kiện kết thúc lượt chơi

Lượt chơi kết thúc trong hai trường hợp. Trường hợp thành công là cả hai tác nhân cùng chạm vạch đích: nhóm nhận phần thưởng một điểm, lượt chơi của cả nhóm được kết thúc và toàn bộ khu vực huấn luyện được đặt lại, gồm vị trí hai tác nhân, khối vật cản, các cờ trạng thái và trạng thái cửa. Trường hợp còn lại là khu vực huấn luyện chạy đủ một trăm nghìn bước vật lý mà nhiệm vụ chưa hoàn thành; khi đó lượt chơi bị ngắt thay vì kết thúc, nghĩa là giá trị của trạng thái cuối vẫn được ước lượng và dùng để cập nhật chính sách, tránh việc thuật toán hiểu nhầm rằng trạng thái đó là một thất bại tuyệt đối.

3.4.8 Cấu hình huấn luyện và học theo chương trình

Ngay cả với đầy đủ các tín hiệu phần thưởng ở trên, việc học màn bàn giao ngay từ đầu vẫn rất khó, vì tác nhân phải đồng thời học cả kỹ năng điều hướng lẫn kỹ năng phối hợp theo trình tự. Để giảm độ khó ban đầu, đề tài áp dụng học theo chương trình với một tham số môi trường điều khiển trạng thái cổng. Ở bài học thứ nhất, hai cánh cửa bị vô hiệu hóa nên cổng luôn mở, tác nhân chỉ cần học kỹ năng điều hướng tới đích. Khi phần thưởng trung bình đã làm tròn vượt ngưỡng 0,6 và duy trì qua ít nhất một trăm lượt, hệ thống tự động chuyển sang bài học thứ hai, trong đó cổng bị khóa và màn bàn giao trở thành bắt buộc:

$$\text{Bài học} = \begin{cases} 0 \text{ (cổng mở)} & \text{nếu } \bar{r} < 0,6, \\ 1 \text{ (cổng khóa)} & \text{nếu } \bar{r} \geq 0,6 \text{ trong ít nhất 100 lượt liên tiếp.} \end{cases} \quad (3.9)$$

Cách chia hai bài học này cho phép tác nhân xây dựng nền tảng điều hướng vững trước khi đối mặt với thách thức phối hợp.

Vì môi trường sử dụng cơ chế phần thưởng nhóm, MA-POCA là thuật toán phù hợp nhất về mặt lý thuyết, nhờ phê bình tập trung kết hợp cơ sở phản thực giúp gán công lao chính xác cho từng tác nhân. Thuật toán được đưa vào so sánh là MAPPO: cũng dùng phê bình tập trung nhìn quan sát của cả nhóm, nhưng tính công lao theo cách khác. Cả hai dùng chung mạng ba lớp ẩn với hai trăm năm mươi sáu đơn vị mỗi lớp, cùng một tín hiệu tò mò cường độ 0,02 để khuyến khích khám phá trong môi trường phần thưởng thưa, và cùng một chương trình học; nhờ đó khác biệt về

kết quả phản ánh đúng bản chất thuật toán. Số bước huấn luyện tối đa được đặt ở năm triệu bước nhằm bảo đảm đủ thời gian cho hành vi phối hợp phức tạp hình thành.

3.5 Môi trường Cross The Road

3.5.1 Mô tả bài toán

Cross The Road là bài toán điều hướng đơn tác nhân, trong đó một tác nhân phải băng qua một con đường có các phương tiện di chuyển liên tục để đến được vạch đích ở phía bên kia mà không bị va chạm. Dù đơn giản về hình thức, đây lại là mô hình thu nhỏ của rất nhiều tình huống điều hướng tránh vật cản động thường gặp trong game, chẳng hạn nhân vật phải né đạn, né kẻ địch hay vượt qua các chướng ngại chuyển động. Thách thức cốt lõi nằm ở chỗ tác nhân phải học được thời điểm nào nên di chuyển và thời điểm nào nên chờ đợi: lao lên đúng lúc có phương tiện đi qua sẽ dẫn đến va chạm, còn chờ đợi quá lâu lại không đạt được mục tiêu.

Khu vực huấn luyện gồm năm làn xe nằm giữa điểm xuất phát và vạch đích. Mỗi làn có một chiếc xe chạy theo một chiều cố định và quay lại đầu làn khi ra khỏi biên; tốc độ của mỗi xe được bốc ngẫu nhiên lại sau mỗi lượt chơi trong một dải riêng của làn đó, dao động từ một tới tám đơn vị trên giây tùy làn. Chính sự ngẫu nhiên này ngăn tác nhân học vẹt một chuỗi hành động cố định và buộc nó phải học một chính sách phản ứng thực sự dựa trên tình huống quan sát được. Ngoài xe, khu vực còn có tường và cây đóng vai trò vật cản tĩnh. Toàn bộ khu vực được đóng gói thành một prefab độc lập nên có thể nhân bản thành nhiều bản sao chạy song song khi cần tăng tốc thu thập kinh nghiệm.

3.5.2 Không gian quan sát

Vector quan sát của tác nhân gồm một trăm ba mươi hai giá trị, chia thành hai phần có vai trò rõ rệt. Sáu giá trị đầu gồm tọa độ ba chiều của chính tác nhân và tọa độ ba chiều của điểm đích, đều tính trong hệ tọa độ địa phương của khu vực. Từ hai bộ tọa độ này, tác nhân suy ra được hướng và khoảng cách cần đi, tức thông tin định hướng tầm xa.

Một trăm hai mươi sáu giá trị còn lại đến từ cảm biến tia: hai mươi một tia trái

đều trong góc quạt một trăm tám mươi độ phía trước với tầm xa bốn mươi đơn vị, nhận diện bốn nhãn vật thể là xe, tường, cây và đích. Mỗi tia sinh ra sáu giá trị gồm bốn giá trị mã hóa một-nóng cho nhãn, một giá trị báo tia có chạm vật thể nào không và một giá trị khoảng cách chuẩn hóa:

$$o_{\text{tia}_j} = (\text{chạm}_j, d_j/d_{\max}, \underbrace{c_{j,1}, \dots, c_{j,m}}_{\text{mã hóa một-nóng}}), \quad \dim(o) = 6 + k \times (m + 2), \quad (3.10)$$

với $k = 21$ là số tia và $m = 4$ là số nhãn vật thể.

Sự phân vai giữa hai phần phản ánh đúng cấu trúc của bài toán: thông tin vị trí trả lời câu hỏi đi về đâu, còn cảm biến tia trả lời câu hỏi lúc nào thì an toàn để đi. Nếu thiếu cảm biến tia, tác nhân không có căn cứ để né tránh và buộc phải đoán mù thời điểm băng qua; nếu thiếu thông tin vị trí, tác nhân có thể né tránh tốt nhưng không biết phải tiến về hướng nào để tới đích.

3.5.3 Không gian hành động và cơ chế điều khiển

Tác nhân di chuyển theo từng ô lưới với bốn hành động rời rạc là đứng yên, sang trái, sang phải và tiến về phía trước. Mỗi hành động di chuyển đặt một ô đích cách vị trí hiện tại đúng một đơn vị theo hướng tương ứng, sau đó tác nhân trượt đều tới ô đó; trong lúc chưa tới nơi, mọi hành động mới đều bị bỏ qua. Khác với hai môi trường còn lại, chuyển động ở đây không đi qua mô phỏng lực mà là phép nội suy vị trí, nên quỹ đạo hoàn toàn xác định. Tác nhân ra quyết định sau mỗi năm bước vật lý, tức mười quyết định mỗi giây mô phỏng.

Việc rời rạc hóa chuyển động theo ô lưới có chủ đích: nó làm cho không gian trạng thái gọn gàng và làm nổi bật bản chất cốt lõi của bài toán là quyết định thời điểm, tức chọn đúng khoảnh khắc để tiến hay để chờ, thay vì quyết định về điều khiển vận động tinh vi. Hành động đứng yên vì thế không phải một lựa chọn thừa mà là một hành động chiến lược quan trọng: đứng yên đúng lúc để nhường một phương tiện đi qua thường là mấu chốt của một lượt chơi thành công.

3.5.4 Hàm phần thưởng

Hàm phần thưởng, trình bày trong Bảng 3.4, được thiết kế theo hướng đơn giản và trực tiếp: một phần thưởng bằng một điểm khi đến đích và một hình phạt bằng 0,025 điểm khi va chạm. Chiếu theo khuôn dạng (3.1), môi trường này chỉ dùng

thành phần mục tiêu, không có thành phần định hình cũng như không có khoản nào cộng dồn theo bước. Sự tối giản đó là chủ ý: quan sát đã chứa sẵn tọa độ đích nên tác nhân không cần tín hiệu dẫn đường nhân tạo, và bài toán đủ ngắn để tín hiệu thừa vẫn xuất hiện thường xuyên.

Tỉ lệ giữa hình phạt va chạm và phần thưởng mục tiêu, chỉ bằng một phần bốn mươi, là tham số nhạy cảm nhất của thiết kế này. Lý do nằm ở chỗ môi trường không đặt giới hạn số bước cho một lượt chơi, nên tác nhân luôn có sẵn một lựa chọn tầm thường là đứng yên vĩnh viễn, với giá trị kỳ vọng đúng bằng không. Gọi p là xác suất băng qua đường thành công của một chính sách và R_- là hình phạt va chạm, lựa chọn tiến lên chỉ được ưa chuộng hơn đứng yên khi

$$p \cdot 1 - (1 - p) R_- > 0 \iff p > \frac{R_-}{1 + R_-}. \quad (3.11)$$

Với $R_- = 0,025$, ngưỡng này chỉ là 2,4 phần trăm: một chính sách vụng về, băng qua thành công chưa tới ba lần trong một trăm lần thử, đã có lợi hơn việc đứng yên, nên quá trình học luôn có động lực tiến về phía trước ngay từ những bước đầu. Nếu đặt hình phạt cân bằng với phần thưởng, tức $R_- = 1$, ngưỡng nhảy lên năm mươi phần trăm và tác nhân sẽ học được rằng đứng yên là chiến lược tối ưu cho tới khi nó giỏi hơn một nửa số lần thử, một điều không thể đạt được nếu nó chưa từng thử. Hiện tượng chính sách tự đóng băng này là một dạng cực tiểu cục bộ do hàm phần thưởng tạo ra, và (3.11) cho thấy nó được điều khiển hoàn toàn bởi tỉ lệ giữa hai hằng số.

Việc giữ hình phạt nhỏ không làm suy yếu tác dụng răn đe, bởi hình phạt không phải cơ chế răn đe chính. Va chạm kết thúc lượt chơi ngay lập tức, nên tổn thất thực sự của một pha va chạm là đánh mất toàn bộ cơ hội nhận một điểm ở đích chứ không phải khoản 0,025 bị trừ. Nói cách khác, chi phí cơ hội đã đảm nhiệm vai trò răn đe, còn hằng số R_- chỉ đóng vai trò phá thế hòa giữa các quỹ đạo thất bại, và vì thế có thể giữ ở mức tối thiểu.

Thành phần còn lại là tín hiệu tò mò nội tại theo cơ chế dự đoán tự giám sát của Pathak và cộng sự [17], trong đó tác nhân được thưởng theo sai số dự đoán trạng thái kế tiếp của chính nó, tức được khuyến khích tìm tới những tình huống mà nó chưa mô hình hóa tốt. Tín hiệu này bù đắp cho việc môi trường không có hình phạt thời gian: đứng yên là trạng thái dễ dự đoán nhất nên hầu như không sinh

phần thưởng nội tại, qua đó chính sách bất động bị triệt tiêu động cơ mà không cần đưa vào một khoản phạt cộng dồn vô hạn theo số bước, vốn sẽ vi phạm nguyên tắc giữ trật tự độ lớn trong một môi trường không giới hạn độ dài lượt chơi. Cường độ được đặt ở mức 0,02, tức bằng một phần năm mươi trọng số của tín hiệu ngoại lai, đủ để dẫn dắt khám phá ở giai đoạn đầu nhưng quá nhỏ để làm sai lệch mục tiêu khi chính sách đã bắt đầu về đích đều đặn.

Bảng 3.4: Hàm phần thưởng của môi trường Cross The Road

Sự kiện	Giá trị	Loại
Đến được vạch đích	+1,0	Thưởng mục tiêu
Va chạm với xe, tường hoặc cây	-0,025	Phạt mục tiêu
Tín hiệu tò mò khuyến khích khám phá	cường độ 0,02	Nội tại

3.5.5 Điều kiện kết thúc lượt chơi

Lượt chơi kết thúc ngay lập tức trong hai trường hợp: tác nhân chạm vạch đích, hoặc tác nhân va chạm với một chiếc xe, một bức tường hay một thân cây. Trong cả hai trường hợp, tác nhân được đưa về vị trí xuất phát, các xe được đặt lại và bốc tốc độ mới. Môi trường không đặt giới hạn số bước cho một lượt chơi; điều này là an toàn vì hình phạt va chạm cùng tín hiệu tò mò đã đủ để tác nhân không dừng lại vĩnh viễn, và trên thực tế độ dài lượt chơi ở cuối quá trình huấn luyện chỉ còn khoảng vài chục bước.

3.5.6 Cấu hình huấn luyện

Đây là môi trường đơn tác nhân với không gian hành động rời rạc nhỏ, rất phù hợp để so sánh trực tiếp cả bốn thuật toán trên cùng một bài toán trong điều kiện tương đương. Mạng nơ-ron sử dụng hai lớp ẩn với một trăm hai mươi tám đơn vị mỗi lớp, một kích thước vừa phải tương xứng với độ phức tạp của bài toán. Bốn thuật toán PPO, SAC, MA-POCA và MAPPO được cấu hình tương đương nhau nhất có thể về kích thước lô và kiến trúc mạng nhằm bảo đảm sự khác biệt về kết quả phản ánh đúng bản chất thuật toán. Cần lưu ý rằng vì đây là môi trường đơn tác nhân không có nhóm, ưu thế phê bình tập trung của MAPPO và MA-POCA không có đối tượng để phát huy, nên trên môi trường này cả hai về cơ bản suy biến thành PPO; do đó

Cross The Road chủ yếu đóng vai trò kiểm chứng tính nhất quán của các thuật toán trên một bài toán điều hướng chuẩn. Số bước huấn luyện tối đa được đặt ở hai triệu bước cho cả bốn thuật toán.

3.6 Bảng tổng hợp đặc trưng ba môi trường

Ba môi trường vừa trình bày tuy khác nhau về hình thức nhưng được lựa chọn một cách có chủ đích để bao phủ ba dạng bài toán học tăng cường điển hình, đồng thời tương ứng với ba dạng cơ chế thường gặp trong game. Bảng 3.5 đối chiếu nhanh các đặc trưng chính của ba môi trường, làm cơ sở cho việc phân tích kết quả ở Chương 4. Ba môi trường lệch nhau trên nhiều trục cùng lúc: dạng bài toán, số lượng tác nhân, dạng không gian hành động, việc có hay không dùng phần thưởng nhóm, nhịp ra quyết định và cơ chế đặc thù đi kèm. Sự lệch đồng thời này là chủ ý, bởi một bộ môi trường chỉ khác nhau ở đúng một trục sẽ không đủ sức phân biệt thuật toán nào thắng nhờ đâu.

Bảng 3.5: Tổng hợp đặc trưng ba môi trường thực nghiệm

Đặc trưng	Football Table	Capture The Flag	Cross The Road
Dạng bài toán	Đối kháng một đối một	Hợp tác đa tác nhân	Điều hướng đơn tác nhân
Số tác nhân	Hai, đối kháng	Hai, hợp tác	Một
Không gian quan sát	60 giá trị	61 giá trị	132 giá trị
Không gian hành động	Liên tục, tám chiều	Rời rạc, bảy hành động	Rời rạc, bốn hành động
Cảm biến tia	Không	Bảy tia, sáu nhãn	Hai mươi một tia, bốn nhãn
Nhịp ra quyết định	2 bước vật lý	20 bước vật lý	5 bước vật lý
Phần thưởng nhóm	Không	Có	Không
Kết thúc lượt chơi	Bàn thắng hoặc 3.000 bước	Cả hai tới đích hoặc 100.000 bước	Tới đích hoặc va chạm
Cơ chế đặc thù	Tự chơi	Bàn giao và học theo chương trình	Vật cản động
Số bước tối đa	Mười triệu	Năm triệu	Hai triệu
Thuật toán kỳ vọng tốt nhất	PPO tự chơi	MA-POCA hoặc MAPPO	SAC

3.7 Triển khai thuật toán ngoài ML-Agents

3.7.1 Vấn đề với SAC trong ML-Agents

Như đã phân tích ở Mục 2.5.6, bộ huấn luyện SAC tích hợp sẵn của ML-Agents gặp hai giới hạn quan trọng đối với đề tài. Giới hạn đáng kể nhất liên quan đến cơ chế tự chơi. Môi trường Football Table đối kháng cần một cơ chế sinh đối thủ có trình độ tăng dần, nhưng cơ chế tự chơi chỉ được ML-Agents hỗ trợ chính thức cho các thuật toán on-policy. Nguyên nhân sâu xa là SAC lưu trữ kinh nghiệm cũ trong một bộ đệm và tái sử dụng chúng nhiều lần; khi đối thủ liên tục thay đổi trong quá trình tự chơi, những kinh nghiệm cũ trở nên lỗi thời vì chúng phản ánh cách chơi của các đối thủ đã không còn tồn tại, và điều này gây nhiều nghiêm trọng cho quá trình học.

Giới hạn còn lại liên quan đến phần thưởng nhóm trong môi trường hợp tác. Cơ chế phần thưởng nhóm của ML-Agents được thiết kế tối ưu cho MA-POCA; khi muốn áp dụng cho SAC, phần thưởng nhóm buộc phải được phân bổ thủ công về từng tác nhân, và cách phân bổ này làm mất đi phần lớn lợi thế gán công lao chính xác. Hệ quả là, để đưa SAC vào so sánh một cách công bằng trên môi trường đối kháng, đề tài cần một hướng tiếp cận cho phép kiểm soát vòng lặp huấn luyện linh hoạt hơn so với việc chỉ nạp một tệp cấu hình vào chương trình huấn luyện có sẵn.

3.7.2 Cách tích hợp thuật toán tùy chỉnh qua Python API

Giải pháp được lựa chọn là sử dụng giao diện lập trình cấp thấp bằng Python mà ML-Agents cung cấp. Thay vì để chương trình huấn luyện có sẵn điều khiển toàn bộ quá trình, đề tài sử dụng một lớp cho phép kết nối trực tiếp với môi trường Unity đang chạy, rồi bọc môi trường đó lại thành một môi trường tuân theo giao diện chuẩn thường dùng trong cộng đồng học tăng cường. Nhờ lớp bọc này, môi trường Unity có thể được đưa vào bất kỳ thư viện học tăng cường bên ngoài nào; ở đây đề tài sử dụng thư viện Stable-Baselines3 [19], một thư viện đã cài đặt sẵn và kiểm chứng kỹ lưỡng thuật toán SAC.

Quy trình tích hợp diễn ra theo trình tự sau. Trước hết, môi trường Unity được khởi chạy ở chế độ chờ kết nối. Sau đó, chương trình Python tạo một đối tượng kết nối tới môi trường và bọc nó bằng lớp chuyển đổi giao diện. Tiếp theo, một tác nhân SAC được khởi tạo với kiến trúc mạng và các siêu tham số mong muốn, rồi vòng lặp huấn luyện của thư viện được gọi để thu thập kinh nghiệm và cập nhật mạng. Trong suốt quá trình, các chỉ số huấn luyện được ghi lại theo định dạng tương thích với TensorBoard, nhờ đó kết quả của SAC có thể được so sánh trực tiếp với kết quả của các thuật toán khác. Ưu điểm lớn của cách tiếp cận này là giữ nguyên môi trường Unity mà không phải sửa đổi mã nguồn của nó, đồng thời trao toàn quyền kiểm soát vòng lặp huấn luyện cho thuật toán off-policy.

3.7.3 Cấu trúc mã nguồn bổ sung

Ngoài các tệp cấu hình cho từng lần huấn luyện, hệ thống bổ sung một số thành phần mã nguồn Python nhằm tự động hóa và phân tích. Một chương trình điều phối chạy tuần tự chín lần huấn luyện, tự động nạp đúng tệp cấu hình cho từng tổ hợp

môi trường và thuật toán, quản lý trạng thái tiến độ, ghi nhật ký và cho phép tiếp tục từ điểm dừng khi quá trình bị gián đoạn. Lớp bọc môi trường đã mô tả ở mục trên phục vụ việc huấn luyện SAC tùy chỉnh. Cuối cùng là một chương trình phân tích, có nhiệm vụ đọc các tệp sự kiện TensorBoard của toàn bộ chín lần chạy, trích xuất các chỉ số như phần thưởng, độ dài lượt chơi, hàm mất mát và entropy, rồi tự động sinh ra các biểu đồ so sánh cùng một báo cáo tổng hợp. Một số trích đoạn mã nguồn minh họa của các thành phần này được đặt trong Phụ lục D.

3.8 Thiết lập thực nghiệm

3.8.1 Cấu hình phần cứng và phần mềm

Toàn bộ thực nghiệm được tiến hành trên một máy tính cá nhân với cấu hình cố định, được tóm tắt trong Bảng 3.6. Việc sử dụng chung một cấu hình cho tất cả các lần chạy là điều kiện cần thiết để bảo đảm tính công bằng khi so sánh thời gian huấn luyện và tốc độ hội tụ giữa các thuật toán, bởi các chỉ số này phụ thuộc đáng kể vào năng lực tính toán của phần cứng. Cấu hình sử dụng một bộ xử lý đồ họa tầm trung, phản ánh điều kiện thực tế mà phần lớn sinh viên và nhà phát triển độc lập có thể tiếp cận, qua đó làm cho các kết luận của đề tài trở nên thực tiễn hơn.

Bảng 3.6: Cấu hình phần cứng và phần mềm thực nghiệm

Thành phần	Chi tiết
Bộ xử lý trung tâm	Intel Core i5-14600KF
Bộ xử lý đồ họa	NVIDIA GeForce RTX 3050
Bộ nhớ trong	32 GB
Hệ điều hành	Windows 11
Unity ML-Agents	phiên bản 4.0.2 trên Unity 6000.3
Nền tảng học sâu	PyTorch 2.2.2 với CUDA 12.1
Thư viện bổ sung	Stable-Baselines3 và giao diện Python của ML-Agents
Hệ số tăng tốc mô phỏng	gấp hai mươi lần thời gian thực

3.8.2 Tham số huấn luyện từng thuật toán

Các siêu tham số chính của ba thuật toán trên từng môi trường được tóm tắt trong Bảng 3.7. Nguyên tắc thiết lập là giữ cho các tham số chung như tốc độ học, hệ số chiết khấu và kiến trúc mạng giống nhau nhiều nhất có thể giữa ba thuật toán trên cùng một môi trường, nhằm bảo đảm sự khác biệt về kết quả phản ánh đúng bản chất thuật toán chứ không phải do chênh lệch tham số. Những khác biệt về tham số chỉ nằm ở các thành phần đặc thù của từng loại thuật toán, chẳng hạn kích thước bộ đệm kinh nghiệm rất lớn của SAC hay hệ số cập nhật mềm của mạng mục tiêu. Cấu hình đầy đủ cho từng thuật toán được liệt kê trong các Phụ lục A, B và C.

Bảng 3.7: Tóm tắt siêu tham số huấn luyện chính

Tham số	PPO	SAC	MA-POCA	MAPPO
Tốc độ học	$3e-4$	$3e-4$	$3e-4$	$3e-4$
Hệ số chiết khấu	0,99	0,99	0,99	0,99
Kích thước lô	512-2048	256-1024	512-2048	512-2048
Kích thước bộ đệm	10K-41K	100K-500K	10K-41K	10K-41K
Số đơn vị ẩn	128-512	128-512	128-512	128-512
Số lớp ẩn	2-3	2-3	2-3	2-3
Cơ chế riêng	cắt tỉ lệ 0,2	cập nhật mềm 0,005	phê bình tập trung	phê bình tập trung + chia sẻ

3.8.3 Tiêu chí đánh giá

Hiệu năng của mỗi thuật toán trên mỗi môi trường được đánh giá theo bốn tiêu chí định lượng, được lựa chọn để phản ánh các khía cạnh khác nhau của chất lượng học. Tỷ lệ hoàn thành nhiệm vụ là tỷ lệ số lượt chơi mà tác nhân đạt được mục tiêu như ghi bàn, đến đích hoặc giải xong câu đố; tiêu chí này phản ánh chất lượng cuối cùng của chính sách học được. Tổng phần thưởng trung bình trên mỗi lượt chơi được đo bằng trung bình của mười phần trăm cuối cùng trong quá trình huấn luyện, nhằm phản ánh mức hiệu năng ổn định sau khi tác nhân đã hội tụ.

Tốc độ hội tụ được đo bằng số bước huấn luyện cần thiết để phần thưởng đạt tới chín mươi phần trăm giá trị cực đại của nó; tiêu chí này phản ánh hiệu quả sử dụng dữ liệu của thuật toán, tức thuật toán cần bao nhiêu kinh nghiệm để đạt được

năng lực tốt. Độ ổn định huấn luyện được đo bằng mức dao động của đường cong phần thưởng sau khi đã hội tụ; đường cong càng ít dao động thì quá trình học càng bền vững và đáng tin cậy. Toàn bộ các chỉ số này được thu thập tự động từ tệp sự kiện TensorBoard của mỗi lần chạy và được trực quan hóa dưới dạng biểu đồ và bảng ở Chương 4.

Về mặt dữ liệu, đề tài đánh giá chín tổ hợp môi trường - thuật toán: ba thuật toán PPO, SAC và MA-POCA trên Football Table, bốn thuật toán PPO, SAC, MA-POCA và MAPPO trên Cross The Road, và hai thuật toán đa tác nhân MA-POCA và MAPPO trên Capture The Flag. Số bước tối đa nêu trong Bảng 3.5 là giới hạn đặt trong tệp cấu hình, còn số bước thực tế của mỗi lần chạy được quyết định bởi thời điểm đường cong phần thưởng đã ổn định và bởi quỹ thời gian tính toán; các lần chạy trên Cross The Road đi hết hai triệu bước, trong khi các lần chạy trên Football Table và Capture The Flag dừng sớm hơn giới hạn cấu hình. Vì độ dài các lần chạy không bằng nhau, mọi so sánh ở Chương 4 đều nêu rõ số bước tương ứng và đổi chiều trên cùng một khoảng bước khi cần. Tất cả số liệu được trích xuất trực tiếp từ tệp sự kiện TensorBoard và từ kết quả chạy thử của các mô hình đã huấn luyện.

4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày kết quả của chín lần huấn luyện và trả lời câu hỏi trung tâm của đề tài: thuật toán nào thắng ở đâu, và vì sao. Kết quả được đọc lần lượt theo từng môi trường, rồi mới đặt cạnh nhau để so sánh theo bốn tiêu chí ở Mục 3.8.3. Có một điều cần nói trước, vì nó chi phối cách đọc toàn bộ chương: trong cơ chế tự chơi, phần thưởng tuyệt đối là chỉ báo gây hiểu nhầm, nên mọi kết luận về Football Table đều dựa trên chỉ số ELO chứ không dựa trên phần thưởng.

Toàn bộ số liệu trong chương được trích xuất trực tiếp từ tệp sự kiện TensorBoard của các lần chạy huấn luyện, thông qua kịch bản `export_training_data.py`. Đại lượng được chọn làm thước đo chính là phần thưởng tích lũy mà tác nhân nhận được từ môi trường, tương ứng với chuỗi `Environment/Cumulative Reward` trong tệp sự kiện. Do chuỗi thô dao động mạnh giữa các tập, mọi đường cong và mọi chỉ số thống kê trong chương đều được tính trên chuỗi đã làm mượt bằng trung bình trượt; nhờ vậy bảng số liệu và đồ thị luôn nhất quán với nhau, đồng

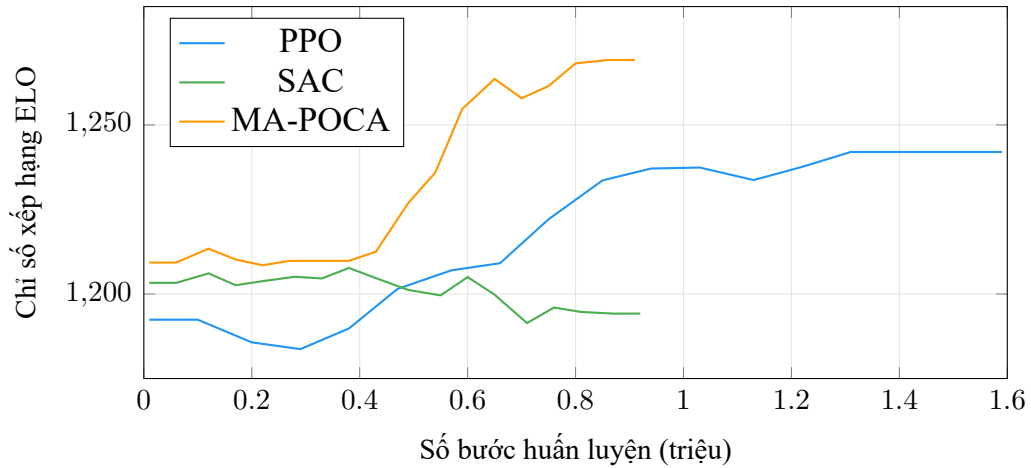
thời trùng khớp với các biểu đồ hiển thị trong ứng dụng trình diễn ở Chương 5. Mỗi thuật toán được biểu diễn bởi đúng một lần chạy hoàn chỉnh, và các lần chạy có độ dài khác nhau được giữ nguyên độ dài thực tế thay vì kéo dài về cùng một trục.

Bốn chỉ số thống kê được sử dụng xuyên suốt gồm: phần thưởng cuối (giá trị tại bước huấn luyện cuối cùng), phần thưởng tốt nhất (giá trị lớn nhất đạt được), phần thưởng trung bình mười phần trăm cuối (mức năng lực ổn định mà chính sách duy trì được ở cuối quá trình) và bước hội tụ. Bước hội tụ được xác định là bước huấn luyện mà từ đó trở đi đường cong không còn tụt xuống dưới chín mươi phần trăm mức ổn định cuối cùng; cách xác định này bền vững hơn tiêu chí “lần đầu chạm ngưỡng” vì không bị một đỉnh nhiễu đơn lẻ kéo về đầu chuỗi.

4.1 Kết quả môi trường Football Table

Môi trường Football Table được đánh giá trên ba lần chạy hoàn chỉnh: FB01 (PPO), Football_SAC_01 (SAC) và Football_POCA_01 (MA-POCA), tương ứng với ba mô hình ONNX cùng tên. Cả ba đều sử dụng cơ chế tự chơi, trong đó tác nhân thi đấu với các phiên bản trước đó của chính mình.

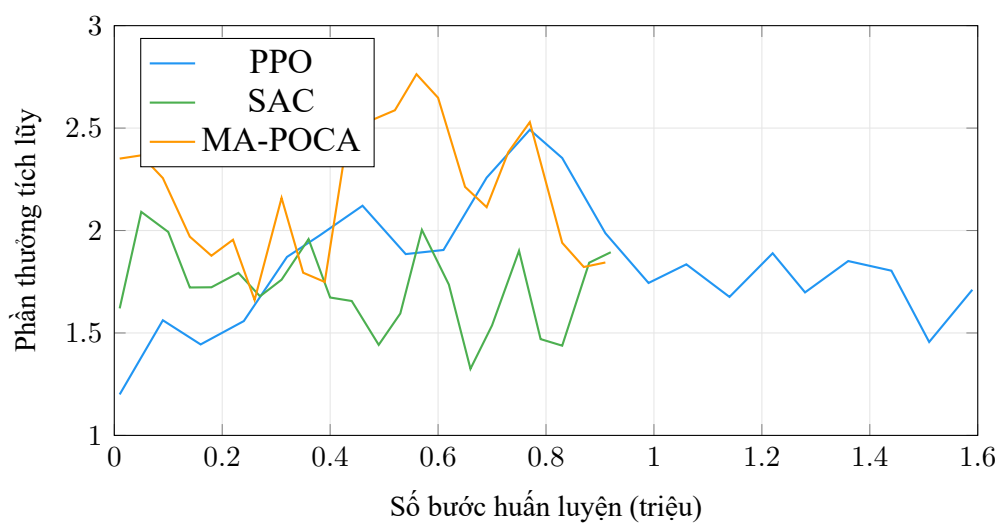
Đặc điểm của cơ chế tự chơi khiến phần thưởng tuyệt đối trở thành một chỉ báo tiến bộ kém tin cậy: khi cả hai phía cùng mạnh lên, lợi thế của bên này bị bù trừ bởi sự tiến bộ của bên kia, nên đường cong phần thưởng có xu hướng dao động quanh một mức gần như không đổi thay vì tăng đều. Chỉ số phản ánh tiến bộ chính xác hơn trong trường hợp này là chỉ số xếp hạng ELO, được ML-Agents tính riêng cho từng lần chạy dựa trên kết quả đối đầu giữa chính sách hiện tại và kho các phiên bản cũ. Hình 4.1 biểu diễn tiến trình ELO thực đo của ba thuật toán.



Hình 4.1: Tiến trình chỉ số xếp hạng ELO trong cơ chế tự chơi trên Football Table

ELO của hai lần chạy khác nhau không so sánh trực tiếp được với nhau, bởi mỗi lần chạy tính điểm dựa trên kho đối thủ của riêng nó; đại lượng có ý nghĩa ở đây là mức tăng so với giá trị khởi tạo một nghìn hai trăm. Theo tiêu chí này, MA-POCA tăng khoảng sáu mươi chín điểm và PPO tăng khoảng bốn mươi hai điểm, tức cả hai đều liên tục vượt qua các phiên bản trước của chính mình. Ngược lại, SAC dao động quanh mốc khởi tạo trong suốt quá trình và kết thúc ở mức thấp hơn điểm xuất phát, cho thấy chính sách của nó gần như không tiến bộ so với chính mình.

Hình 4.2 biểu diễn đường cong phần thưởng tích lũy của ba thuật toán, còn Bảng 4.1 tổng hợp các chỉ số định lượng.



Hình 4.2: Đường cong phần thưởng tích lũy trên môi trường Football Table

Bảng 4.1: Kết quả ba thuật toán trên môi trường Football Table

Thuật toán	Số bước	Reward TB (10% cuối)	Reward tốt nhất	Độ lệch chuẩn	ELO cuối
PPO	1,59M	1,554	2,554	0,125	1242
SAC	0,92M	1,804	2,147	0,162	1194
MA-POCA	0,91M	1,804	2,844	0,077	1269

Trên thước đo ELO, MA-POCA và PPO là hai thuật toán thực sự tiến bộ trong cơ chế tự chơi, trong đó MA-POCA đạt mức tăng lớn nhất chỉ sau khoảng chín trăm nghìn bước, còn PPO tiến bộ chậm hơn nhưng đều đặn và duy trì được xu hướng đi lên cho tới cuối lần chạy dài nhất. Cơ chế tự chơi tạo ra một chương trình học tự nhiên trong đó độ khó của đối thủ tăng dần cùng với năng lực của tác nhân, nhờ đó tác nhân không bị mắc kẹt ở một chiến lược tầm thường mà liên tục được thúc đẩy tiến bộ.

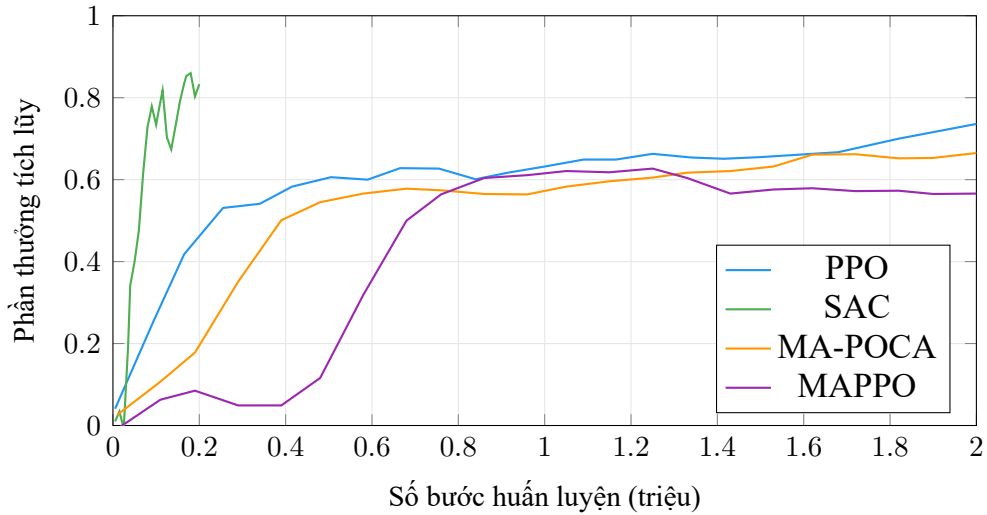
Trên thước đo phần thưởng, SAC và MA-POCA đạt mức trung bình cuối tương đương nhau (1,80), cao hơn PPO (1,55); tuy nhiên con số này cần được đọc cùng với ELO. Phần thưởng của SAC cao trong khi ELO không tăng nghĩa là chính sách của nó khai thác tốt các thành phần phần thưởng định hướng nhưng không thực sự mạnh lên trong đối kháng. Ngoài ra, đường cong của PPO bị kéo xuống bởi giai đoạn đầu khi đối thủ còn yếu và kho phiên bản còn nghèo; xét riêng nửa sau quá trình, PPO dao động quanh mức tương đương hai thuật toán còn lại. MA-POCA đồng thời đạt độ lệch chuẩn nhỏ nhất (0,077), tức đường cong ổn định nhất trong ba thuật toán.

Ba lần chạy này không có chung ngân sách bước: 1,59M cho PPO so với khoảng 0,9M cho SAC và MA-POCA. Chênh lệch đó đủ lớn để mọi so sánh phần thưởng tuyệt đối ở trên chỉ còn giá trị tham khảo, và đây là điểm yếu rõ nhất trong thiết kế thực nghiệm của đề tài (Mục 6.3).

MAPPO không được đánh giá trên môi trường này do cơ chế chia sẻ tham số của nó buộc hai đội dùng chung một chính sách. Trong cơ chế tự chơi đối kháng, hai phía cần các phiên bản chính sách khác nhau để có thể phân hóa chiến thuật, nên ràng buộc chia sẻ tham số xung đột trực tiếp với bản chất của bài toán.

4.2 Kết quả môi trường Cross The Road

Cross The Road là môi trường duy nhất được huấn luyện đầy đủ cả bốn thuật toán, tương ứng với các lần chạy ctr01 (PPO), CrossTheRoad_SAC_01 (SAC), ctr_poca_v1 (MA-POCA) và ctr_mappo_v1 (MAPPO), do đó cho phép một so sánh trực tiếp và đáng tin cậy. Hình 4.3 biểu diễn đường cong phần thưởng thực đo của bốn thuật toán, còn Bảng 4.2 tổng hợp các chỉ số định lượng.



Hình 4.3: Đường cong phần thưởng trên môi trường Cross The Road

Bảng 4.2: Kết quả bốn thuật toán trên môi trường Cross The Road

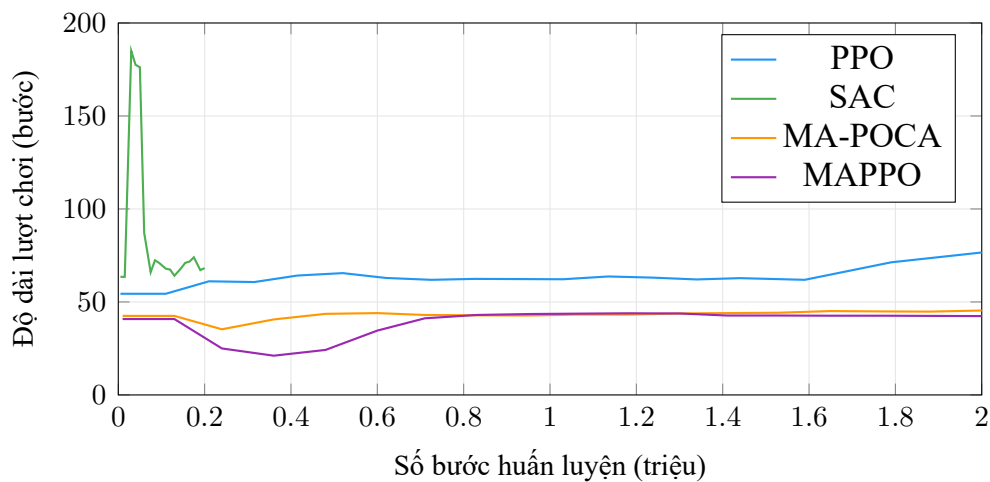
Thuật toán	Số bước	Reward TB (10% cuối)	Reward tốt nhất	Bước hội tụ	Độ dài lượt chơi
PPO	2,00M	0,703	0,737	~1,02M	76,5
SAC	0,20M	0,818	0,860	~150K	67,7
MA-POCA	2,00M	0,655	0,672	~1,22M	45,3
MAPPO	2,00M	0,567	0,628	~700K	42,4

SAC cho kết quả vượt trội rõ rệt trên môi trường điều hướng đơn tác nhân. Thuật toán này đạt phần thưởng trung bình cuối cao nhất (0,818) chỉ sau hai trăm nghìn bước, tức nhanh hơn PPO khoảng mười lần về số mẫu cần thiết. Kết quả này khẳng định ưu thế về hiệu quả sử dụng dữ liệu của thuật toán off-policy, vốn tái sử dụng nhiều lần các kinh nghiệm đã lưu, kết hợp với cơ chế entropy khuyến khích khám

phá phù hợp với môi trường có phần thưởng thưa. Tuy nhiên cần lưu ý một điểm quan trọng để tránh hiểu lầm về ưu thế của SAC: mặc dù cần ít mẫu hơn, mỗi bước cập nhật của SAC lại tốn nhiều thời gian tính toán hơn PPO do phải huấn luyện đồng thời ba mạng nơ-ron. Nói cách khác, lợi thế về số mẫu của SAC bị thu hẹp đáng kể khi xét theo thời gian đồng hồ.

PPO xếp thứ hai và là thuật toán duy nhất còn cải thiện ở cuối quá trình: đường cong của nó tăng từ 0,65 lên 0,74 trong khoảng năm trăm nghìn bước cuối, cho thấy ngân sách hai triệu bước vẫn chưa khai thác hết tiềm năng. MA-POCA cho kết quả thấp hơn PPO, phù hợp với nhận định rằng MA-POCA suy biến gần về PPO khi được áp dụng cho bài toán đơn tác nhân, nơi không có phần thưởng nhóm để phát huy thế mạnh phê bình tập trung, trong khi vẫn phải gánh thêm chi phí của kiến trúc phức tạp hơn. MAPPO cho kết quả kém nhất trên tác vụ này, do thiết kế dành riêng cho môi trường hợp tác đa tác nhân: việc chia sẻ tham số và đánh giá toàn cục không mang lại lợi thế nào, đồng thời làm giảm hiệu quả.

Bên cạnh phần thưởng, độ dài trung bình của mỗi lượt chơi là một chỉ báo bổ sung đáng quan tâm, vì nó phản ánh cách chính sách kết thúc lượt chơi. Hình 4.4 biểu diễn độ dài lượt chơi thực đo trong quá trình huấn luyện.

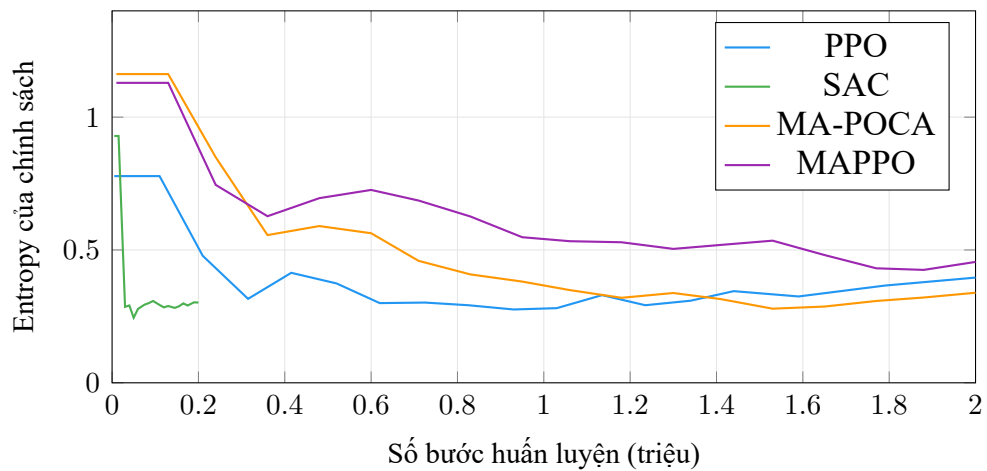


Hình 4.4: Độ dài lượt chơi trên môi trường *Cross The Road*

Đường cong của SAC cho thấy một giai đoạn tăng vọt lên gần một trăm tám mươi lăm bước ở khoảng ba mươi nghìn bước đầu, tương ứng với thời kỳ tác nhân khám phá mạnh và di chuyển lộn xộn, sau đó nhanh chóng ổn định quanh bảy mươi

bước. Điều này phù hợp với bản chất của SAC, khi cơ chế entropy khuyến khích tác nhân thử nhiều hành động khác nhau trong giai đoạn đầu trước khi hội tụ về một chính sách hiệu quả. Hai thuật toán đa tác nhân duy trì độ dài lượt chơi thấp hơn hẳn (khoảng bốn mươi hai đến bốn mươi lăm bước) nhưng lại đạt phần thưởng thấp hơn; kết hợp hai chỉ số này có thể suy ra rằng lượt chơi của chúng thường kết thúc sớm do va chạm chứ không phải do về đích nhanh.

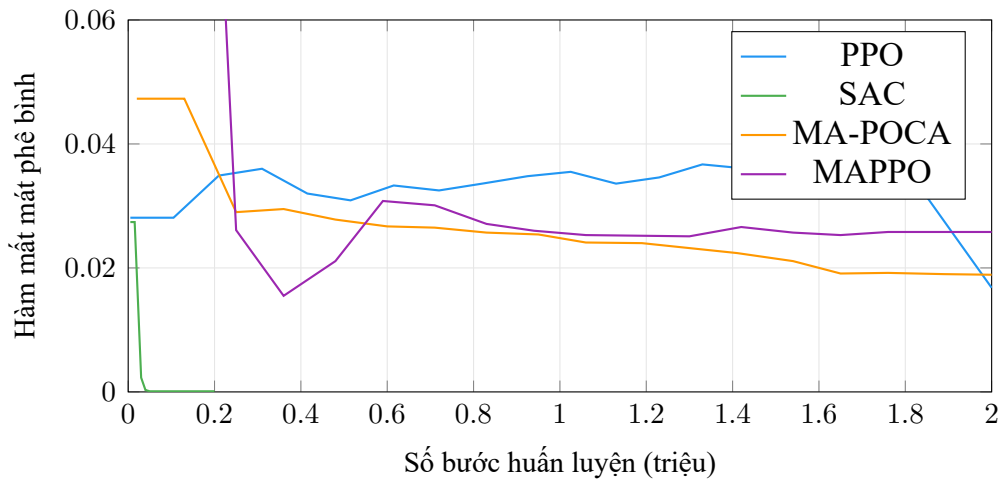
Để hiểu sâu hơn hành vi học của bốn thuật toán, ngoài phần thưởng ta còn có thể quan sát các chỉ số nội tại của quá trình huấn luyện. Chỉ số đầu tiên là entropy của chính sách, tức mức độ ngẫu nhiên trong việc lựa chọn hành động của tác nhân. Entropy cao đồng nghĩa với việc tác nhân còn đang khám phá nhiều hành động khác nhau, còn entropy thấp cho thấy tác nhân đã hội tụ về một chính sách xác định. Hình 4.5 thể hiện diễn biến entropy của bốn thuật toán.



Hình 4.5: Diễn biến entropy của chính sách trên môi trường Cross The Road

Biểu đồ entropy củng cố trực tiếp cho kết luận về tốc độ hội tụ. Cả bốn thuật toán đều khởi đầu ở mức entropy cao, phản ánh giai đoạn khám phá ngẫu nhiên ban đầu, nhưng tốc độ giảm rất khác nhau. SAC giảm dốc đứng và ổn định ở mức thấp chỉ sau khoảng ba mươi nghìn bước, tương ứng chính xác với thời điểm phần thưởng của nó hội tụ. PPO và MA-POCA giảm chậm và mượt hơn nhiều, kéo dài quá trình khám phá, qua đó lý giải vì sao chúng cần nhiều mẫu hơn để đạt cùng một mức năng lực. MAPPO duy trì entropy cao nhất trong suốt quá trình và vẫn còn ở mức khoảng 0,46 ở cuối, cho thấy chính sách của nó vẫn chưa quyết đoán - một dấu hiệu nhất quán với phần thưởng thấp nhất mà nó đạt được.

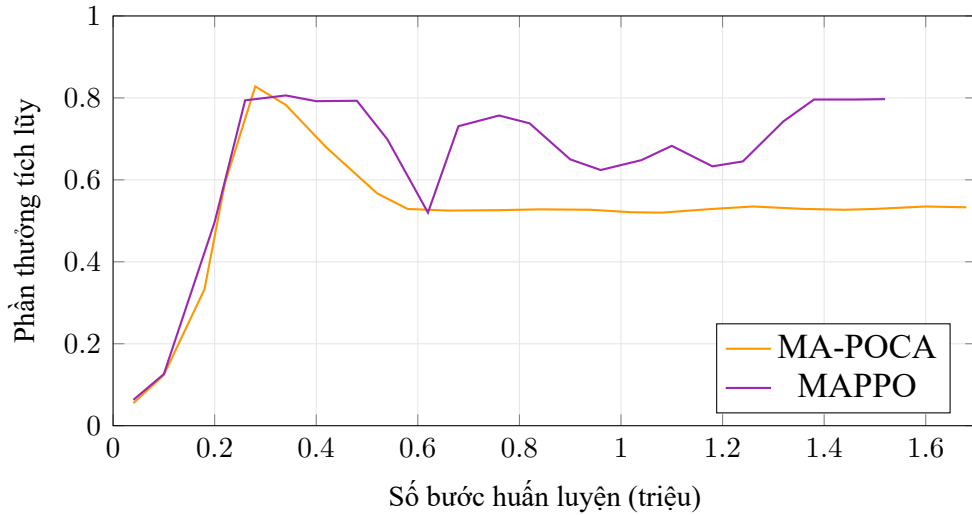
Chỉ số còn lại là hàm mất mát của mạng phê bình, phản ánh mức độ chính xác của việc ước lượng giá trị trạng thái. Hình 4.6 cho thấy một khác biệt rõ rệt: hàm mất mát của SAC gần như triệt tiêu về không chỉ sau vài chục nghìn bước, cho thấy mạng phê bình của SAC học rất nhanh nhờ cơ chế off-policy tái sử dụng dữ liệu. Trong khi đó, hàm mất mát của ba thuật toán họ PPO duy trì ở một mức dương ổn định trong suốt quá trình, đặc trưng cho các thuật toán on-policy khi mạng phê bình liên tục phải thích ứng với dữ liệu mới thu thập từ chính sách đang thay đổi.



Hình 4.6: Hàm mất mát của mạng phê bình trên môi trường Cross The Road

4.3 Kết quả môi trường Capture The Flag

Môi trường Capture The Flag là bài toán khó nhất trong ba môi trường, bởi nó đòi hỏi hai tác nhân phối hợp chặt chẽ trong khi phần thưởng lại thưa và bị trì hoãn. Trong nhóm này, chỉ hai thuật toán chuyên biệt cho đa tác nhân được huấn luyện và đánh giá: MA-POCA (lần chạy ctf_poca_v2) và MAPPO (lần chạy ctf_mappo_v2). Hình 4.7 và Bảng 4.3 trình bày kết quả tương ứng.



Hình 4.7: Đường cong phần thưởng trên môi trường Capture The Flag

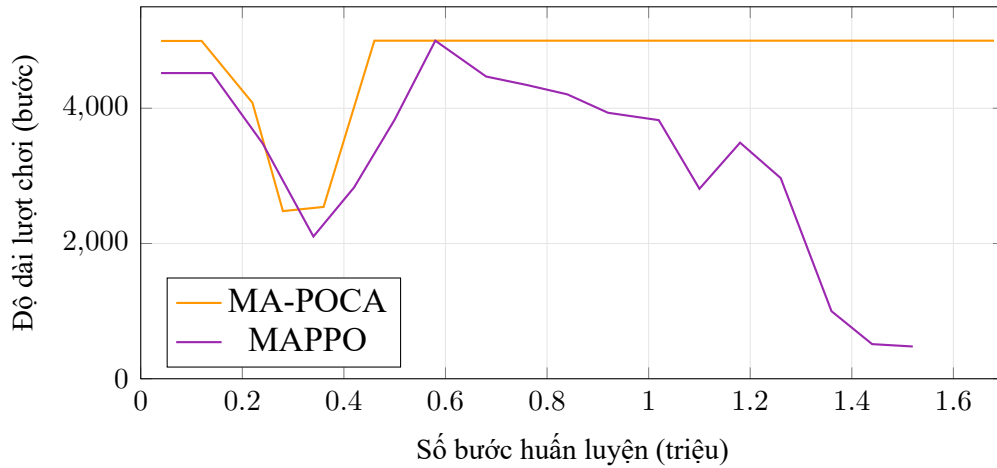
Bảng 4.3: Kết quả hai thuật toán trên môi trường Capture The Flag

Thuật toán	Số bước	Reward TB (10% cuối)	Reward tốt nhất	Bước hội tụ	Độ dài lượt chơi
MA-POCA	1,68M	0,534	0,828	~200K	4999
MAPPO	1,52M	0,795	0,818	~1,28M	468

MAPPO đạt phần thưởng trung bình cuối cao hơn MA-POCA khoảng năm mươi phần trăm (0,795 so với 0,534), qua đó khẳng định ưu thế của cơ chế phê bình tập trung kết hợp chia sẻ tham số trong bài toán hợp tác có các tác nhân đồng nhất. Tuy nhiên, bằng chứng thuyết phục nhất không nằm ở giá trị phần thưởng mà ở độ dài lượt chơi, được trình bày trong Hình 4.8.

Cả hai thuật toán đều khởi đầu với lượt chơi chạm trần bốn nghìn chín trăm chín mươi chín bước, tức luôn hết giờ mà không giải xong câu đố. MA-POCA duy trì tình trạng này cho tới cuối quá trình huấn luyện: phần thưởng của nó đi ngang ở mức 0,53 ngay từ khoảng hai trăm nghìn bước, nghĩa là chính sách đã sớm mắc kẹt ở một trạng thái thu được phần thưởng định hướng nhưng không hoàn thành nhiệm vụ. Ngược lại, từ khoảng một triệu ba trăm nghìn bước, độ dài lượt chơi của MAPPO sụt mạnh xuống còn khoảng năm trăm bước và phần thưởng đồng thời vọt lên mức cao nhất của cả quá trình. Đây là dấu hiệu rõ ràng của việc chính sách đã học được chuỗi phối hợp bàn giao và bắt đầu giải xong câu đố một cách đều đặn

thay vì để hết giờ.



Hình 4.8: Độ dài lượt chơi trên môi trường Capture The Flag: MAPPO giải xong câu đố ở giai đoạn cuối, MA-POCA luôn để hết giờ

Một câu hỏi chính đáng đặt ra là liệu MA-POCA có thực sự học được gì hay không. Để trả lời, đề tài xem xét các chỉ số nội tại của quá trình huấn luyện. Entropy của chính sách MA-POCA giảm đều đặn từ mức gần cực đại 1,93 (ứng với bảy hành động khi chọn ngẫu nhiên) xuống còn khoảng 0,86 ở cuối quá trình; sự suy giảm bền vững này là bằng chứng cho thấy tác nhân đang dần định hình một chính sách có cấu trúc thay vì hành động ngẫu nhiên. Song song đó, phần thưởng tò mò - vốn phản ánh mức độ “bất ngờ” của tác nhân trước các trạng thái nó gặp - giảm từ đỉnh khoảng 33 xuống còn 19, cho thấy tác nhân ngày càng quen thuộc với động lực của môi trường.

Đáng chú ý là cùng hai chỉ số này lại phân hóa rất rõ giữa hai thuật toán ở giai đoạn cuối: phần thưởng tò mò của MAPPO sụt xuống chỉ còn khoảng 2,3, tức thấp hơn MA-POCA gần một bậc độ lớn, đồng thời entropy của nó giảm mạnh xuống 0,80 đúng vào thời điểm độ dài lượt chơi sụt giảm. Ba chỉ số độc lập cùng thay đổi đồng thời tại một mốc là bằng chứng nhất quán cho thấy MAPPO đã thực sự nắm được lời giải của câu đố, trong khi MA-POCA dừng lại ở một chính sách chỉ khai thác phần thưởng định hướng.

4.4 Đánh giá tổng hợp

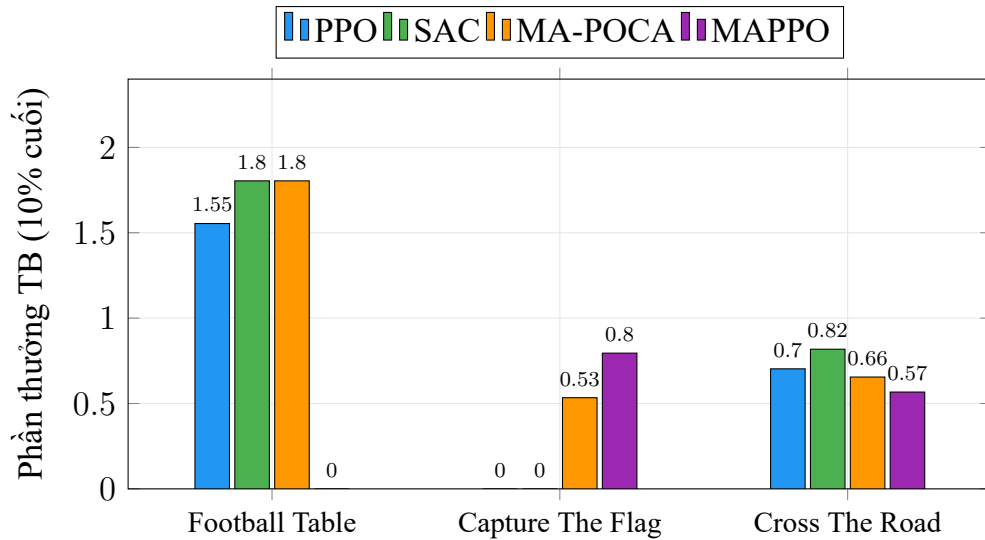
4.4.1 So sánh tốc độ hội tụ

Xét về số mẫu cần thiết để hội tụ, SAC vượt trội rõ rệt trên môi trường điều hướng khi chỉ cần khoảng một trăm năm mươi nghìn bước, so với khoảng một triệu bước của PPO và một triệu hai trăm nghìn bước của MA-POCA trên cùng môi trường. Đây là hệ quả trực tiếp của cơ chế off-policy, cho phép mỗi mẫu kinh nghiệm được tái sử dụng nhiều lần thay vì bị loại bỏ ngay sau một lần cập nhật như ở các thuật toán on-policy. Tuy nhiên, nếu xét theo thời gian đồng hồ thay vì số mẫu, lợi thế này bị thu hẹp đáng kể do chi phí tính toán trên mỗi bước của SAC cao hơn nhiều.

Trên bài toán hợp tác, con số hội tụ cần được diễn giải cẩn thận. MA-POCA đạt mức ổn định rất sớm (khoảng hai trăm nghìn bước) nhưng đó là sự ổn định của một chính sách chưa giải được câu đố; trong khi MAPPO chỉ “hội tụ” ở khoảng một triệu ba trăm nghìn bước, đúng vào thời điểm nó bắt đầu giải xong nhiệm vụ. Điều này cho thấy tốc độ hội tụ chỉ có ý nghĩa khi được đọc kèm với mức năng lực mà chính sách hội tụ tới.

4.4.2 So sánh mức năng lực đạt được

Hình 4.9 so sánh trực quan phần thưởng trung bình mười phần trăm cuối của bốn thuật toán trên cả ba môi trường - cũng chính là đại lượng được ứng dụng trình diễn sử dụng ở tab so sánh. Biểu đồ cho thấy một quy luật rõ ràng và nhất quán: mỗi thuật toán chiếm ưu thế trên đúng loại bài toán mà nó được thiết kế để giải quyết. SAC dẫn đầu trên môi trường điều hướng, MAPPO dẫn đầu trên môi trường hợp tác, còn trên môi trường đối kháng, MA-POCA và PPO là hai thuật toán duy nhất cho thấy tiến bộ thực sự theo chỉ số ELO. Quy luật này là minh chứng thực nghiệm cho luận điểm trung tâm của đề tài, rằng không tồn tại một thuật toán tối ưu duy nhất cho mọi bài toán, và việc lựa chọn thuật toán nên căn cứ vào bản chất của từng cơ chế game cụ thể.



Hình 4.9: So sánh phần thưởng trung bình mười phần trăm cuối của bốn thuật toán trên ba môi trường (giá trị 0 biểu thị thuật toán không được đánh giá do không tương thích với cơ chế của môi trường)

Chiều cao các cột chỉ so sánh được trong nội bộ từng môi trường, bởi thang phần thưởng của ba môi trường được thiết kế khác nhau: Football Table cộng dồn nhiều thành phần định hướng nên có thang lớn hơn hẳn, còn Cross The Road và Capture The Flag có phần thưởng chuẩn hóa quanh mức một đơn vị cho mỗi lần hoàn thành nhiệm vụ.

4.4.3 So sánh độ ổn định

Về độ ổn định của quá trình huấn luyện, đo bằng độ lệch chuẩn của đường cong phần thưởng trên mười phần trăm cuối, các thuật toán họ PPO cho đường cong mượt hơn đáng kể. Trên Cross The Road, MA-POCA và MAPPO đạt độ lệch chuẩn khoảng 0,004, PPO đạt 0,023, trong khi SAC là 0,014 nhưng trên một chuỗi ngắn hơn nhiều. Trên Football Table, MA-POCA ổn định nhất (0,077) còn SAC dao động mạnh nhất (0,162). Kết quả này bắt nguồn từ cơ chế cắt tỉ lệ đặc trưng của họ PPO, vốn giới hạn độ lớn của mỗi bước cập nhật và do đó ngăn chính sách thay đổi quá đột ngột; ngược lại, SAC dao động lớn hơn do bản chất khám phá tích cực của cơ chế entropy cùng tính off-policy của nó.

4.4.4 Bảng chứng từ chỉ số huấn luyện nội tại

Ngoài phần thưởng, các chỉ số nội tại của quá trình huấn luyện cung cấp thêm bằng chứng khách quan củng cố cho những nhận định ở trên. Bảng 4.4 tổng hợp các chỉ số này cho những lần chạy có hành động rời rạc, nơi entropy của chính sách có cùng bản chất và có thể đối chiếu tương đối với nhau. Mức entropy giảm mạnh từ đầu đến cuối ở tất cả các lần chạy khẳng định rằng các tác nhân đều chuyển dịch từ khám phá ngẫu nhiên sang một chính sách có định hướng.

Bảng 4.4: Chỉ số huấn luyện nội tại (các môi trường hành động rời rạc)

Lần chạy	Thuật toán	Entropy đầu → cuối	Mất mát phê bình cuối
Cross The Road	PPO	0,78 → 0,40	0,017
Cross The Road	SAC	0,93 → 0,30	$\approx 0,000$
Cross The Road	MA-POCA	1,16 → 0,34	0,019
Cross The Road	MAPPO	1,13 → 0,46	0,026
Capture The Flag	MA-POCA	1,93 → 0,86	$\approx 0,000$
Capture The Flag	MAPPO	1,92 → 0,81	0,011

Hai quan sát quan trọng có thể rút ra từ bảng này. Xét trên cùng môi trường Cross The Road, tốc độ giảm entropy tỉ lệ thuận với tốc độ hội tụ đã ghi nhận trước đó: SAC giảm entropy nhanh nhất và cũng hội tụ nhanh nhất, trong khi PPO và MA-POCA giảm chậm hơn và cần nhiều mẫu hơn, còn MAPPO giữ entropy cao nhất và đạt phần thưởng thấp nhất. Đây là sự nhất quán nội tại giữa hai nhóm chỉ số độc lập, làm tăng độ tin cậy cho kết luận về hiệu quả sử dụng dữ liệu. Ở khía cạnh khác, hàm mất mát phê bình của SAC gần như bằng không phản ánh mạng phê bình hội tụ rất nhanh nhờ tái sử dụng dữ liệu; trong khi đó, giá trị gần không của MA-POCA trên môi trường Capture The Flag lại có nguyên nhân khác, đó là do phần thưởng nhóm trong môi trường này rất nhỏ nên giá trị mục tiêu mà mạng phê bình phải học cũng nhỏ theo. Việc phân biệt được hai nguyên nhân khác nhau cho cùng một hiện tượng cho thấy tầm quan trọng của việc diễn giải chỉ số trong đúng ngữ cảnh của từng thuật toán và môi trường.

4.4.5 Phân tích ưu nhược điểm từng thuật toán

Bảng 4.5 tổng hợp thứ hạng của bốn thuật toán trên từng môi trường, cho thấy rõ sự phân hóa vai trò giữa chúng.

Bảng 4.5: Xếp hạng thuật toán theo từng môi trường

Môi trường	Hạng 1	Hạng 2	Hạng 3	Hạng 4
Football Table	MA-POCA	PPO	SAC	MAPPO (N/A)
Cross The Road	SAC	PPO	MA-POCA	MAPPO
Capture The Flag	MAPPO	MA-POCA	PPO (N/A)	SAC (N/A)

Thứ hạng trên Football Table được xếp theo mức tăng ELO, thứ hạng trên hai môi trường còn lại được xếp theo phần thưởng trung bình mười phần trăm cuối. Từ toàn bộ kết quả, có thể rút ra bức tranh tổng quát về bốn thuật toán. PPO nổi bật ở tính ổn định và tính đa năng: nó hoạt động tốt trên mọi môi trường, tiến bộ đều đặn trong cơ chế tự chơi và là thuật toán duy nhất còn cải thiện ở cuối ngân sách hai triệu bước trên Cross The Road. Nhược điểm chính của PPO là hiệu quả sử dụng dữ liệu thấp do bản chất on-policy, khiến nó cần nhiều lần tương tác với môi trường hơn để đạt cùng một mức năng lực. Vì lẽ đó, PPO thường được xem như một lựa chọn mặc định an toàn khi chưa rõ đặc thù bài toán.

SAC nổi bật ở hiệu quả sử dụng dữ liệu và khả năng khám phá, khiến nó trở thành lựa chọn phù hợp cho các bài toán điều hướng đơn tác nhân với phần thưởng thưa. Đổi lại, SAC có chi phí tính toán cao trên mỗi bước và không tận dụng được cơ chế tự chơi: trên Football Table, phần thưởng của nó cao nhưng ELO không tăng, cho thấy chính sách không mạnh lên trong đối kháng.

MA-POCA và MAPPO là hai thuật toán chuyên biệt cho bài toán hợp tác nhưng bộc lộ những thế mạnh khác nhau. MA-POCA tỏ ra linh hoạt hơn, cho kết quả tốt cả trong cơ chế tự chơi đối kháng lẫn bài toán đơn tác nhân nhờ cơ chế phê bình dựa trên sự chú ý xử lý được số lượng tác nhân thay đổi. MAPPO vượt trội khi môi trường có phần thưởng nhóm và các tác nhân đồng nhất, nhờ chia sẻ tham số giúp mọi kinh nghiệm của cả hai tác nhân cùng đóng góp vào một chính sách duy nhất; đây chính là lý do chỉ MAPPO giải được câu đố hợp tác. Ngược lại, ràng buộc

chung chính sách khiến MAPPO không áp dụng được cho cơ chế đối kháng, và cả hai thuật toán đều kém hiệu quả trên bài toán đơn tác nhân do mang theo chi phí kiến trúc không cần thiết.

4.5 Thảo luận

4.5.1 Khả năng ứng dụng trong phát triển game

Kết quả thực nghiệm gợi mở một số hàm ý thực tiễn cho việc ứng dụng học tăng cường vào phát triển game. Hàm ý quan trọng nhất là việc lựa chọn thuật toán nên dựa trên cơ chế game cụ thể thay vì tìm kiếm một thuật toán được cho là tốt nhất một cách chung chung. Đối với các game đối kháng giữa người chơi với nhau hoặc với máy, chẳng hạn game thể thao hay game đấu trường, các thuật toán họ PPO kết hợp tự chơi là lựa chọn phù hợp vì chúng tạo ra được đối thủ máy có trình độ tăng dần một cách tự nhiên, mang lại trải nghiệm thử thách nhưng công bằng cho người chơi.

Đối với các game điều hướng, né tránh vật cản hay giải đồ đơn tuyến, SAC giúp rút ngắn thời gian huấn luyện nhờ hiệu quả sử dụng dữ liệu cao, đặc biệt hữu ích khi nhóm phát triển có tài nguyên hạn chế và cần lập nhanh. Đối với các game hợp tác đòi hỏi nhiều nhân vật máy phối hợp với nhau hoặc với người chơi, MAPPO và MA-POCA là những lựa chọn hàng đầu nhờ khả năng gán công lao nhóm; trong đó MAPPO cho thấy lợi thế rõ rệt nếu các nhân vật máy đồng nhất về hành vi, còn MA-POCA phù hợp hơn khi đội hình có các vai trò bất đối xứng hoặc số lượng tác nhân thay đổi trong lượt chơi. Ngoài giá trị hướng dẫn lựa chọn thuật toán, ba môi trường được xây dựng trong đề tài cùng quy trình thiết kế quan sát, hành động và phần thưởng có thể được tái sử dụng trực tiếp làm nền tảng thử nghiệm trí tuệ nhân tạo trong các dự án game Unity khác, giúp tiết kiệm đáng kể công sức xây dựng môi trường từ đầu.

5. XÂY DỰNG ỨNG DỤNG TRÌNH DIỄN TƯƠNG TÁC

Các chương trước đã trình bày quá trình huấn luyện và so sánh bốn thuật toán học tăng cường trên ba môi trường game ba chiều. Tuy nhiên, kết quả huấn luyện dưới dạng biểu đồ và bảng số liệu chưa phản ánh trực tiếp chất lượng hành vi của tác

nhân trong game. Để khép kín quy trình từ huấn luyện đến triển khai, đề tài xây dựng thêm một ứng dụng trình diễn tương tác, cho phép người dùng lựa chọn môi trường, lựa chọn chế độ chơi và lựa chọn mô hình đã huấn luyện cho tác nhân ngay trong game, đồng thời tra cứu lại chính các số liệu huấn luyện đã phân tích ở Chương 4 mà không cần rời khỏi ứng dụng. Ứng dụng này vừa là công cụ minh họa trực quan cho các kết quả so sánh, vừa chứng minh tính khả thi của việc triển khai các mô hình ONNX đã huấn luyện vào một sản phẩm game hoàn chỉnh.

5.1 Mục tiêu và yêu cầu chức năng

Ứng dụng trình diễn được thiết kế với bốn nhóm yêu cầu chức năng chính. Nhóm yêu cầu về lựa chọn nội dung chơi quy định ứng dụng phải cung cấp một màn hình menu cho phép người dùng chọn một trong ba môi trường (Cross The Road, Capture The Flag và Football Table) và một trong bốn chế độ chơi tương ứng với các cách phân vai giữa người, tác nhân máy và mô hình ngôn ngữ lớn (LLM).

Nhóm yêu cầu về các chế độ chơi quy định bốn cách phân vai. Chế độ *Người chơi* cho phép người dùng trực tiếp điều khiển tác nhân bằng bàn phím, qua đó tự mình trải nghiệm độ khó của nhiệm vụ mà tác nhân phải học. Chế độ *Agent tự chơi* để mô hình đã huấn luyện điều khiển toàn bộ tác nhân, phục vụ việc quan sát và đánh giá trực quan chất lượng chính sách. Chế độ *Chơi với Agent* kết hợp cả hai: người chơi và tác nhân máy cùng xuất hiện trong một phiên chơi, với vai trò đồng đội hoặc đối thủ tùy theo bản chất của từng môi trường. Cuối cùng, chế độ *Agent đấu LLM* cho phép một mô hình ngôn ngữ lớn điều khiển một phe để đối đầu hoặc phối hợp với tác nhân RL.

Nhóm yêu cầu về lựa chọn mô hình quy định rằng ở mọi chế độ có sự tham gia của tác nhân máy, người dùng phải chọn được mô hình cụ thể (tương ứng với thuật toán PPO, SAC, MA-POCA, MAPPO hoặc các phiên bản huấn luyện khác nhau) cho tác nhân sử dụng. Yêu cầu này biến ứng dụng thành một công cụ so sánh trực quan: người dùng có thể lần lượt chạy cùng một môi trường với các mô hình khác nhau để quan sát sự khác biệt về hành vi, hoặc cho hai mô hình đối đầu trực tiếp trong môi trường Football Table.

Nhóm yêu cầu cuối cùng là tra cứu kết quả huấn luyện ngay trong ứng dụng.

Thay vì buộc người xem mở TensorBoard hoặc lật lại báo cáo, menu được bổ sung hai tab chuyên biệt: một tab hiển thị đường cong phần thưởng và bảng thống kê của từng thuật toán trên môi trường đang chọn, một tab tổng hợp và xếp hạng các thuật toán trên cả ba môi trường. Nhờ đó, người dùng có thể xem số liệu trước rồi chuyển sang tab chơi để đối chiếu với hành vi quan sát được.

Bên cạnh các yêu cầu chức năng, ứng dụng còn phải thỏa mãn một ràng buộc quan trọng về mặt kỹ thuật: không được phá vỡ cấu hình huấn luyện hiện có của các scene. Ba scene môi trường vẫn được sử dụng hằng ngày cho việc huấn luyện, do đó mọi thay đổi phục vụ chế độ chơi chỉ được phép diễn ra tại thời gian chạy khi người dùng đi vào từ menu, còn khi mở scene trực tiếp trong Unity Editor để huấn luyện thì mọi thứ phải giữ nguyên như cũ.

5.2 Kiến trúc hệ thống

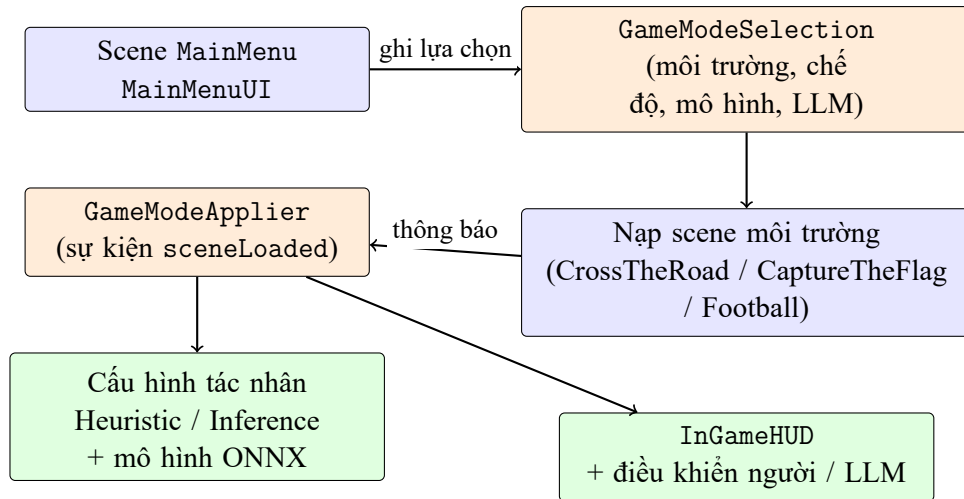
Toàn bộ mã nguồn của ứng dụng trình diễn được tổ chức trong một thư mục riêng `Assets/GameHub`, tách biệt hoàn toàn với mã nguồn của ba môi trường. Hệ thống gồm các thành phần chính được tóm tắt trong Bảng 5.1.

Bảng 5.1: Các thành phần chính của ứng dụng trình diễn

Thành phần	Vai trò
<code>GameModeSelection</code>	Lớp tính lưu lựa chọn của người dùng (môi trường, chế độ chơi, tên mô hình, LLM); tồn tại xuyên suốt giữa các scene.
<code>ModelRegistry</code>	Liệt kê và nạp các mô hình ONNX từ thư mục <code>Resources/AgentModels</code> tại thời gian chạy.
<code>MainMenuUI</code>	Dựng toàn bộ giao diện menu ba tab bằng mã lệnh khi scene <code>MainMenu</code> khởi động.
<code>UIBuilder</code>	Thư viện dựng thành phần giao diện (canvas, panel bo góc, nút, văn bản) dùng chung cho menu và HUD.
<code>Preview3D</code>	Dựng bản sao “chỉ có hình khối” của prefab môi trường và render bằng camera riêng ra <code>RenderTexture</code> để xem trước xoay được.
<code>TrainingDataStore</code>	Nạp tệp <code>training_data.json</code> chứa đường cong và thống kê huấn luyện thật đã xuất từ TensorBoard.
<code>UILineChart</code>	Biểu đồ đường tự vẽ bằng mesh trên uGUI, hiển thị nhiều chuỗi reward trong cùng một hệ trục.

Thành phần	Vai trò
GameModeApplier	Lắng nghe sự kiện nạp scene; cấu hình lại các tác nhân (loại hành vi, mô hình, chu kỳ quyết định) theo lựa chọn từ menu.
InGameHUD	Giao diện phủ trong game: nút quay về menu, thông tin chế độ, dòng hướng dẫn phím và trạng thái LLM.
Các thành phần điều khiển	CrossRoadHumanInput, PuzzleHumanInput, FootballHumanInput: chuyển tín hiệu bàn phím thành hành động của tác nhân ở chế độ người chơi.
Gói GameHub/LLM	LLMConfig, LLMClient và ba driver (LLMCrossRoadDriver, LLMPuzzleDriver, LLMFootballDriver) phục vụ chế độ Agent đấu LLM.

Luồng hoạt động của hệ thống được minh họa trong Hình 5.1. Khi người dùng bấm nút bắt đầu ở menu, các lựa chọn được ghi vào GameModeSelection rồi scene môi trường tương ứng được nạp. Thành phần GameModeApplier đăng ký với sự kiện sceneLoaded của Unity ngay từ khi ứng dụng khởi động nhờ thuộc tính [RuntimeInitializeOnLoadMethod], nên nó luôn ở trạng thái sẵn sàng trước mọi lần chuyển scene; khi scene môi trường được nạp xong, thành phần này kiểm tra xem người dùng có đi vào từ menu hay không. Nếu có, nó tìm các tác nhân trong scene và cấu hình lại từng tác nhân theo chế độ đã chọn; nếu không (trường hợp mở scene trực tiếp để huấn luyện), nó không can thiệp gì và scene giữ nguyên cấu hình gốc. Cách tiếp cận dựa trên sự kiện này cho phép bổ sung đầy đủ chức năng chơi game mà không cần sửa đổi bất kỳ scene môi trường nào.

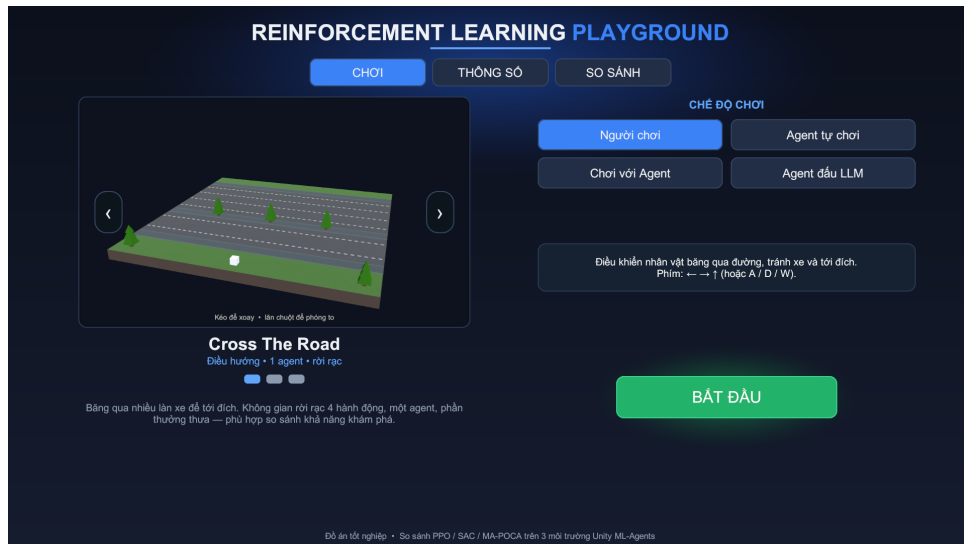


Hình 5.1: Luồng hoạt động của ứng dụng trình diễn

Điểm mấu chốt trong khâu cấu hình tác nhân là việc chuyển đổi giữa các loại hành vi của ML-Agents. Mỗi tác nhân đều mang một thành phần BehaviorParameters quy định nguồn ra quyết định của nó: ở chế độ người chơi (và ở phe do LLM điều khiển), loại hành vi được đặt thành HeuristicOnly để hành động lấy từ hàm Heuristic; ở chế độ tác nhân máy, loại hành vi được đặt thành InferenceOnly và mô hình ONNX được nạp vào để mạng nơ-ron suy luận trực tiếp trong engine. Nhờ hạ tầng suy luận tích hợp sẵn của Unity, quá trình suy luận diễn ra cục bộ trên máy người chơi với chi phí tính toán không đáng kể, không cần bất kỳ tiến trình Python nào như khi huấn luyện.

5.3 Giao diện menu và luồng sử dụng

Khi khởi động ứng dụng, người dùng được đưa vào màn hình menu chính. Toàn bộ giao diện được dựng bằng mã lệnh (không dùng prefab giao diện) và được tổ chức thành ba tab: *CHƠI*, *THÔNG SỐ* và *SO SÁNH*. Tab *CHƠI* (Hình 5.2) chia màn hình thành hai cột: cột trái là khung xem trước môi trường ở dạng ba chiều kèm tên, khẩu hiệu phân loại bài toán và mô tả ngắn; cột phải là lưới bốn nút chọn chế độ chơi, khu vực chọn mô hình, khối mô tả và nút bắt đầu. Khối mô tả thay đổi động theo tổ hợp môi trường - chế độ đang chọn, hiển thị tóm tắt luật chơi cùng sơ đồ phím điều khiển tương ứng, giúp người dùng nắm được cách chơi trước khi vào game.



Hình 5.2: Tab CHƠI của menu chính: khung xem trước ba chiều (trái) và các lựa chọn chế độ chơi (phải)

Khung xem trước ba chiều do thành phần Preview3D đảm nhiệm. Thay vì dùng ảnh tĩnh, thành phần này nạp prefab của môi trường từ thư mục Resources/EnvPrefabs và sao chép chỉ các cặp lưới - vật liệu (MeshFilter và MeshRenderer) sang một “sân khấu” đặt xa khu vực menu. Nhờ chỉ giữ lại phần hình học, bản xem trước không mang theo mã điều khiển hay thành phần vật lý nào, do đó không có mô phỏng nào chạy ngầm và không gây ảnh hưởng chéo tới scene menu. Một camera riêng render sân khấu này ra RenderTexture rồi gắn vào giao diện, cho phép kéo chuột để xoay và lăn chuột để phóng to; khi không có thao tác nào, mô hình tự xoay chậm. Khoảng cách camera ban đầu được tính từ hộp bao của toàn bộ hình khối nên mọi môi trường đều vừa khung mà không cần chỉnh tay; hai nút mũi tên hai bên khung cho phép chuyển nhanh giữa ba môi trường theo kiểu băng chuyển.

Khu vực chọn mô hình chỉ xuất hiện khi chế độ đang chọn có sự tham gia của tác nhân máy. Số lượng ô chọn mô hình cũng thay đổi theo ngữ cảnh: các tổ hợp chỉ cần một mô hình sẽ hiển thị một ô duy nhất, riêng môi trường Football Table ở chế độ *Agent tự chơi* hiển thị hai ô riêng biệt cho đội Xanh và đội Đỏ (Hình 5.3), cho phép hai mô hình khác nhau đối đầu trực tiếp. Nhãn của mỗi ô cũng được sinh theo ngữ cảnh (“Model cho cả 2 agent”, “Model đội Xanh”, “Model agent đồng đội”...) để người dùng biết chính xác mô hình mình chọn sẽ điều khiển ai. Danh

sách mô hình trong mỗi ô được liệt kê tự động từ thư mục tài nguyên, vì vậy khi bổ sung một mô hình mới chỉ cần sao chép tệp ONNX vào đúng thư mục mà không phải sửa bất kỳ dòng mã nào.



Hình 5.3: Khu vực chọn mô hình khi Football Table ở chế độ Agent tự chơi: hai ô chọn riêng cho đội Xanh và đội Đỏ

Nút bắt đầu chỉ được kích hoạt khi mọi điều kiện đã đủ. Nếu thư mục mô hình của môi trường đang chọn rỗng, hoặc nếu chế độ *Agent đấu LLM* được chọn mà chưa có khóa API, ứng dụng sẽ khóa nút bắt đầu và hiển thị một dòng cảnh báo màu đỏ chỉ rõ nguyên nhân cùng vị trí tệp cần bổ sung. Cách xử lý này giúp người dùng không rơi vào tình huống bấm chơi rồi mới gặp lỗi giữa chừng.

Trong mọi phiên chơi, một thanh thông tin mỏng được phủ ở mép trên màn hình, hiển thị tên môi trường, chế độ chơi, mô hình đang sử dụng và nút quay về menu (hoặc phím Esc); mép dưới màn hình là dòng nhắc phím điều khiển. Người dùng vì thế luôn biết mình đang ở tổ hợp thực nghiệm nào và có thể quay lại menu để đổi lựa chọn bất cứ lúc nào mà không phải khởi động lại ứng dụng.

5.4 Nạp mô hình ONNX tại thời gian chạy

Mỗi lần huấn luyện trong Chương 4 kết thúc, ML-Agents xuất chính sách đã học ra một tệp ONNX. Để các tệp này dùng được trong ứng dụng, chúng được sao chép vào cây thư mục tài nguyên của Unity theo quy ước mỗi môi trường một thư mục con:

```
Assets/GameHub/Resources/AgentModels/{CrossTheRoad |
CaptureTheFlag | Football} /
```

Tên tệp được giữ đúng bằng mã của lần chạy huấn luyện (ví dụ ctr01, ctf_mappo_v2), và tên này cũng chính là nhãn hiển thị trong menu; nhờ vậy mô hình trong ứng dụng luôn truy vết được về đúng thư mục kết quả và đúng biểu đồ đã phân tích ở Chương 4. Unity nhập mỗi tệp ONNX thành một tài sản `ModelAsset` của gói suy luận tích hợp. Nhờ đó, ứng dụng liệt kê được danh sách mô hình bằng lời gọi `Resources.LoadAll`, rồi gán mô hình đã chọn cho tác nhân qua giao diện `SetModel` của `ML-Agents`, như minh họa trong đoạn mã dưới đây.

```
1 ModelAsset[] models =
2     Resources.LoadAll<ModelAsset>("AgentModels/" + envName);
3
4 agent.SetModel(behaviorName, selectedModel);
5 behaviorParameters.BehaviorType = BehaviorType.InferenceOnly;
```

Listing 1: Liệt kê và gán mô hình ONNX tại thời gian chạy

Cách tổ chức này tách hoàn toàn dữ liệu mô hình khỏi mã nguồn: danh mục mô hình trong menu phản ánh trực tiếp nội dung thư mục tài nguyên. Trong phạm vi đồ án, ứng dụng được đóng gói sẵn chín mô hình, liệt kê trong Bảng 5.2, bao phủ kết quả huấn luyện tốt nhất của từng thuật toán trên từng môi trường.

Bảng 5.2: Các mô hình ONNX được đóng gói sẵn trong ứng dụng

Môi trường	Tệp mô hình	Thuật toán	Số bước
Cross The Road	ctr01	PPO	2,0M
	CrossTheRoad_SAC_01	SAC	2,0M
	ctr_poca_v1	MA-POCA	2,0M
	ctr_mappo_v1	MAPPO	2,0M
Capture The Flag	ctf_poca_v2	MA-POCA	1,68M
	ctf_mappo_v2	MAPPO	1,68M
Football Table	FB01	PPO (self-play)	1,59M
	Football_SAC_01	SAC	1,59M
	Football_POCA_01	MA-POCA	1,59M

5.5 Trực quan hóa kết quả huấn luyện trong ứng dụng

Một điểm khác biệt của ứng dụng so với một demo game thông thường là hai tab số liệu được nhúng ngay trong menu. Số liệu hiển thị không phải là con số nhập tay mà được trích xuất tự động từ chính các tệp nhật ký huấn luyện, nhờ vậy mọi lần huấn luyện lại đều có thể cập nhật vào ứng dụng chỉ bằng một lệnh.

5.5.1 Đường ống dữ liệu từ *TensorBoard* vào game

Trong quá trình huấn luyện, ML-Agents ghi các chỉ số ra tệp `tfevents` trong thư mục `results/<run_id>/`. Một kịch bản Python (`export_training_data.py`) đọc các tệp này bằng `EventAccumulator` của *TensorBoard* và thực hiện bốn bước xử lý nối tiếp. Kịch bản gộp chuỗi `Environment/Cumulative Reward` của tất cả tệp nhật ký thuộc cùng một lần chạy và sắp xếp theo số bước; làm mượt chuỗi bằng trung bình trượt để loại bớt nhiễu giữa các tệp; rút gọn chuỗi đã làm mượt xuống tối đa một trăm hai mươi điểm cách đều nhằm giữ kích thước tệp nhỏ, phù hợp để nạp trong game; và cuối cùng tính các thống kê tóm tắt gồm phần thưởng cuối, phần thưởng tốt nhất, trung bình mười phần trăm cuối, bước hội tụ ước lượng và tổng số bước.

Toàn bộ kết quả được ghi thành một tệp duy nhất `Resources/TrainingData/training_data.json`, có cấu trúc phân cấp gồm danh sách môi trường, mỗi môi trường chứa danh sách thuật toán, mỗi thuật toán chứa mã lần chạy, các thống kê tóm tắt và hai mảng song song `cs` (số bước) và `cv` (giá trị phần thưởng đã làm mượt). Bên phía Unity, lớp `TrainingDataStore` nạp tệp này một lần duy nhất bằng `JsonUtility` rồi lưu đệm; mỗi thuật toán được gán một màu cố định (PPO - xanh dương, SAC - xanh lá, MA-POCA - cam, MAPPO - tím) dùng thống nhất cho cả biểu đồ, chú giải và bảng. Vì mã lần chạy được lưu kèm, mọi con số hiển thị trong ứng dụng đều truy vết được về đúng thư mục kết quả đã dùng để phân tích ở Chương 4.

5.5.2 Tab THÔNG SỐ

Tab THÔNG SỐ (Hình 5.4) hiển thị đường cong phần thưởng tích lũy của mọi thuật toán đã huấn luyện trên môi trường đang chọn, kèm bảng thống kê phía dưới. Biểu

đồ do thành phần `UILineChart` vẽ: đây là một `MaskableGraphic` tự sinh lưới tam giác trong hàm `OnPopulateMesh`, trong đó mỗi đoạn của đường cong được dựng thành một hình chữ nhật có độ dày cố định, kèm lưới nền và trục. Cách vẽ trực tiếp bằng mesh giúp biểu đồ không phụ thuộc vào bất kỳ thư viện đồ thị ngoài nào và vẫn co giãn đúng theo kích thước màn hình. Miền giá trị của hai trục được tính lại mỗi lần đổi môi trường từ chính dữ liệu đang vẽ, có thêm mười phần trăm để đường cong không chạm mép khung.

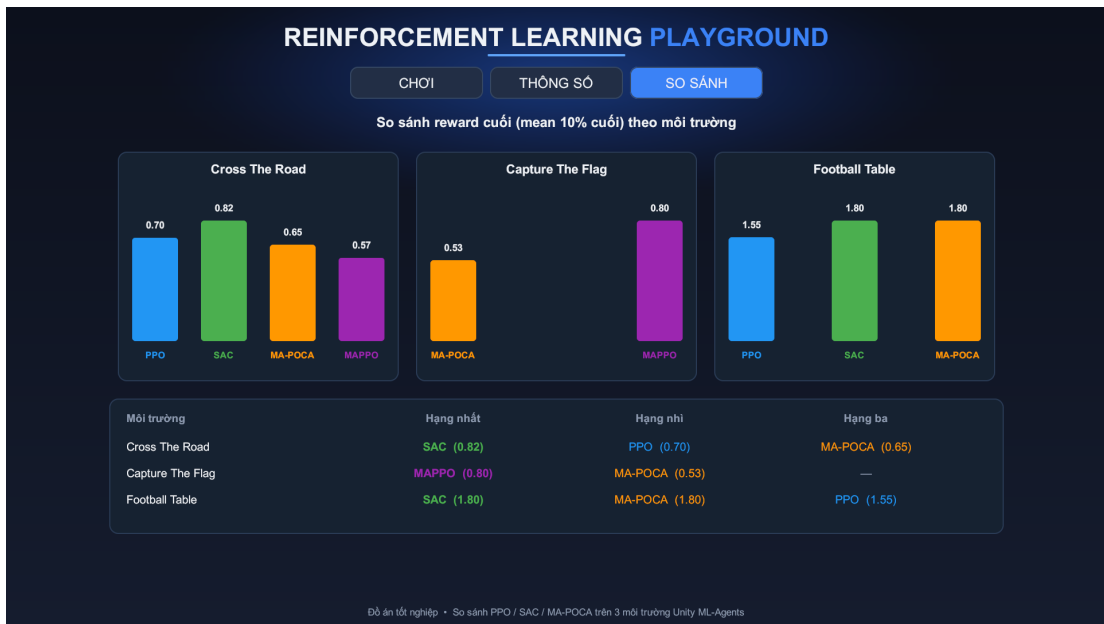


Hình 5.4: Tab THÔNG SỐ: đường cong phần thưởng tích lũy và bảng thống kê của bốn thuật toán trên môi trường Cross The Road

Bảng thống kê bên dưới biểu đồ liệt kê theo từng thuật toán các cột: phần thưởng cuối, phần thưởng tốt nhất, trung bình mười phần trăm cuối, bước hội tụ ước lượng và tổng số bước. Những thuật toán chưa được huấn luyện trên môi trường đó hiển thị “N/A” thay vì bị ẩn đi, nhờ vậy người xem thấy rõ phạm vi thực nghiệm của đề tài đến đâu. Đại lượng được vẽ ở đây là phần thưởng tích lũy môi trường (Environment/Cumulative Reward) lấy nguyên từ nhật ký huấn luyện, nên thang giá trị có thể lệch so với các đường cong phần thưởng ngoại sinh ở Chương 4. Hai tab số liệu trong ứng dụng phục vụ tra cứu nhanh, không thay thế phân tích chi tiết của báo cáo.

5.5.3 Tab SO SÁNH

Tab *SO SÁNH* (Hình 5.5) đưa ba môi trường lên cùng một màn hình. Mỗi môi trường là một thẻ chứa biểu đồ cột, mỗi cột là một thuật toán với chiều cao tỉ lệ theo phần thưởng trung bình mười phần trăm cuối và được tô đúng màu quy ước; giá trị số được ghi ngay trên đỉnh cột. Do thang phần thưởng của ba môi trường rất khác nhau (Football Table dùng thang lớn hơn hẳn hai môi trường còn lại), chiều cao cột được chuẩn hóa theo giá trị lớn nhất trong từng thẻ chứ không dùng chung một trục; nhờ vậy việc so sánh luôn diễn ra trong nội bộ một môi trường, đúng với kết luận xuyên suốt của đề tài rằng thứ hạng thuật toán chỉ có ý nghĩa khi gắn với một dạng bài toán cụ thể. Bảng xếp hạng phía dưới tổng hợp lại ba vị trí đầu của từng môi trường, cho phép đọc ngay kết luận so sánh mà không cần nhìn biểu đồ. Có một chỗ vênh cần nói rõ: bảng xếp hạng trong ứng dụng sắp thứ tự thuần theo phần thưởng, nên với Football Table nó không trùng với thứ hạng ở Bảng 4.5, nơi môi trường đối kháng được xếp theo mức tăng chỉ số ELO như đã lập luận ở Mục 4.



Hình 5.5: Tab *SO SÁNH*: biểu đồ cột theo từng môi trường và bảng xếp hạng thuật toán

5.6 Bốn chế độ chơi trong từng môi trường

Do bản chất khác nhau (đơn tác nhân, hợp tác, đối kháng), bốn chế độ chơi được ánh xạ vào mỗi môi trường theo một cách riêng, tổng hợp trong Bảng 5.3.

Bảng 5.3: Ánh xạ bốn chế độ chơi vào ba môi trường

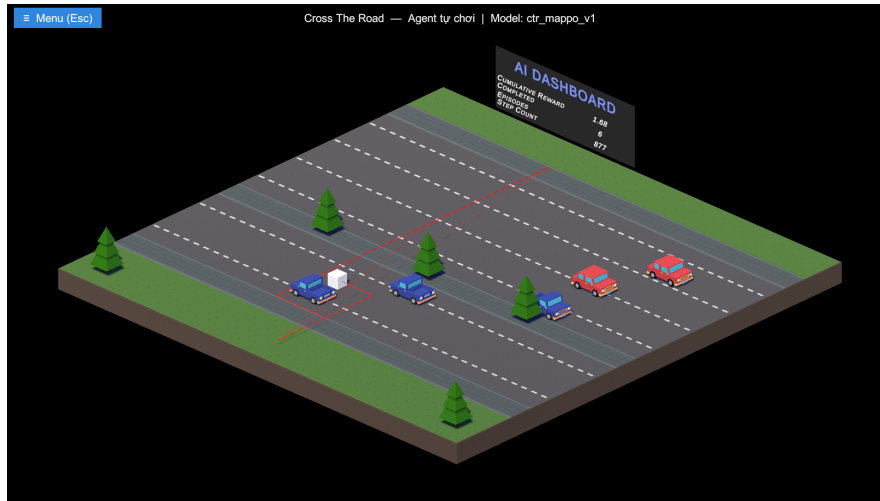
Môi trường	Người chơi	Agent tự chơi	Chơi với Agent	Agent đấu LLM
Cross The Road	Người điều khiển nhân vật trong khu vực	Mô hình điều khiển nhân vật trong khu vực	Người và tác nhân đua song song ở hai khu vực liền kề*	RL khu trái, LLM khu phải*
Capture The Flag	Hai người chơi trên cùng bàn phím	Hai tác nhân cùng dùng một mô hình	Người phối hợp cùng tác nhân đồng đội	RL một nhân vật, LLM nhân vật còn lại
Football Table	Người đấu với bot luật đơn giản	Mô hình đấu với mô hình (có thể khác nhau)	Người đấu với mô hình	Xanh = RL, Đỏ = LLM
* Hai chế độ này cần scene Cross The Road chứa từ hai khu vực huấn luyện trở lên; xem Mục 5.9.				

5.6.1 Cross The Road: trải nghiệm và quan sát chính sách

Ở chế độ *Người chơi*, người dùng điều khiển nhân vật bằng các phím mũi tên (hoặc A/D/W) đúng theo bốn hành động rời rạc mà tác nhân được phép dùng khi huấn luyện: đứng yên, sang trái, sang phải và tiến lên. Việc ràng buộc người chơi vào đúng không gian hành động của tác nhân là có chủ đích: ràng buộc này giúp người dùng đánh giá đúng độ khó của bài toán, thay vì được điều khiển tự do hơn tác nhân và đi tới kết luận sai lệch.

Ở chế độ *Agent tự chơi*, mọi tác nhân trong scene được chuyển sang suy luận bằng mô hình đã chọn và camera tự động đóng khung vào vùng chơi (Hình 5.6). Thanh thông tin phía trên hiển thị tên mô hình đang chạy, còn bảng chỉ số dựng sẵn trong môi trường hiển thị phần thưởng tích lũy, số tập đã hoàn thành và số bước của tập hiện tại; nhờ vậy người xem đối chiếu được ngay giữa hành vi quan sát được và chỉ số mà mô hình đạt được. Đây cũng là cách trực quan nhất để thấy khác biệt giữa các thuật toán: mô hình SAC băng qua đường với ít va chạm hơn, trong khi

mô hình MAPPO thường dừng lại lâu hơn ở các làn có mật độ phương tiện cao, phù hợp với thứ hạng phần thưởng đã trình bày ở Chương 4.

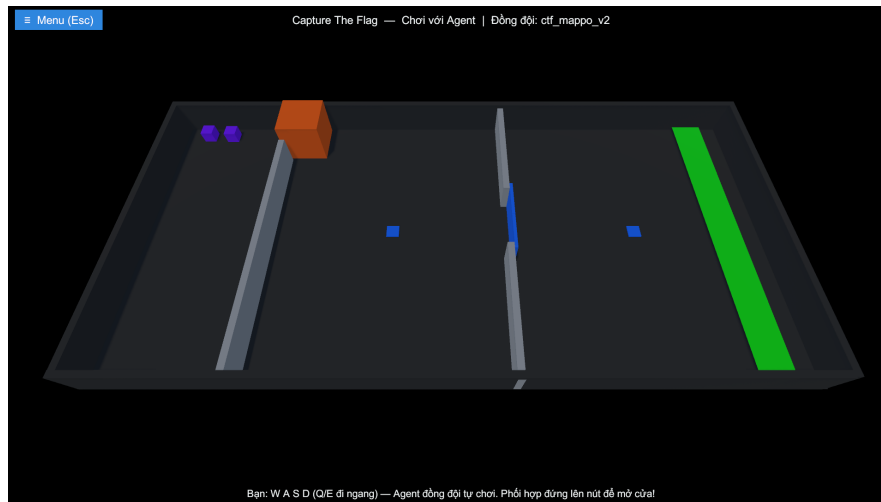


Hình 5.6: Chế độ Agent tự chơi trong Cross The Road: mô hình *ctr_mappo_v1* điều khiển tác nhân, chỉ số hiển thị trực tiếp trong cảnh

Hai chế độ còn lại của môi trường này (*Chơi với Agent* và *Agent đấu LLM*) được thiết kế theo mô hình đua song song: ứng dụng giữ lại hai khu vực huấn luyện cạnh nhau, giao khu vực bên trái cho người chơi (hoặc tác nhân RL) và khu vực bên phải cho tác nhân máy (hoặc LLM), rồi tính lại khung nhìn để cả hai khu vực cùng nằm gọn trong màn hình. Cấu hình scene hiện tại chỉ còn một khu vực huấn luyện nên hai chế độ này tạm thời chưa dùng được với Cross The Road; hạn chế cùng cách khắc phục được nêu ở Mục 5.9.

5.6.2 Capture The Flag: phối hợp người - máy

Môi trường hợp tác cho phép một kịch bản đáng chú ý: con người phối hợp với tác nhân máy trong cùng một đội. Ở chế độ *Chơi với Agent*, người chơi điều khiển một nhân vật bằng các phím W A S D (kèm Q/E để đi ngang), còn nhân vật thứ hai do mô hình đã chọn điều khiển (Hình 5.7). Nhiệm vụ đòi hỏi đúng chuỗi phối hợp đã phân tích ở Mục 3.4: một bên đứng lên bàn đạp giữ cửa, bên kia đi qua, rồi đổi vai - người chơi vì vậy phải suy đoán được ý định của đồng đội máy và hành động ăn khớp, qua đó minh họa trực tiếp cho khái niệm phối hợp đa tác nhân.



Hình 5.7: Chế độ Chơi với Agent trong Capture The Flag: người chơi phối hợp cùng tác nhân đồng đội dùng mô hình *ctf_mappo_v2*

Ở chế độ *Người chơi*, cả hai nhân vật đều do con người điều khiển theo kiểu hai người chơi chung bàn phím: một người dùng cụm W A S D (Q/E đi ngang), người còn lại dùng cụm phím mũi tên (dấu ngoặc vuông để đi ngang). Chế độ *Agent tự chơi* để cả hai nhân vật chạy cùng một mô hình, tái hiện đúng cấu hình lúc huấn luyện vì hai tác nhân vốn chia sẻ chung một chính sách.

5.6.3 Football Table: đối đầu người - máy và máy - máy

Trong môi trường đối kháng, chế độ *Chơi với Agent* cho người chơi cầm đội Xanh đấu trực tiếp với mô hình ở đội Đỏ. Vì mỗi đội có bốn thanh gạt với tám kênh điều khiển liên tục, sơ đồ phím được thiết kế theo nguyên tắc “một thanh tại một thời điểm”: phím Q/E hoặc các phím số 1-4 chọn thanh đang điều khiển, W/S trượt thanh và A/D xoay thanh để sút (Hình 5.8); tên thanh đang chọn (thủ môn, hậu vệ, tiền vệ, tiền đạo) hiển thị ở góc trên trái màn hình. Giá trị hành động do người chơi sinh ra còn được nhân với dấu của đội, đúng như cách chuẩn hóa hệ tọa độ khi huấn luyện, nhờ vậy cùng một sơ đồ phím dùng được cho cả hai phía sân. Ở chế độ *Người chơi*, đối thủ là bot luật cố định bám theo vị trí bóng - chính là đối thủ mốc đã dùng khi huấn luyện ở Mục 3.3 - với độ khó vừa phải, phù hợp để người dùng làm quen với sơ đồ điều khiển trước khi đối đầu với các mô hình đã huấn luyện.



Hình 5.8: Chế độ Người chơi trong Football Table: người chơi cầm đội Xanh, góc trên trái hiển thị thanh gạt đang được chọn

Đáng chú ý nhất là chế độ *Agent tự chơi* với hai ô chọn mô hình độc lập: người dùng có thể cho mô hình PPO huấn luyện bằng tự chơi đấu với mô hình MA-POCA hoặc SAC, hoặc cho hai bản sao của cùng một mô hình đối đầu; thanh thông tin phía trên hiển thị đồng thời tên mô hình của cả hai đội. Đây thực chất là một công cụ đánh giá đối kháng trực tiếp thu nhỏ, bổ sung một kênh so sánh định tính cho các kết quả định lượng ở Chương 4: hai mô hình có phần thưởng trung bình gần nhau vẫn có thể cho ra thế trận rất khác nhau khi gặp trực tiếp.

5.7 Tích hợp mô hình ngôn ngữ lớn

Nhằm mở rộng khả năng tương tác và tạo ra một điểm đối chiếu cho các tác nhân RL, đề tài xây dựng thêm chế độ chơi *Agent đấu LLM*. Trong chế độ này, tác nhân học tăng cường (sử dụng các mô hình PPO, SAC, MA-POCA hoặc MAPPO đã huấn luyện) sẽ đối đầu hoặc phối hợp trực tiếp với một mô hình ngôn ngữ lớn. Khác với các hướng nhúng LLM làm bộ não chiến lược cấp cao, ở đây LLM đóng vai trò như một người chơi độc lập: nó nhận trạng thái môi trường dưới dạng văn bản và trực tiếp sinh ra hành động điều khiển thông qua đúng cơ chế Heuristic mà người chơi dùng. Nhờ vậy, ba nguồn ra quyết định - người, mạng nơ-ron ONNX và LLM - đều đi qua cùng một giao diện hành động của ML-Agents và có thể so sánh trên cùng một sân chơi.

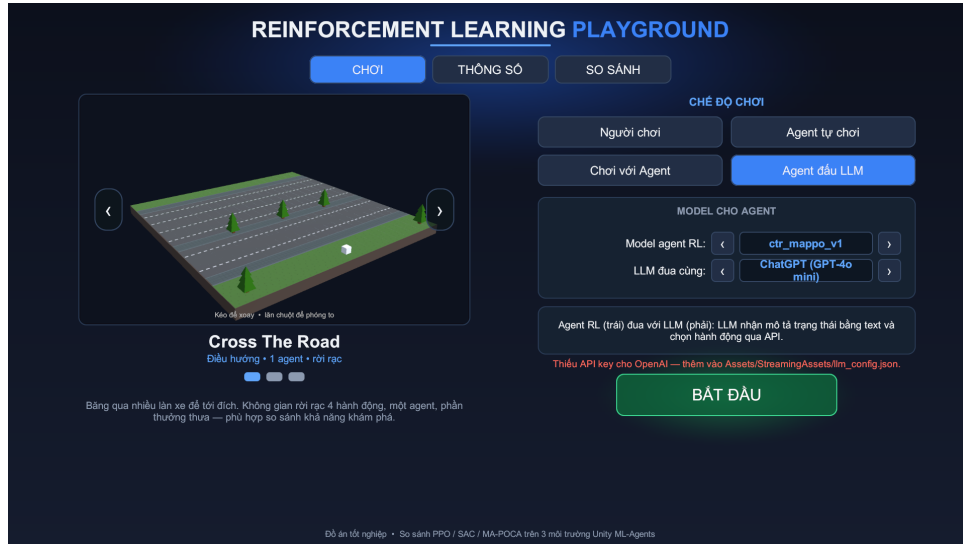
Tùy thuộc vào bản chất của từng môi trường, tương tác giữa tác nhân RL và LLM diễn ra dưới các hình thức khác nhau:

- *Cross The Road*: hai tác nhân đua song song. Tác nhân RL điều khiển khu vực bên trái, LLM điều khiển khu vực bên phải. LLM nhận mô tả vị trí của bản thân, vị trí đích và danh sách phương tiện đang di chuyển (tọa độ, hướng, tốc độ), sau đó chọn một trong bốn hành động rời rạc.
- *Capture The Flag*: hai tác nhân phối hợp. Tác nhân RL điều khiển một nhân vật, LLM điều khiển nhân vật còn lại. LLM nhận tọa độ của hai nhân vật, trạng thái nút bấm và cửa, từ đó chọn hành động di chuyển để cùng đồng đội hoàn thành nhiệm vụ.
- *Football Table*: hai đội đối kháng trực tiếp. Đội Xanh do mô hình RL điều khiển, đội Đỏ do LLM điều khiển. Trạng thái gồm vị trí và vận tốc bóng cùng vị trí, góc xoay của bốn thanh gạt, tất cả đã chuẩn hóa về đoạn $[-1, 1]$ theo dấu của đội để LLM luôn “nhìn” sân theo một hướng cố định. LLM trả về tám số nguyên tương ứng với cặp (trượt, xoay) của từng thanh.

Giao tiếp giữa Unity và LLM được thực hiện qua REST API. Lớp trừu tượng `LLMDriverBase` quản lý vòng lặp hỏi - đáp: mỗi chu kỳ (khoảng 1,0 đến 1,5 giây tùy môi trường) driver dựng một đoạn văn bản mô tả trạng thái, gửi kèm system prompt tới `LLMClient`; lớp này gọi API của OpenAI hoặc Google Gemini bằng `UnityWebRequest` chạy trên một coroutine ẩn nên không chặn vòng lặp game. Một cờ trạng thái bảo đảm không bao giờ có hai yêu cầu chồng nhau, và khi gặp lỗi liên tiếp (sai khóa, hết hạn mức, mất mạng) khoảng cách giữa hai lần gọi được giãn dần tới tối đa mười giây để tránh gọi API vô ích. Phản hồi được phân tích bằng bộ trích số nguyên chịu lỗi: LLM có thể kèm theo lời giải thích, hệ thống vẫn lấy đúng các số cần thiết; nếu không phân tích được, hành động cũ được giữ nguyên và lý do hiển thị trên HUD. Hành động sau khi phân tích được ghi vào bộ đệm và tác nhân tiêu thụ qua `Heuristic` ở mọi bước quyết định cho tới lần trả lời kế tiếp, tương tự việc người chơi giữ phím.

Khóa API được đọc từ tệp `Assets/StreamingAssets/llm_config.json` (hoặc từ biến môi trường `OPENAI_API_KEY` / `GEMINI_API_KEY`). Kho mã nguồn

chỉ chứa tệp mẫu `llm_config.example.json` và tệp thật được loại khỏi quản lý phiên bản, tránh rò rỉ khóa bí mật. Khi chưa có khóa, menu vẫn cho phép chọn chế độ nhưng khóa nút bắt đầu và hiển thị cảnh báo chỉ rõ tệp cần bổ sung (Hình 5.9).



Hình 5.9: Chế độ Agent đấu LLM: ô chọn mô hình RL và ô chọn LLM đối thủ; nút bắt đầu bị khóa kèm cảnh báo khi chưa cấu hình khóa API

Mạng nơ-ron ONNX chỉ mất vài mili-giây cho một quyết định và có thể chạy ở mọi bước vật lý, trong khi một vòng gọi API mất khoảng một đến hai giây - chậm hơn hai đến ba bậc độ lớn. Hệ quả là chính sách của LLM buộc phải hoạt động ở tần số thấp, giữ nguyên hành động trong nhiều bước liên tiếp, và điều này thể hiện rõ nhất ở Football Table, nơi bóng di chuyển nhanh và mọi phản xạ chậm đều bị trừng phạt. Đổi lại, LLM thể hiện khả năng suy luận tổng quát đáng chú ý: không trải qua bất kỳ bước huấn luyện nào, nó vẫn nêu đúng ý đồ “chờ xe đi qua rồi băng” hay “đứng lên nút để mở cửa cho đồng đội”. Nói cách khác, LLM mạnh về hiểu luật chơi nhưng yếu về điều khiển thời gian thực, còn tác nhân RL thì ngược lại; sự tương phản này chính là giá trị minh họa của chế độ chơi nói trên.

5.8 Các vấn đề kỹ thuật và giải pháp

Quá trình xây dựng ứng dụng phát sinh một số vấn đề kỹ thuật đặc thù của việc ghép nối vòng lặp quyết định của ML-Agents với tương tác thời gian thực của người chơi; các vấn đề tiêu biểu và giải pháp tương ứng được tổng hợp dưới đây.

Độ trễ điều khiển do chu kỳ quyết định: Khi huấn luyện, các tác nhân chỉ yêu cầu quyết định sau mỗi vài bước vật lý (thành phần `DecisionRequester`) nhằm giảm chi phí tính toán. Tần suất này khiến nhân vật phản hồi phím rất chậm ở chế độ người chơi. Giải pháp là khi cấu hình tác nhân cho người điều khiển (hoặc cho LLM), chu kỳ quyết định được hạ xuống 1 - tác nhân yêu cầu quyết định ở mọi bước vật lý - trong khi các tác nhân máy giữ nguyên chu kỳ gốc để tái hiện đúng điều kiện huấn luyện.

Mất phím bấm giữa hai lần quyết định: Hàm `Heuristic` của `Cross The Road` đọc sự kiện nhấn phím tức thời; nếu người chơi nhấn phím vào một khung hình không trùng thời điểm lấy quyết định thì thao tác bị bỏ sót. Đề tài bổ sung một thành phần đệm đầu vào: phím được ghi nhận trong vòng lặp `Update` (chạy mỗi khung hình) và giữ lại cho đến khi hàm `Heuristic` tiêu thụ ở lần quyết định kế tiếp, bảo đảm không thao tác nào bị mất.

Đóng khung camera với máy quay trực giao: Scene `Cross The Road` dùng một máy quay trực giao bao quát toàn bộ khu vực huấn luyện; khi số khu vực còn hoạt động thay đổi theo chế độ chơi, khung hình gốc không còn phù hợp. Ứng dụng tính hợp bao của các khu vực còn hoạt động, dời máy quay về trục nhìn đi qua tâm hộp và đặt lại kích thước trực giao bằng cách chiếu tám đỉnh hộp lên hai trục ngang - dọc của máy quay. Phép đóng khung này được hoãn đến cuối khung hình đầu tiên, vì tại thời điểm scene vừa nạp xong, các phương tiện chưa được đặt về đúng vị trí xuất phát nên hộp bao tính sớm sẽ bị sai lệch.

Xung đột thứ tự khởi tạo trong Football Table: Thành phần `Agent` của `ML-Agents` có thể gọi `OnEpisodeBegin`, kéo theo việc đặt lại bàn và bóng, ngay trong pha `OnEnable` và do đó chạy trước khi bộ khởi tạo của scene kịp hoàn tất, gây lỗi tham chiếu rỗng ngắt quãng. Đề tài sửa các lớp `Team` và `Ball` theo hướng tự khởi tạo an toàn: thao tác đặt lại sẽ tự gọi khởi tạo nếu phát hiện chưa được khởi tạo, còn việc khởi tạo lặp được chặn bằng cờ trạng thái.

Tác nhân thừa ở chế độ người chơi: Trong `Football Table` chế độ *Người chơi*, đội Đỏ do bot luật điều khiển trực tiếp, nhưng tác nhân `ML-Agents` của đội này vẫn tồn tại trong scene và có thể phát sinh hành động nhiễu. Giải pháp là vô hiệu hóa bộ yêu cầu quyết định của tác nhân đó (để nó không bao giờ yêu cầu quyết định

mới) và đặt giới hạn bước của tập về không, nhờ vậy các sự kiện bàn thắng và việc đặt lại trận đấu vẫn được xử lý bình thường.

Phạt thời gian chia cho không: Chính việc đặt giới hạn bước về không ở trên lại làm lộ một lỗi tiềm ẩn trong hàm phần thưởng của Football Table: thành phần phạt thời gian được tính bằng nghịch đảo của giới hạn bước, nên khi giới hạn bằng không, giá trị phần thưởng trở thành vô cực và ML-Agents ném ngoại lệ ở mỗi bước. Đề tài sửa bằng cách chỉ áp dụng phạt thời gian khi giới hạn bước lớn hơn không, đúng với ngữ nghĩa “tập không giới hạn bước thì không có áp lực thời gian”. Đây là một ví dụ điển hình cho việc chuyển một môi trường từ chế độ huấn luyện sang chế độ chơi có thể kích hoạt những nhánh mã chưa từng được thực thi trong huấn luyện.

5.9 Kiểm thử ứng dụng và hạn chế

Ứng dụng được kiểm thử trực tiếp trong Unity Editor theo từng luồng chức năng. Với mỗi tổ hợp môi trường - chế độ, quá trình kiểm thử xác nhận bốn điểm: (1) scene được nạp và cấu hình đúng vai trò từng tác nhân (loại hành vi, mô hình, chu kỳ quyết định); (2) mô hình ONNX được nạp thành công và tác nhân hành xử đúng như kỳ vọng từ kết quả huấn luyện; (3) giao diện phủ hiển thị đúng thông tin và nút quay về menu hoạt động; (4) không phát sinh lỗi nào trong bảng điều khiển của Unity. Kết quả kiểm thử được tổng hợp trong Bảng 5.4.

Bảng 5.4: Kết quả kiểm thử các luồng chức năng của ứng dụng

Luồng kiểm thử	Nội dung xác nhận	Kết quả
Menu: chọn môi trường, chế độ, mô hình	Giao diện cập nhật đúng; ô mô hình hiện/ẩn theo ngữ cảnh	Đạt
Menu: xem trước 3D, tab THÔNG SỐ, SO SÁNH	Xoay/phóng to mượt; biểu đồ và bảng khớp dữ liệu trong training_data.json	Đạt
Cross The Road - Người chơi, Agent tự chơi	Camera đóng khung đúng; mô hình đúng lựa chọn	Đạt
Cross The Road - Chơi với Agent, Agent đấu LLM	Cần từ hai khu vực huấn luyện trở lên trong scene	Chưa dùng được
Capture The Flag - cả 4 chế độ	Phân vai người/máy/LLM đúng; hai sơ đồ phím độc lập	Đạt
Football Table - cả 4 chế độ	Người cầm đội Xanh; bot/mô hình/LLM cầm đội Đỏ; chọn được 2 mô hình	Đạt
Quay về menu (nút và phím Esc)	Trở về menu, đổi lựa chọn, vào lại bình thường	Đạt
Mở scene trực tiếp (không qua menu)	Cấu hình huấn luyện gốc được giữ nguyên	Đạt

Về hiệu năng, suy luận mạng nơ-ron cho các tác nhân đồng thời cùng với giao diện không gây sụt giảm tốc độ khung hình đáng kể trên cấu hình thực nghiệm ở Mục 3.8.1, khẳng định tính khả thi của việc nhúng trực tiếp các mô hình đã huấn luyện vào sản phẩm game. Toàn bộ chín mô hình ONNX đi kèm ứng dụng đều nạp và vận hành chính xác với đúng phổ quan sát - hành động đã khai báo khi huấn luyện.

Ứng dụng còn ba hạn chế đã biết. Scene Cross The Road hiện chỉ giữ lại một khu vực huấn luyện, trong khi hai chế độ đua song song cần ít nhất hai khu vực; khắc phục chỉ đơn giản là nhân bản thêm khu vực trong scene, còn phần mã nguồn chọn và đóng khung khu vực đã sẵn sàng cho số lượng khu vực bất kỳ. Chế độ Agent đấu LLM phụ thuộc vào dịch vụ bên ngoài: cần khóa API hợp lệ, cần kết nối mạng, và chất lượng chơi bị giới hạn bởi độ trễ một đến hai giây mỗi lượt. Cuối cùng, hai tab số liệu phản ánh nội dung tệp training_data.json tại thời điểm xuất, nên sau mỗi đợt huấn luyện mới cần chạy lại kịch bản xuất dữ liệu thì ứng

dụng mới hiển thị số liệu cập nhật.

6. KẾT LUẬN

6.1 Kết quả đạt được

Đồ án đã hoàn thành các mục tiêu đề ra ban đầu. Cụ thể, đề tài đã xây dựng thành công ba môi trường game 3D có tính chất khác nhau trên nền tảng Unity ML-Agents: Football Table (đối kháng một đối một), môi trường Capture The Flag đa tác nhân và Cross The Road (điều hướng tránh vật cản). Với mỗi môi trường, hệ thống không gian quan sát, không gian hành động và hàm phần thưởng đã được thiết kế phù hợp với đặc thù bài toán.

Đề tài đã triển khai huấn luyện và so sánh bốn thuật toán PPO, SAC, MA-POCA và MAPPO theo bốn tiêu chí định lượng, đồng thời hiện thực hóa quy trình tích hợp SAC ra ngoài trainer tích hợp sẵn của ML-Agents thông qua Python Low-Level API.

Kết quả thực nghiệm khẳng định không tồn tại một thuật toán tối ưu duy nhất cho mọi bài toán: trong môi trường đối kháng, MA-POCA và PPO kết hợp self-play là hai thuật toán duy nhất cho thấy tiến bộ thực sự theo chỉ số ELO, SAC đạt hiệu quả mẫu vượt trội trong bài toán điều hướng đơn tác nhân, và đặc biệt, MAPPO vượt qua MA-POCA trên môi trường hợp tác đa tác nhân với tỷ lệ giải xong cao hơn đáng kể, cho thấy cơ chế chia sẻ tham số và phê bình tập trung đơn giản có thể hiệu quả hơn cơ chế attention phức tạp khi các tác nhân đồng nhất.

Bên cạnh phần thực nghiệm so sánh, đề tài đã xây dựng hoàn chỉnh một ứng dụng trình diễn tương tác, trong đó chín mô hình đã huấn luyện được triển khai trực tiếp vào game dưới định dạng ONNX. Ứng dụng cho phép người dùng lựa chọn môi trường, lựa chọn một trong bốn chế độ chơi (tự điều khiển, tác nhân tự chơi, chơi cùng tác nhân, tác nhân đấu mô hình ngôn ngữ lớn), lựa chọn mô hình cụ thể cho tác nhân và tra cứu ngay đường cong huấn luyện của từng thuật toán trên giao diện menu. Các luồng chức năng đã được kiểm thử trong Unity Editor và đều hoạt động đúng, ngoại trừ hai chế độ đua song song của Cross The Road đang chờ khôi phục số khu vực trong scene (Mục 5.9). Kết quả này chứng minh tính khả thi của việc nhúng mô hình học tăng cường vào sản phẩm game hoàn chỉnh với chi phí suy

luận không đáng kể.

6.2 Đóng góp của đề tài

Nếu phải xếp các đóng góp của đề tài theo mức độ hữu ích, tôi đặt kết quả so sánh lên đầu. Việc đưa PPO, SAC, MA-POCA và MAPPO về cùng một bộ môi trường, cùng một cấu hình phần cứng và cùng một quy trình đo cho phép rút ra những kết luận mà việc đối chiếu các tài liệu rời rạc không thể cho được. Trong số đó, phát hiện MAPPO vượt MA-POCA trên bài toán hợp tác có tác nhân đồng nhất đi ngược lại trực giác thông thường rằng thuật toán chuyên biệt hơn thì mạnh hơn, và là kết quả có giá trị tham khảo trực tiếp nhất cho người làm game.

Kế đến là phần đóng góp mang tính công cụ. Ba môi trường ba chiều đại diện cho ba dạng bài toán học tăng cường phổ biến đều được đóng gói thành prefab độc lập nên tái sử dụng được như một benchmark mở. Đi kèm chúng là quy trình đưa một thuật toán off-policy tùy chỉnh vào Unity thông qua `mlagents_envs` và `Stable-Baselines3` - con đường mà đề tài buộc phải tự mở ra vì trainer tích hợp sẵn không hỗ trợ, và cũng chính vì thế mà nó dùng lại được cho bất kỳ nghiên cứu nào vấp phải đúng rào cản ấy.

Đóng góp còn lại là ứng dụng trình diễn với bốn chế độ chơi, cơ chế chọn mô hình linh hoạt và hai tab số liệu huấn luyện nhúng thẳng trong game. Giá trị của nó không nằm ở phần game, mà ở chỗ nó khép kín quãng đường từ một tệp cấu hình YAML tới một hành vi mà người chơi cảm nhận được bằng tay. Riêng chế độ tác nhân đấu mô hình ngôn ngữ lớn dựng lên một phép đối chiếu mà đề tài không dự tính từ đầu, giữa chính sách học từ tương tác và chính sách suy luận zero-shot từ văn bản. Toàn bộ mã nguồn môi trường, cấu hình huấn luyện và dữ liệu kết quả đều được công bố dưới dạng mở.

6.3 Hạn chế

Hạn chế nặng nhất của đề tài là mỗi tổ hợp môi trường - thuật toán chỉ được chạy với một seed duy nhất. Nghĩa là mọi chênh lệch báo cáo ở Chương 4 đều chưa tách được phần do bản chất thuật toán khỏi phần do may rủi của một lần khởi tạo. Những khoảng cách lớn, như của SAC trên Cross The Road hay của MAPPO trên Capture

The Flag, nhiều khả năng vẫn đứng vững; nhưng các thứ hạng sát nhau thì không nên đọc như một kết luận thống kê. Đây là hệ quả của quỹ thời gian tính toán chứ không phải của một lựa chọn phương pháp luận, và nếu chỉ được sửa một điều duy nhất trong đồ án này thì tôi sẽ sửa đúng điều đó.

Các lần chạy cũng không có chung ngân sách bước. Trên Football Table, PPO chạy 1,59 triệu bước trong khi SAC và MA-POCA dừng ở khoảng 0,9 triệu, nên phần so sánh phần thưởng tuyệt đối ở môi trường này chỉ nên xem là tham khảo; chỉ số ELO mới là căn cứ đáng tin.

Sau cùng, Capture The Flag vẫn là bài toán chưa được giải trọn vẹn. MA-POCA mắc kẹt suốt quá trình ở một chính sách không bao giờ về đích, còn MAPPO cũng chỉ giải xong câu đố ở khoảng một phần năm cuối của lần chạy. Hàm phần thưởng, không gian quan sát và thời lượng huấn luyện của môi trường này đều còn chỗ để cải thiện.

6.4 Hướng phát triển

Việc cần làm trước tiên là chạy lại toàn bộ chín tổ hợp với nhiều seed, vì chừng nào chưa có phương sai giữa các lần chạy thì mọi so sánh trong đồ án vẫn còn treo lơ lửng. Ngay sau đó là câu hỏi thú vị nhất mà đề tài đặt ra nhưng chưa trả lời được: MAPPO thắng MA-POCA nhờ đội hình đồng nhất, vậy ranh giới ấy nằm ở đâu? Một môi trường hợp tác với hai tác nhân có vai trò bất đối xứng, hoặc có tác nhân rời nhóm giữa lượt chơi, sẽ cho biết chia sẻ tham số bắt đầu thua attention từ điểm nào.

Ở tầm xa hơn, bộ so sánh có thể được mở rộng bằng các thuật toán như DreamerV3 hay QMIX, còn ba môi trường có thể được nâng độ phức tạp lên gần với game thương mại. Ứng dụng trình diễn thì cần một bản chạy độc lập (WebGL hoặc desktop), kèm bảng xếp hạng người chơi đấu với tác nhân và một khảo sát trải nghiệm người dùng - đây là cách duy nhất để trả lời câu hỏi mà không chỉ số huấn luyện nào chạm tới được, rằng hành vi của tác nhân có thực sự “tự nhiên” trong mắt người chơi hay không.

Tài liệu tham khảo

- [1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [2] Richard Bellman. A markovian decision process. *Journal of Mathematics and Mechanics*, 6(5):679–684, 1957.
- [3] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pages 41–48, 2009.
- [4] Andrew Cohen, Ervin Teng, Vincent-Pierre Berges, Ruo-Ping Dong, Hunter Henry, Marwan Mattar, Alexander Zook, and Sujoy Ganguly. On the use and misuse of absorbing states in multi-agent reinforcement learning. *arXiv preprint arXiv:2111.05992*, 2021.
- [5] Sam Devlin and Daniel Kudenko. Dynamic potential-based reward shaping. *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 433–440, 2012.
- [6] Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, pages 2974–2982, 2018.
- [7] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1861–1870, 2018.
- [8] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.

- [9] Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. Unity: A general platform for intelligent agents. *arXiv preprint arXiv:1809.02627*, 2020.
- [10] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [11] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.
- [12] Ian Millington and John Funge. *Artificial Intelligence for Games*. CRC Press, Boca Raton, FL, 3 edition, 2019.
- [13] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1928–1937, 2016.
- [14] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [15] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- [16] OpenAI, Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębniak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.

- [17] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International Conference on Machine Learning (ICML)*, pages 2778–2787. PMLR, 2017.
- [18] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, New York, 1994.
- [19] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- [20] Jette Randløv and Preben Alstrøm. Learning to drive a bicycle using reinforcement learning and shaping. In *Proceedings of the Fifteenth International Conference on Machine Learning (ICML)*, pages 463–471, 1998.
- [21] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1889–1897, 2015.
- [22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.
- [23] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [24] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.

- [25] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.
- [26] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- [27] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [28] Georgios N. Yannakakis and Julian Togelius. *Artificial intelligence and games*. Springer, 2018.
- [29] Chao Yu, Akash Velu, Eugene Vinyals, Jiaxuan Gao, Yu Wang, Alexandre Baez, Siddharth Bhatt, Jianyu Peng, Minghao Shen, Xionghao Wang, et al. The surprising effectiveness of ppo in cooperative multi-agent games. In *Advances in Neural Information Processing Systems*, volume 35, pages 24611–24624, 2022.

PHỤ LỤC

Phụ lục A. Cấu hình huấn luyện PPO

Dưới đây là tệp cấu hình YAML của thuật toán PPO cho môi trường Football Table (kết hợp self-play). Các môi trường khác dùng cấu trúc tương tự với siêu tham số điều chỉnh theo Bảng 3.7. Lưu ý rằng cấu hình MAPPO hoàn toàn tương tự PPO, chỉ khác biệt ở việc khai báo `trainer_type: mappo`.

```

1 behaviors:
2   Football:
3     trainer_type: ppo
4     hyperparameters:
5       batch_size: 2048
6       buffer_size: 40960
7       learning_rate: 3.0e-4
8       beta: 0.005
9       epsilon: 0.2
10      lambd: 0.95
11      num_epoch: 5
12      learning_rate_schedule: linear
13    network_settings:
14      normalize: true
15      hidden_units: 512
16      num_layers: 3
17    reward_signals:
18      extrinsic:
19        gamma: 0.99
20        strength: 1.0
21    self_play:
22      save_steps: 50000
23      team_change: 200000
24      swap_steps: 2000000
25      window: 10
26      play_against_latest_model_ratio: 0.5
27      initial_elo: 1200.0
28    keep_checkpoints: 10
29    max_steps: 10000000
30    time_horizon: 1000
31    summary_freq: 50000

```

Listing 2: football_ppo.yaml

Phụ lục B. Cấu hình huấn luyện SAC

Tệp cấu hình YAML của SAC cho môi trường Cross The Road:

```

1 behaviors:
2   CrossTheRoad:
3     trainer_type: sac
4     hyperparameters:
5       batch_size: 256
6       buffer_size: 100000
7       learning_rate: 3.0e-4
8       learning_rate_schedule: constant
9       buffer_init_steps: 1000
10      tau: 0.005
11      steps_per_update: 1
12      save_replay_buffer: false
13      init_entcoef: 0.5
14      reward_signal_steps_per_update: 1
15    network_settings:
16      normalize: false

```

```

17     hidden_units: 128
18     num_layers: 2
19     reward_signals:
20       extrinsic:
21         gamma: 0.99
22         strength: 1.0
23       curiosity:
24         gamma: 0.99
25         strength: 0.02
26         learning_rate: 3.0e-4
27     keep_checkpoints: 5
28     max_steps: 2000000
29     time_horizon: 64
30     summary_freq: 10000

```

Listing 3: ctr_sac.yaml

Đối với môi trường đối kháng, SAC được tích hợp qua Python Low-Level API và Stable-Baselines3 (xem Mục 2.6.2). Đoạn mã minh họa lớp bọc môi trường:

```

1 from mlagents_envs.environment import UnityEnvironment
2 from mlagents_envs.envs.unity_gym_env import UnityToGymWrapper
3 from stable_baselines3 import SAC
4
5 # Ket noi truc tiep voi moi truong Unity dang chay
6 unity_env = UnityEnvironment(file_name=None, base_port=5005)
7 env = UnityToGymWrapper(unity_env, allow_multiple_obs=False)
8
9 # Khoi tao tac nhan SAC voi mang 3 lop x 512 don vi
10 model = SAC(
11     "MlpPolicy", env,
12     learning_rate=3e-4,
13     buffer_size=500000,
14     batch_size=1024,
15     tau=0.005,
16     gamma=0.99,
17     ent_coef="auto",
18     policy_kwargs=dict(net_arch=[512, 512, 512]),
19     tensorboard_log="./results/football_sac_v1",
20     verbose=1,
21 )
22 model.learn(total_timesteps=10_000_000)
23 model.save("results/football_sac_v1/Football_sac")
24 env.close()

```

Listing 4: Tích hợp SAC qua mlagents_envs + Stable-Baselines3

Phụ lục C. Cấu hình huấn luyện MA-POCA

Tập cấu hình YAML của MA-POCA cho môi trường Capture The Flag:

```

1 behaviors:
2   PuzzleBehavior:
3     trainer_type: poca
4     hyperparameters:
5       batch_size: 1024
6       buffer_size: 20480
7       learning_rate: 3.0e-4
8       beta: 0.01
9       epsilon: 0.2

```

```

10     lambda: 0.95
11     num_epoch: 3
12     learning_rate_schedule: linear
13 network_settings:
14     normalize: false
15     hidden_units: 256
16     num_layers: 3
17     vis_encode_type: simple
18 reward_signals:
19     extrinsic:
20         gamma: 0.99
21         strength: 1.0
22     curiosity:
23         gamma: 0.99
24         strength: 0.02
25         learning_rate: 3.0e-4
26 keep_checkpoints: 10
27 max_steps: 5000000
28 time_horizon: 128
29 summary_freq: 20000

```

Listing 5: ctf_poca.yaml

Phụ lục D. Mã nguồn minh họa

D.1. Không gian quan sát (Cross The Road):

```

1 public override void CollectObservations(VectorSensor sensor)
2 {
3     // 3 gia tri: toa do x, y, z cua agent
4     sensor.AddObservation(transform.localPosition);
5     // 3 gia tri: toa do x, y, z cua dich
6     sensor.AddObservation(goal.transform.localPosition);
7 }

```

Listing 6: CollectObservations - CrossTheRoadAgent.cs

D.2. Hàm phần thưởng (Cross The Road):

```

1 public void GivePoints() // Khi den dich
2 {
3     AddReward(1.0f);
4     EndEpisode();
5 }
6
7 public void TakeAwayPoints() // Khi va cham phuong tien
8 {
9     AddReward(-0.025f);
10    EndEpisode();
11 }

```

Listing 7: GivePoints / TakeAwayPoints - CrossTheRoadAgent.cs

D.3. Phần thưởng nhóm (môi trường Capture The Flag):

```

1 // Ca hai agent cung roi phong -> thuong nhom
2 if (agents[0].agent.ThisAgentLeft && agents[1].agent.ThisAgentLeft)
3     agentGroup.AddGroupReward(0.5f / MaxEnvironmentSteps);
4
5 // Phat thoi gian (hurry-up penalty)

```

```

6 agentGroup.AddGroupReward(-0.25f / MaxEnvironmentSteps);
7
8 // Ca hai den checkpoint -> thuong lon + ket thuc episode
9 if (allFound) {
10     agentGroup.AddGroupReward(reward);
11     agentGroup.EndGroupEpisode();
12 }

```

Listing 8: Group reward - EnvController.cs (trich doan)

D.4. Phần thưởng định hướng (Football Table):

```

1 void AddShotReward()
2 {
3     if (Stats.HasBall) {
4         Vector3 delta = opponentTeam.Goal.transform.position
5                         - ball.transform.position;
6         float reward = shotRewardMultiplier *
7                       Vector3.Dot(delta.normalized, ball.Velocity);
8         AddReward(reward);
9     }
10 }

```

Listing 9: AddShotReward - FootballAgent.cs

Phụ lục E. Kết quả TensorBoard

Toàn bộ số liệu huấn luyện (Cumulative Reward, Episode Length, Policy Loss, Entropy, ELO...) đều được ghi lại tự động qua TensorBoard cho từng lần chạy và lưu trong thư mục `results/` của dự án. Bảng 6.1 tóm tắt các chỉ số cuối cùng thực đo được của những lần chạy đã huấn luyện đầy đủ.

Bảng 6.1: Tóm tắt chỉ số cuối thực đo từ TensorBoard

Lần chạy	Thuật toán	Số bước	Reward/ELO	Thời gian
FB01 (Football)	PPO	1,59M	ELO 1242	~109 phút
ctr01 (CrossRoad)	PPO	2M	0,703	~114 phút
CrossRoad_SAC	SAC	200K	0,818	~104 phút
ctr_poca (CrossRoad)	MA-POCA	2M	0,655	~67 phút
ctr_mappo (CrossRoad)	MAPPO	2M	0,567	~75 phút
ctf_poca (Capture)	MA-POCA	1,68M	0,534	~626 phút
ctf_mappo (Capture)	MAPPO	1,52M	0,795	~499 phút

Mã nguồn và mô hình: Toàn bộ mã nguồn dự án Unity, các tệp cấu hình, script huấn luyện và các mô hình đã huấn luyện (định dạng .onnx) được lưu trữ tại kho mã nguồn của đề tài.